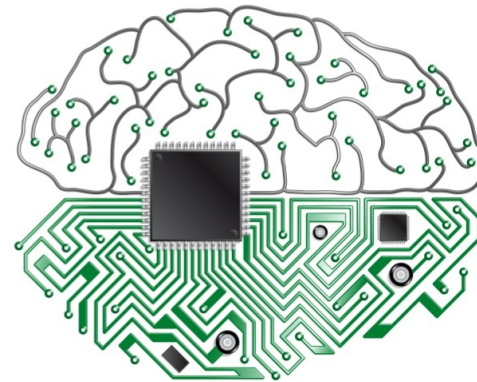
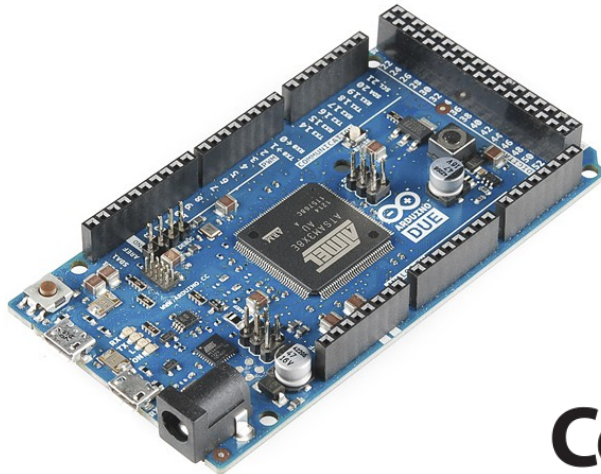
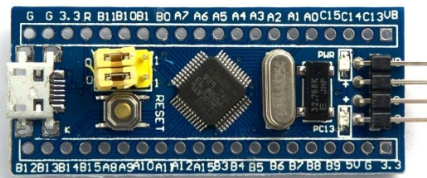


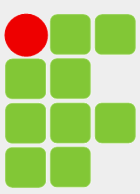
ARM Cortex-M3/M4



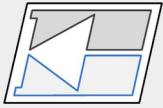
Cortex
Intelligent Processors by ARM®



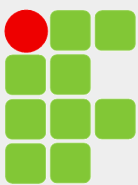
Prof. Charles Borges de Lima.



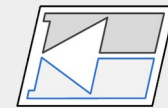
SUMÁRIO



- Introdução
- Família ARM
- Arquitetura
- Memória
- Suporte a Sistemas Operacionais
- Interrupções e Exceções
- Economia de Energia
- Instruções
- Conclusão



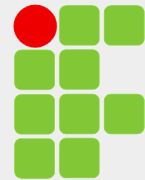
INTRODUÇÃO



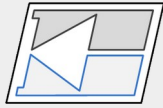
Atualmente a maioria dos novos μ controladores são baseados na arquitetura ARM, especificamente nos populares Cortex-M3, M4 e M0.

Essa tendência se deve ao eficiente desempenho, consumo energético e compacidade de código. Por isso, esses μ controladores de 32 bits estão substituindo aplicações antes dominadas por μ controladores de 8 e 16 bits.

Outras características que tornam os μ controladores ARM tão populares é a disponibilidade de uma infinidade de ferramentas de desenvolvimento, exemplos de projetos e fabricantes de chip.



Vantagens dos processadores Cortex-M



Low Power – vários modos de economia de energia e configurações de consumo.

Desempenho – pipeline com eficiente CPI (*cycles per instruction*). Instruções eficientes.

Eficiência energética – uma tarefa pode ser feita rapidamente e o sistema pode entrar em modo de economia de energia.

Densidade de código – o modo Thumb produz uma compactação de código. Permitindo aplicações com menores memórias, diminuindo o consumo energético.

Interrupções – possui um controlador de interrupções configurável que permite até 240 interrupções com diferentes níveis de prioridade e aninhadas, com latência de somente 12 ciclos de relógio.

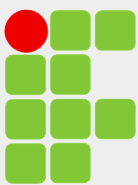
C friendly – quase tudo pode ser programado em C.

Scalabilidade, portabilidade e reuso – facilidade de uso e reuso do código para diferentes μ controladores ARM em diferentes projetos.

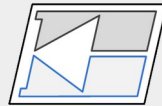
Várias funcionalidades - para depuração e ferramentas auxiliares, bem como bibliotecas de funções.

Suporte a Sistemas Operacionais – hardware projetado para empregar OS.

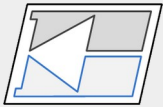
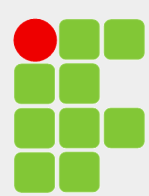
Versatilidade – MPU, FPU, instruções DSP, bit band.



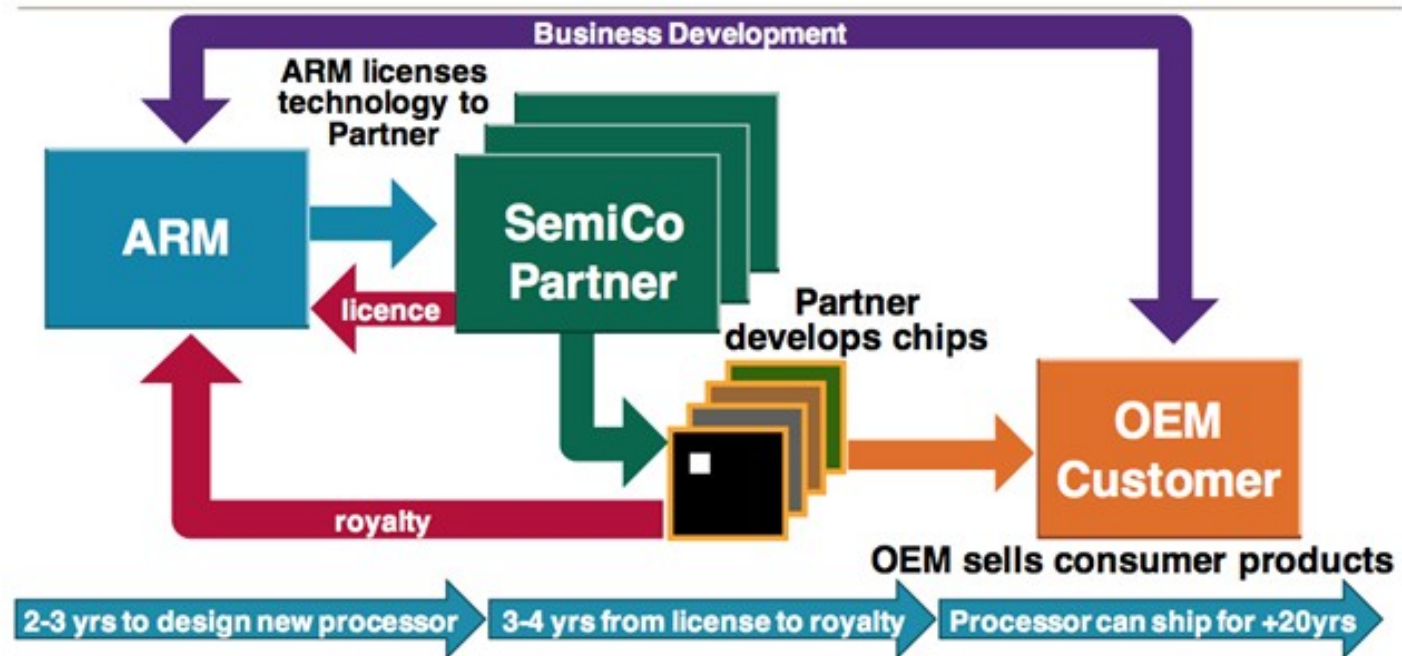
ARM Ltda



- Fundada em Novembro de 1990, como uma joint-venture entre a Acorn Computers, Apple Computer e VLSI Technology, com a intenção de aprimorar o desenvolvimento do μ processador RISC (inicialmente conhecido como Acorn RISC Machine).
- Projeta uma série de núcleos ARM com processadores RISC.
- Licencia núcleos ARM projetados para associados, que fabricam e vendem para seus clientes (não fabrica o chip).
- Também desenvolve tecnologias para auxiliar no projeto com a arquitetura ARM: ferramentas de software, GPUs, placas, hardware de depuração, software de aplicação, arquiteturas de barramento, periféricos, etc.

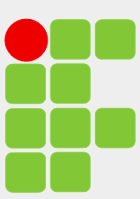


ARM Business Model

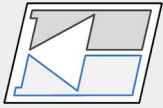


■ Innovative business model yields high margins

- Upfront license fee – flexible licensing models
- Ongoing royalties – typically based on percentage of chip price
- Technology suitable for multiple applications – can ship for decades



Produtos



CPU e NPUs

Processor IP for the widest range of devices—from sensors to servers.

Multimídia

Mali processors offer a complete multimedia solution for SoC.

Físico IP

Solutions for the efficient implementation of complex SoC designs.

Sistema IP

Perfect for high-efficiency IoT endpoints or a high-performance server SoC.

Segurança

Security IP designed to protect against a variety of different vulnerabilities.

Subsistemas

Corstone accelerates the development of secure SoCs for IoT and embedded devices.

Neoverse

Processors for HPC and cloud-to-edge infrastructure workloads and solutions.

Ethos

NPUs with enhanced processing capabilities to deliver highest performance for machine learning inference.

Cortex-A

Applications processors for devices undertaking complex compute tasks.

Cortex-R

Real-time processors offering fast, reliable performance for time-critical systems.

Cortex-M

Low-power processors for microcontrollers and constrained, energy-efficient applications.

Mali GPUs

Graphics processors for a range of mobile devices from smartwatches to autonomous vehicles.

Development Tools and Software

- SoC Design
- Embedded & IoT Software Tools
- Server & HPC Application Tools
- Graphics Development Tools
- Development Boards

Mbed OS

Open-source IoT operating system for building intelligent connected products.

Arm Architecture

Arm Architecture enables our partners to build their products in an efficient, affordable, and secure way. The new Armv9 architecture delivers greater performance, enhanced security and DSP and ML capabilities.

Arm Flexible Access

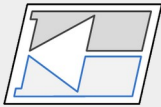
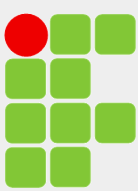
Arm Flexible Access provides quick, easy, and unlimited access to a wide range of IP, tools and support to evaluate and fully design solutions.

Downloads

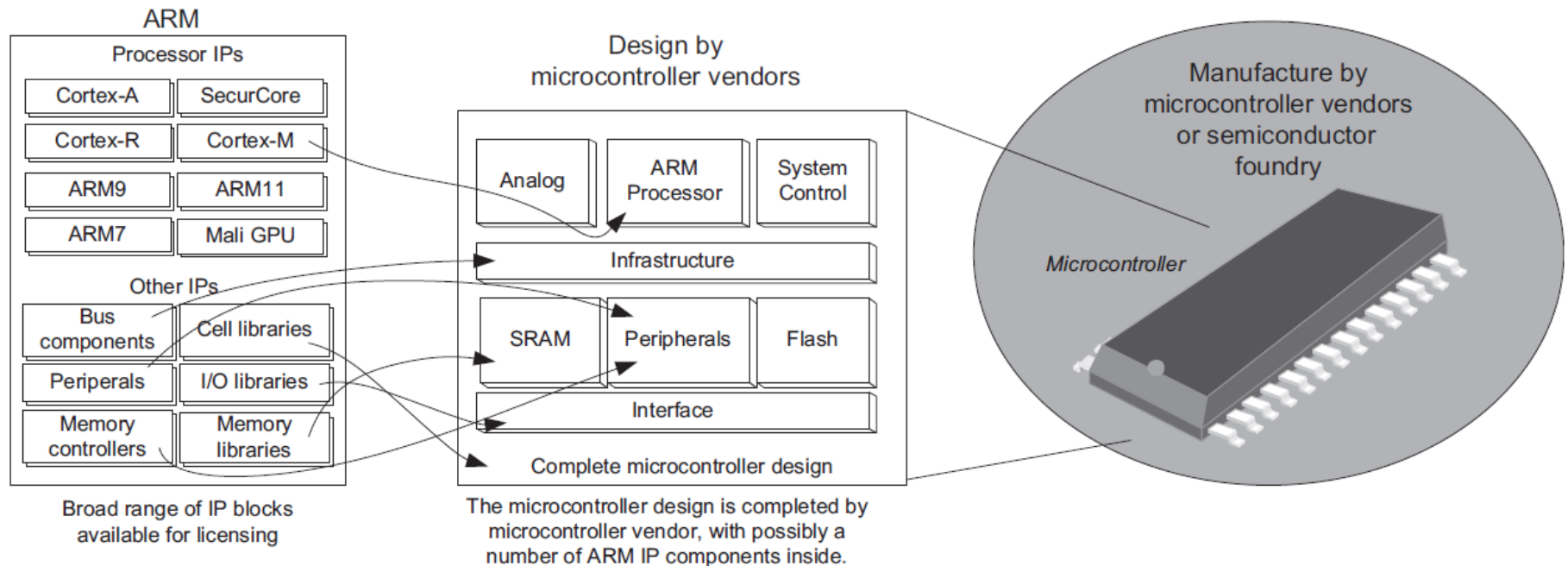
Search, browse, and download Arm products, software, and tools.

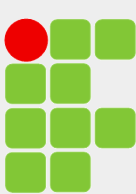
Documentation

Get technical manuals and documentation for Arm products.

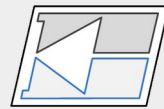


Um μ controlador pode conter múltiplos produtos da ARM





Parcerias



ATAP Partners

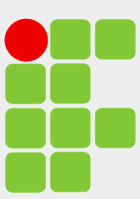
Tools Partners

RTOS Partners

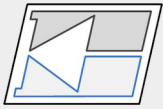
Software Partners

ARM in Partnership

Logos include: DNP, Infinite Technology Corporation, SIEMENS, NSW, STEP MIND, Think, MacroTech, COMIT, YOGITECH, SIDSA, WIPRO, SEODU INCHIF, TOPPAN, HOYA, INICORE, FARADAY, MAZ, SCIWORX, nordic, TALITY, parthus, synopsys, FLETRONICS, FIRMWARE SYSTEMS, realogy, eSOL, jmi, QNX, LINEO, Accelerated Technology Inc, ACCESS, GEOWORKS, KADAK Products Ltd, Microsoft, FIRMWORKS, WindRiver, MICROWARE, AXE, EONIC, symbian, ETNOTEAM, LYNEXWORKS, CMX COMPANY, Embedded System Products, ASAHI CHEMICAL INDUSTRY CO.,LTD., ONX, Dolby, Symmetricom, Heartbeat of the Net, CPS, JAVA, liquid audio, ZI corporation, ERICSSON, INTERTRUST, EMBLAZE, Packet Video, FRONTIER, Microsoft, interniche technologies, Inc., ZEEVO, intel, Virata, SONY, Triscend, ALTERA, Infineon, ZARLINK, RESONEXT, National Semiconductor, KANASAKI MICRO, PRAIRIE COMM, COGENANT, YAMAHA, OAK, SHARP, BROADCOM, HYUNDAI, agere, CIRBUS LOGIC, ALCATEL, SANYO, EPSON, MICRONAS, PHILIPS, TOSHIBA, FUJITSU, OKI, MITSUBISHI ELECTRIC, QUALCOMM, ZTEIC, GOODRICH, ERICSSON, ADMtek, NEC, LANBOND, AMI, UMC, Chartered, ANALOG DEVICES, AKM, TEXAS INSTRUMENTS, TIKOS, COGENT, ALIANT, SYNOPSIS, Mentor Graphics, cadence, LAUTERBACH, Agilent Technologies, Verisity, Aptix, Sophia systems, WindRiver, Tektronix, ADS, YOKOGAWA, virtio, Green Hills, inNOVEDA, Computex, CoWare, ASHLING, EPI.



Produtos Equipados com ARM



Tele-parking



Intelligent toys



Utility Meters



IR Fire Detector



Exercise Machines



Energy Efficient Appliances



Intelligent Vending

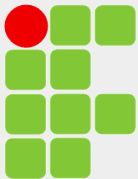


intelbras

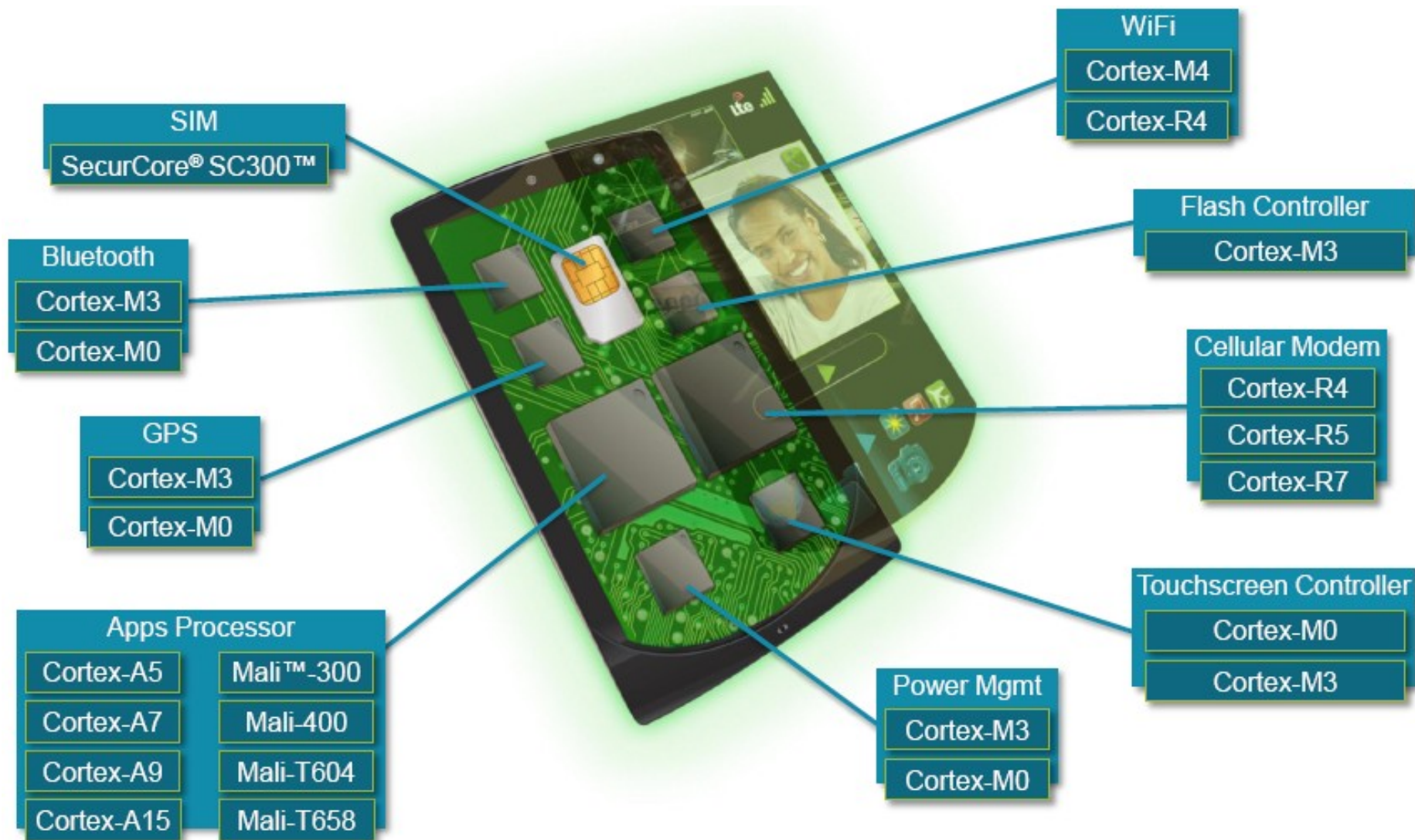
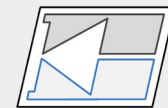


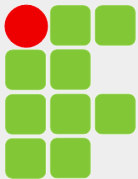
NETFLIX



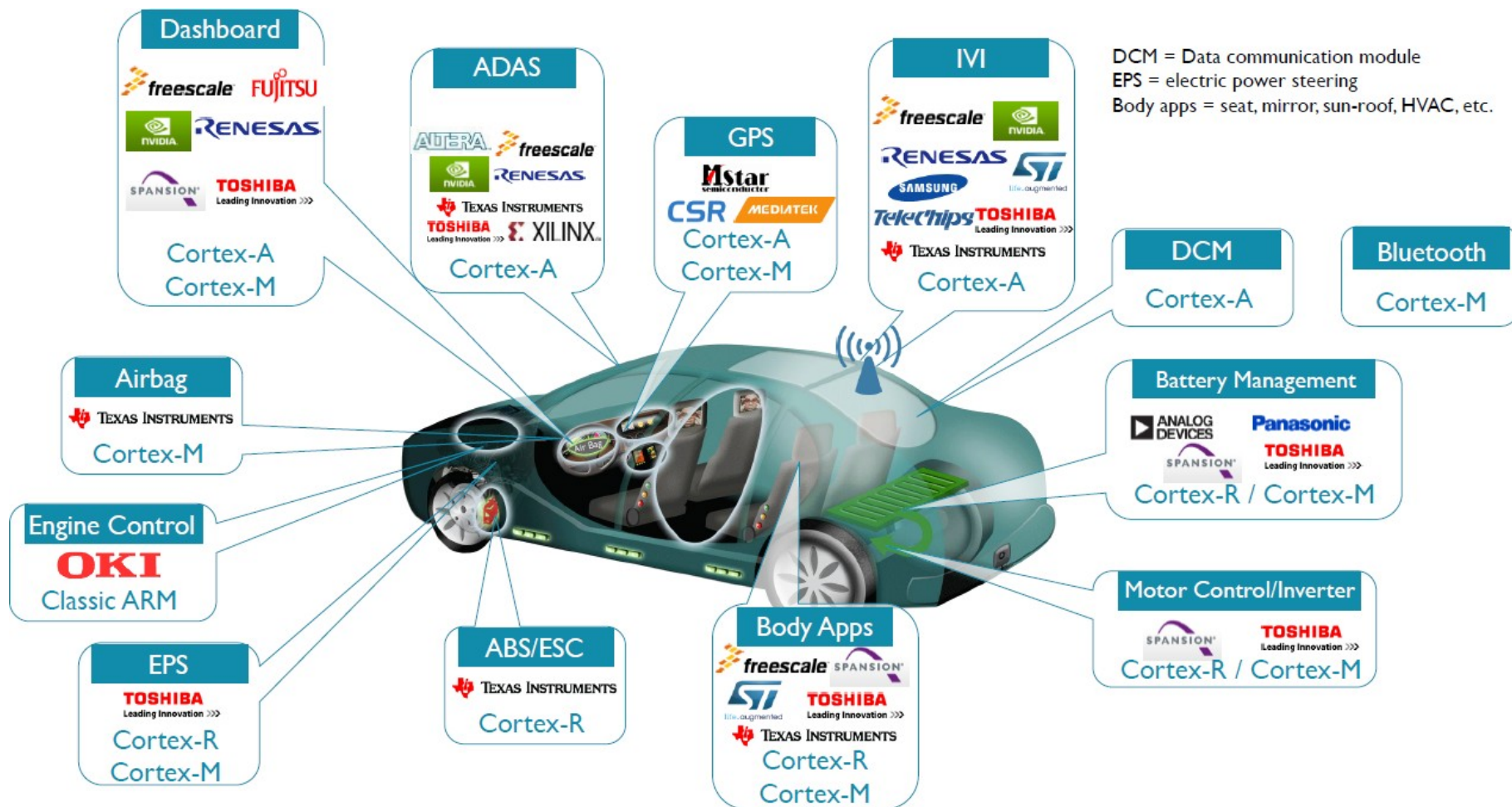
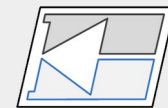


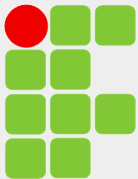
Produtos Equipados com ARM



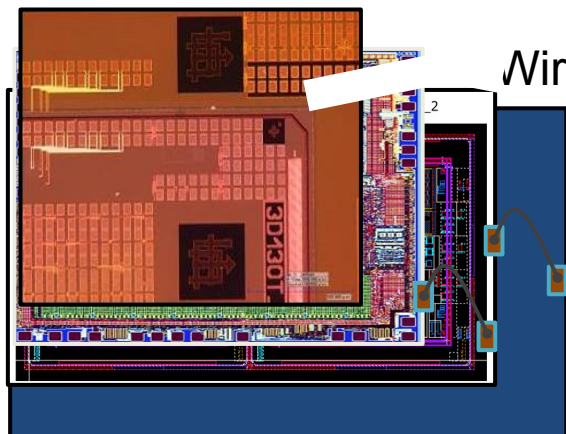
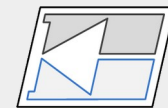


Produtos Equipados com ARM





O menor computador ARM



Wireless Sensor Network

Sensors, timers

Cortex-M0 +16KB RAM 65nm
UWB Radio antenna

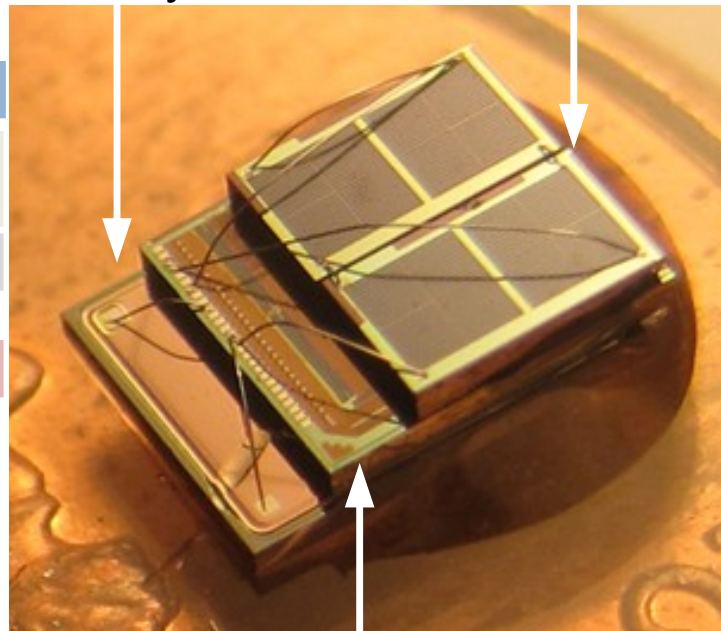
10 kB Storage memory ~3fW/bit

12μAh Li-ion Battery



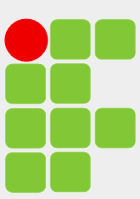
Battery

Solar Cells

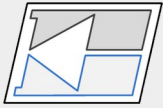


Processor, SRAM and PMU

Wirelessly networked into large scale
sensor arrays



O maior computador ARM



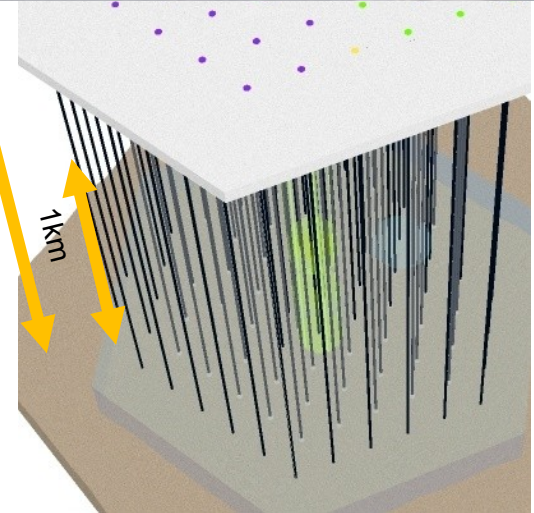
4200 ARM powered
Neutrino Detectors



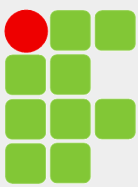
70 bore holes 2.5km deep

60 detectors per string
starting 1.5km down

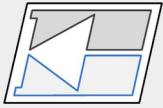
1km³ of active telescope



Work supported by the National Science Foundation and University of Wisconsin-Madison



Super Computador



Fujitsu A64FX: Arm-powered Heart of World's Fastest Supercomputer

The Fujitsu A64FX is one of the most powerful Arm-based processors in the world, and the world's fastest supercomputer features over 150,000 of them.

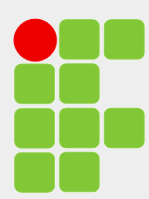


Posted on 10th July 2020
By **Arm Blueprint staff**

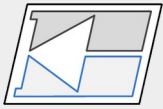
High-Performance Computing (HPC)

Reading Time: 5 mins

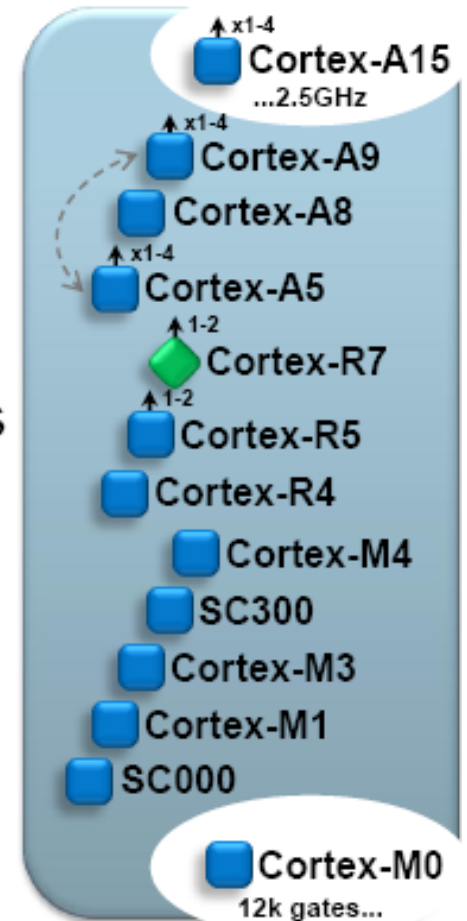


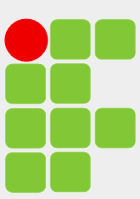


Família ARM Cortex

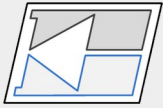


- ARM Cortex-**A** family:
 - Applications processors
 - Targeted for OS's, graphics, demanding tasks
- ARM Cortex-**R** family:
 - Embedded processors
 - Real-time signal processing, control applications
- ARM Cortex-**M** family:
 - Microcontroller-oriented processors
 - MCU, ASSP, and SoC applications

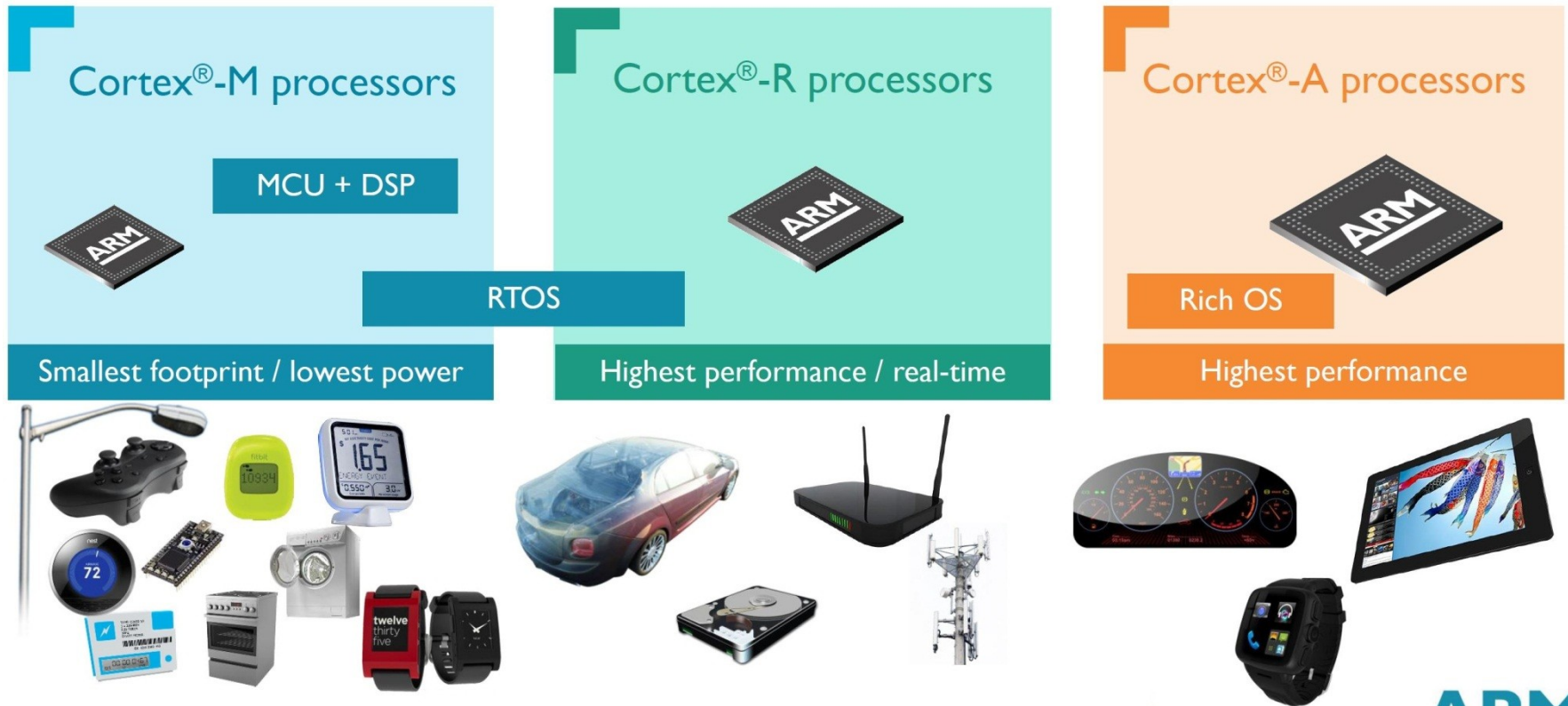


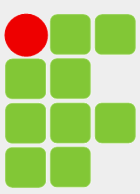


Família ARM Cortex

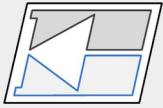


ARM® Cortex® Processors across the Embedded Market





Família ARM Cortex



Cortex - A

Highest performance

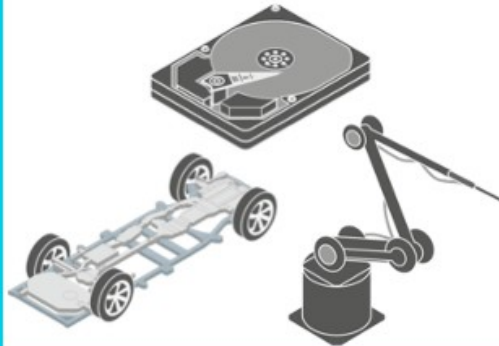
Optimised for
rich operating systems



Cortex - R

Fast response

Optimised for
high performance,
hard real-time applications

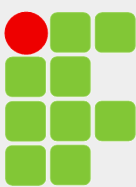


Cortex - M

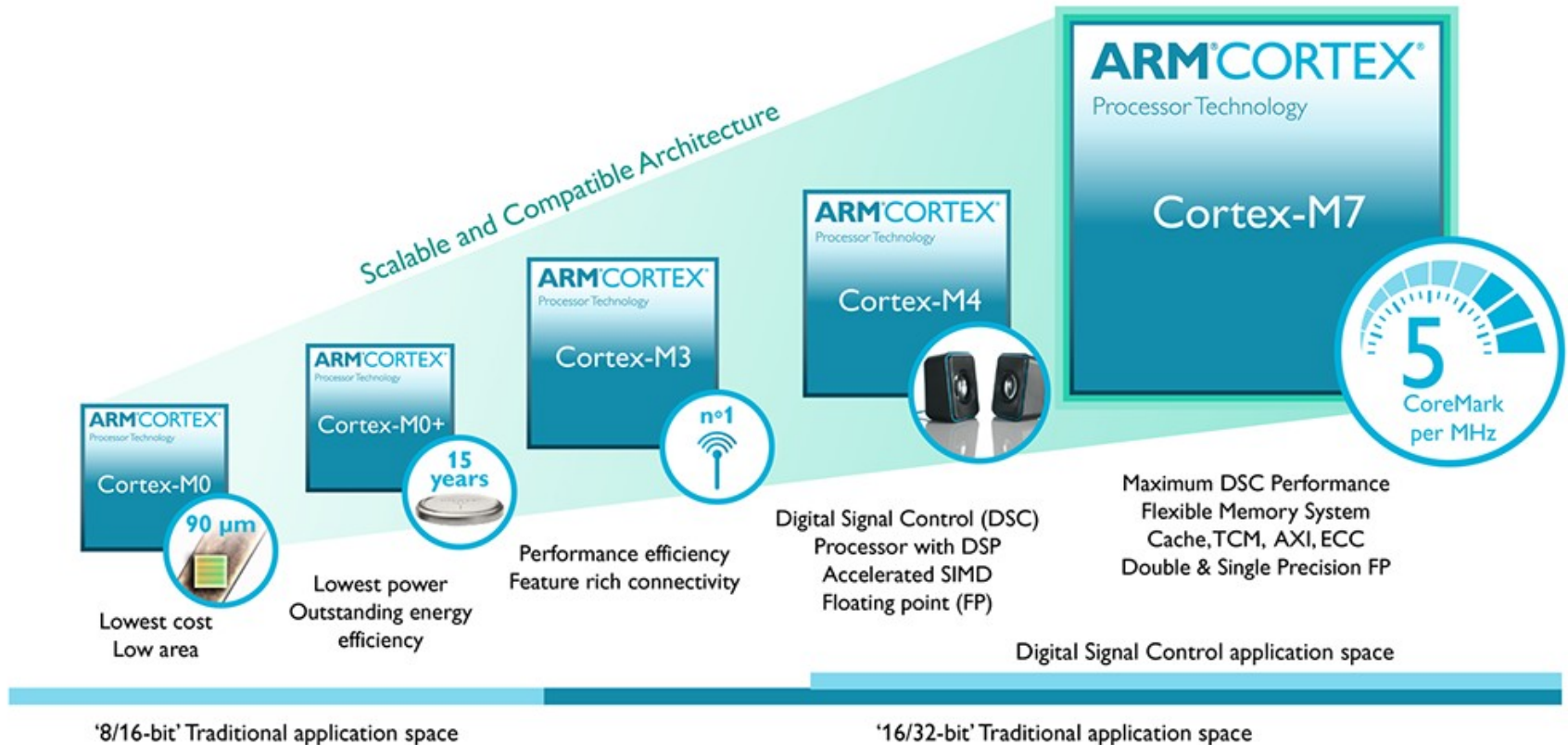
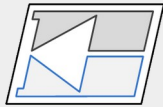
Smallest/lowest power

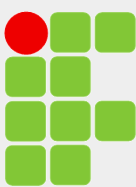
Optimised for
discrete processing and
microcontrollers



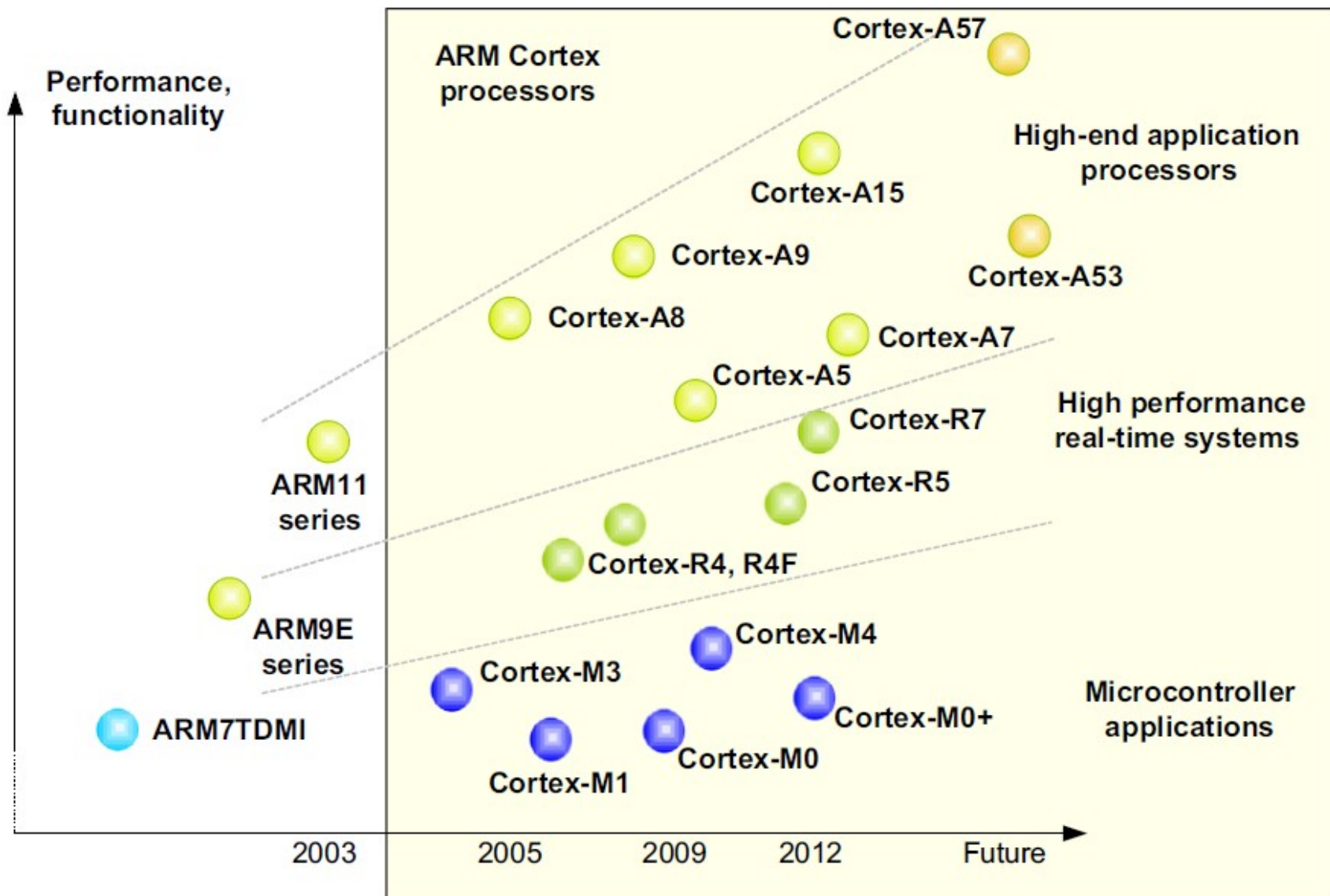
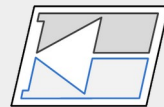


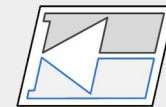
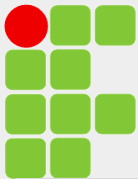
ARM Cortex



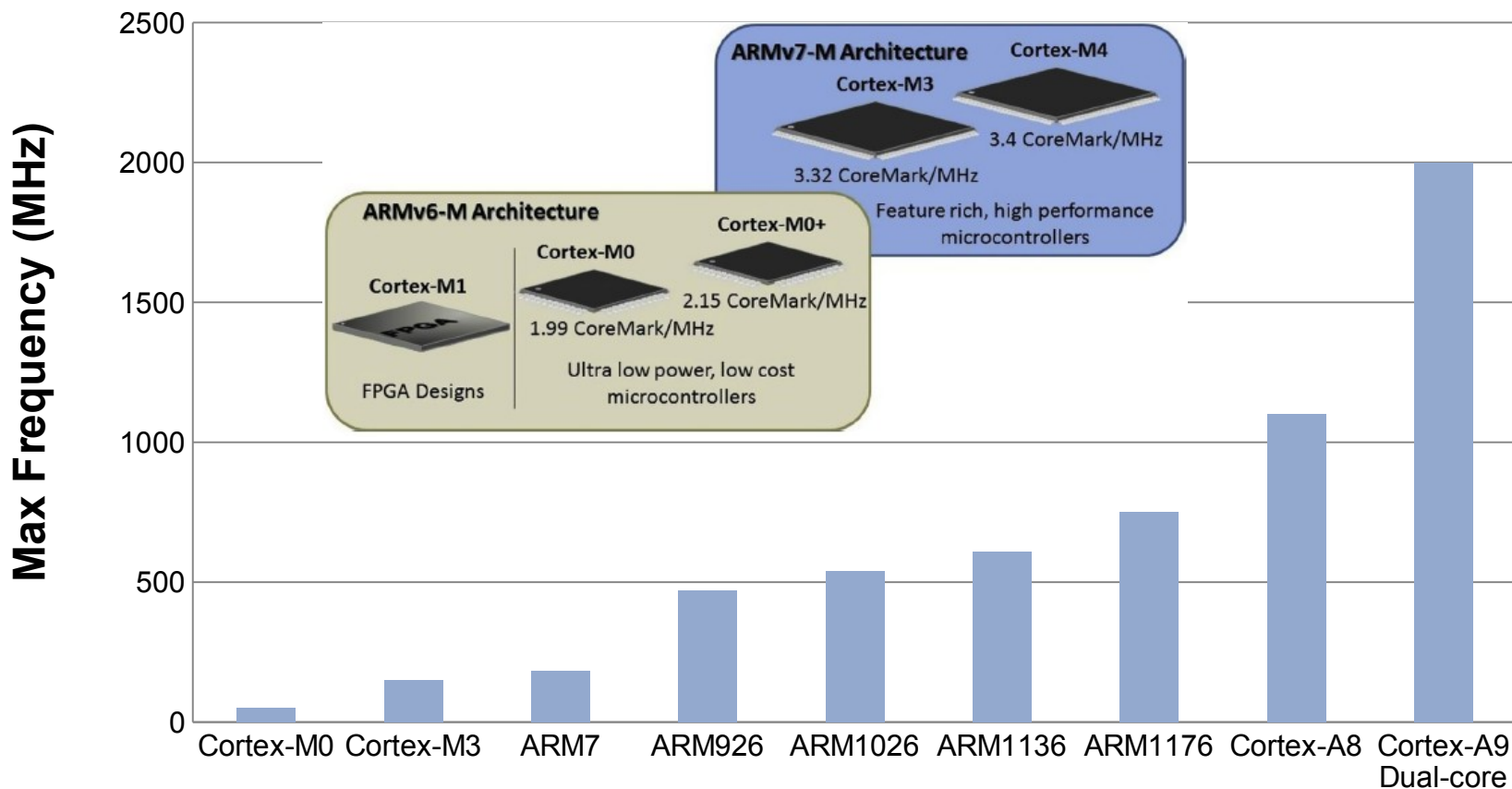


Família ARM

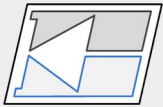
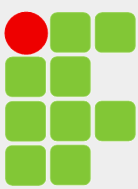




Desempenho Relativo



*Represents attainable speeds in 130, 90, 65, or 45nm processes



Desempenho Relativo

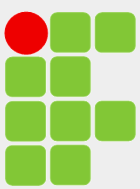
	90LP (7-track, typical 1.2v, 25C)		40G (9-track, typical 0.9v, 25C)	
	Dynamic power (μ W/MHz)	Area mm ²	Dynamic power (μ W/MHz)	Area mm ²
Cortex-M0	16	0.04	4	0.01
Cortex-M0+	9.8	0.035	3	0.009
Cortex-M3	32	0.12	7	0.03
Cortex-M4	33	0.17	8	0.04

- Static power <0.7 μ W/MHz

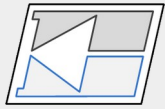
CoreMark[®] and Dhrystone

Dhrystone (official)	Dhrystone (max options)	CoreMark
DMIPS/MHz	DMIPS/MHz	CoreMark/MHz
0.84	1.21	2.33
0.94	1.31	2.42
1.25	1.89	3.32
1.25	1.95	3.40

* CoreMark data from ARM website & CoreMark.org website



Evolução da Arquitetura ARM



Architecture v4 / v4T

ARM7TDMI,
920T,
Intel StrongARM

Architecture v5 / v5E

ARM926, 946,
966,
Intel XScale

Architecture v6

ARM1136, 1176,
1156T-2

Architecture v6-M

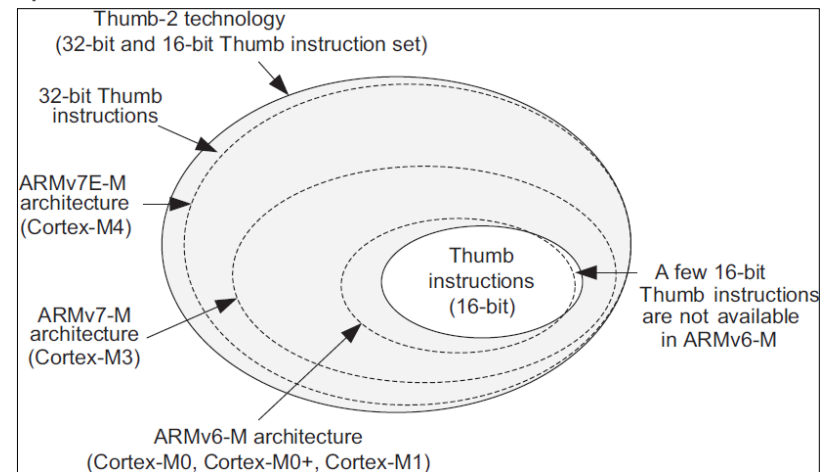
Cortex-M0+,
Cortex-M0
Cortex-M1 (FPGA)

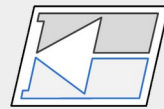
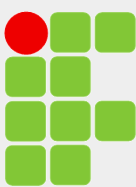
Architecture v7

v7-A (Application)
Cortex-A8, Cortex-A9,
Cortex-A15

v7-R (Real-Time)
Cortex-R4, Cortex-R5,
Cortex-R7

v7-M (Microcontroller)
Cortex-M3, Cortex-M4





Evolução da Arquitetura ARM

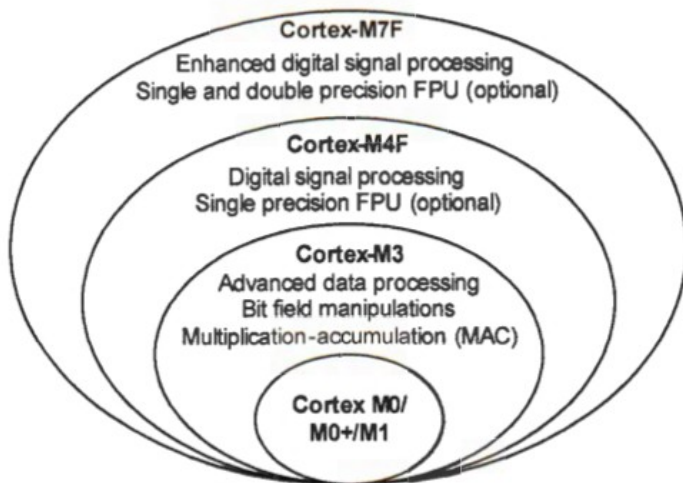


Figure 3-2. ARM Cortex-M family

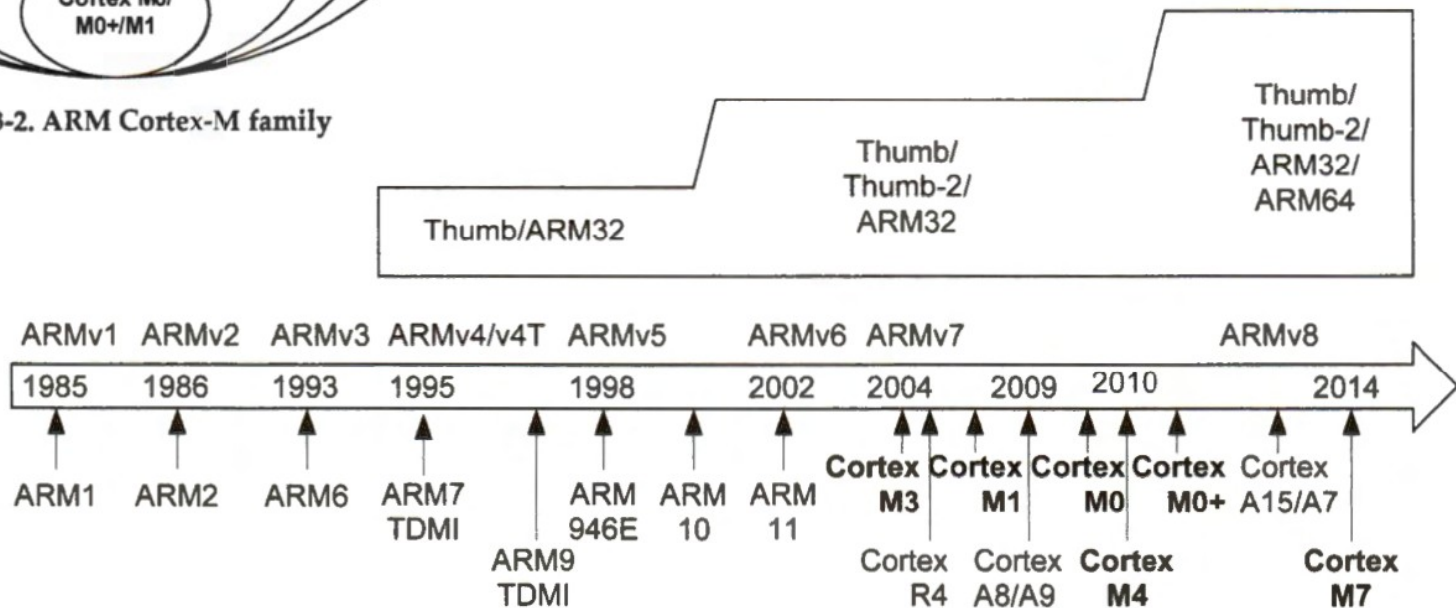
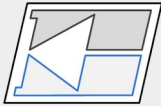
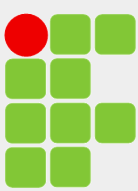


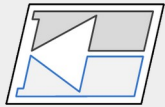
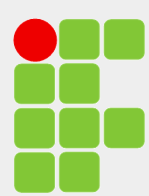
Figure 3-1. History of ARM architecture and instruction sets



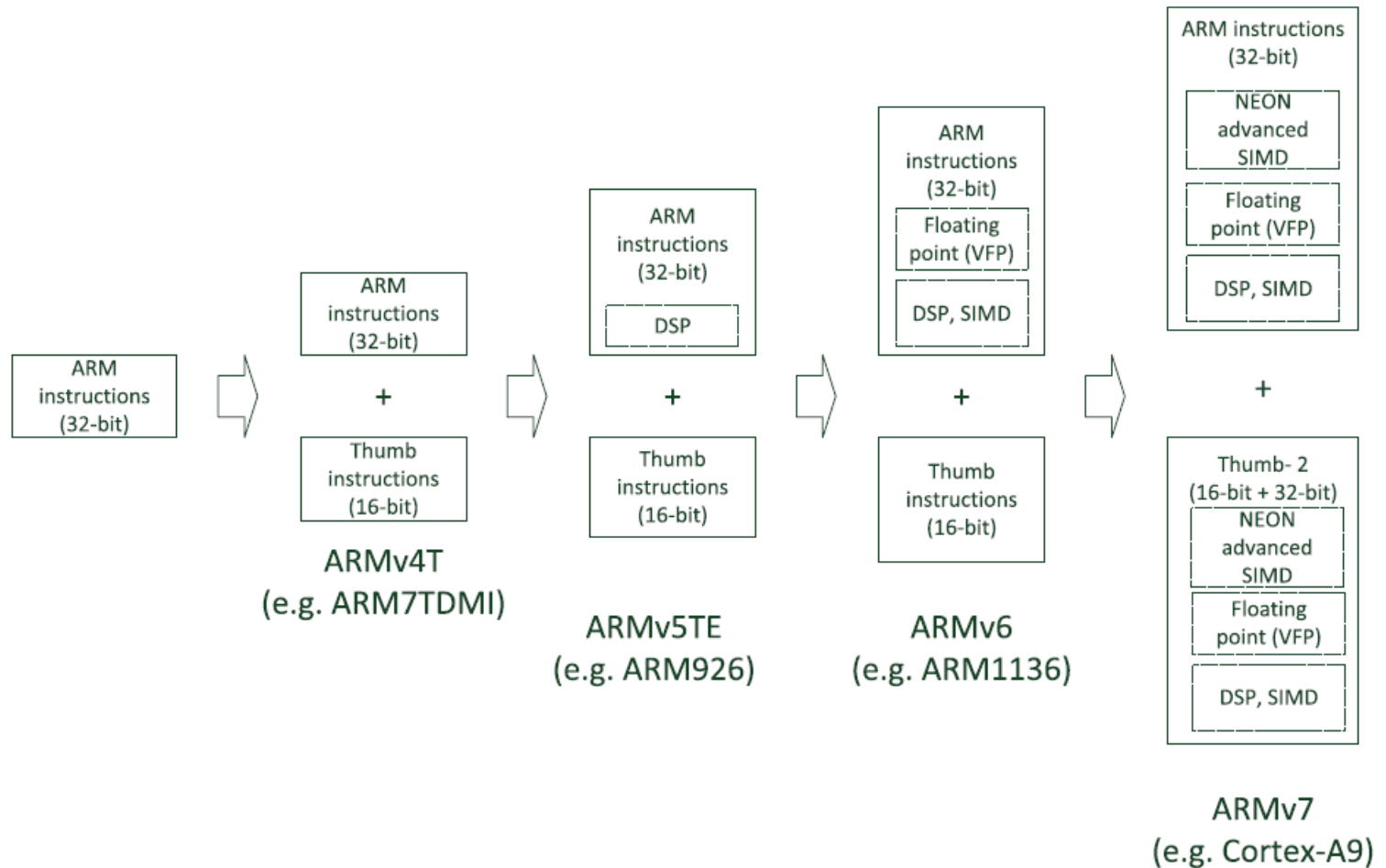
Thumb / ARM32 / ARM64

- **Thumb.** The objective of the Thumb instruction set is to improve the code density. Because an instruction in Thumb has only 16 bits in length, the size of their executable files is small. The space saving is achieved by reducing the possibilities of operands and limiting the number of registers that are accessible by an instruction. Reducing the size of instruction memory benefits many embedded systems demanding for low cost and long battery life.
- **ARM32.** Each instruction in ARM32 has 32 bits and provides more coding flexibility than a Thumb instruction. More operand options, more flexible memory addressing schemes, larger immediate numbers, and more addressable registers can be encoded in a 32-bit word. Furthermore, ARM32 instructions run faster than Thumb because an instruction can perform more operations or include more operands. However, the disadvantage is its code density.
- **Thumb-2.** It provides an outstanding compromise between ARM32 and Thumb. It optimizes the tradeoff between code density and processor performance. It consists of 16-bit Thumb instructions and a subset of 32-bit ARM32 instructions. The goal of Thumb-2 is to achieve higher code density like Thumb and fast performance comparable to ARM32.
- **ARM64.** ARM 64-bit processors are often used in desktops and servers. These processors have a set of 64-bit assembly instructions.

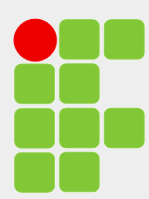
Thumb-2 optimizes the tradeoff between code density and application speed.



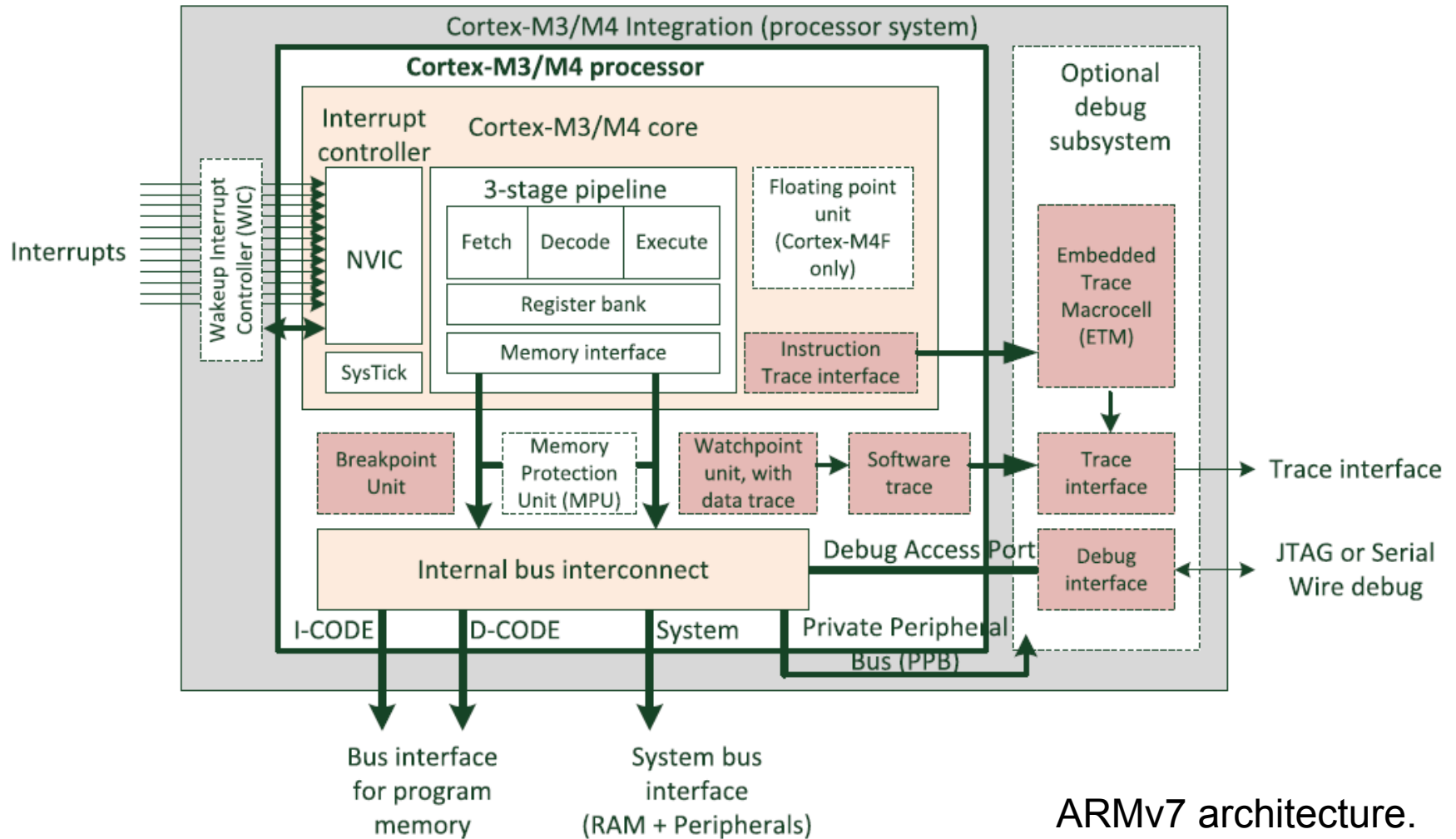
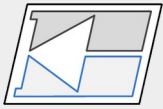
Evolução do ISA



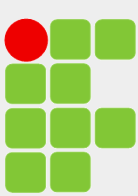
Evolution of the ARM Instruction Set Architecture



O Processador Cortex-M3/M4



ARMv7 architecture.



STM32L4

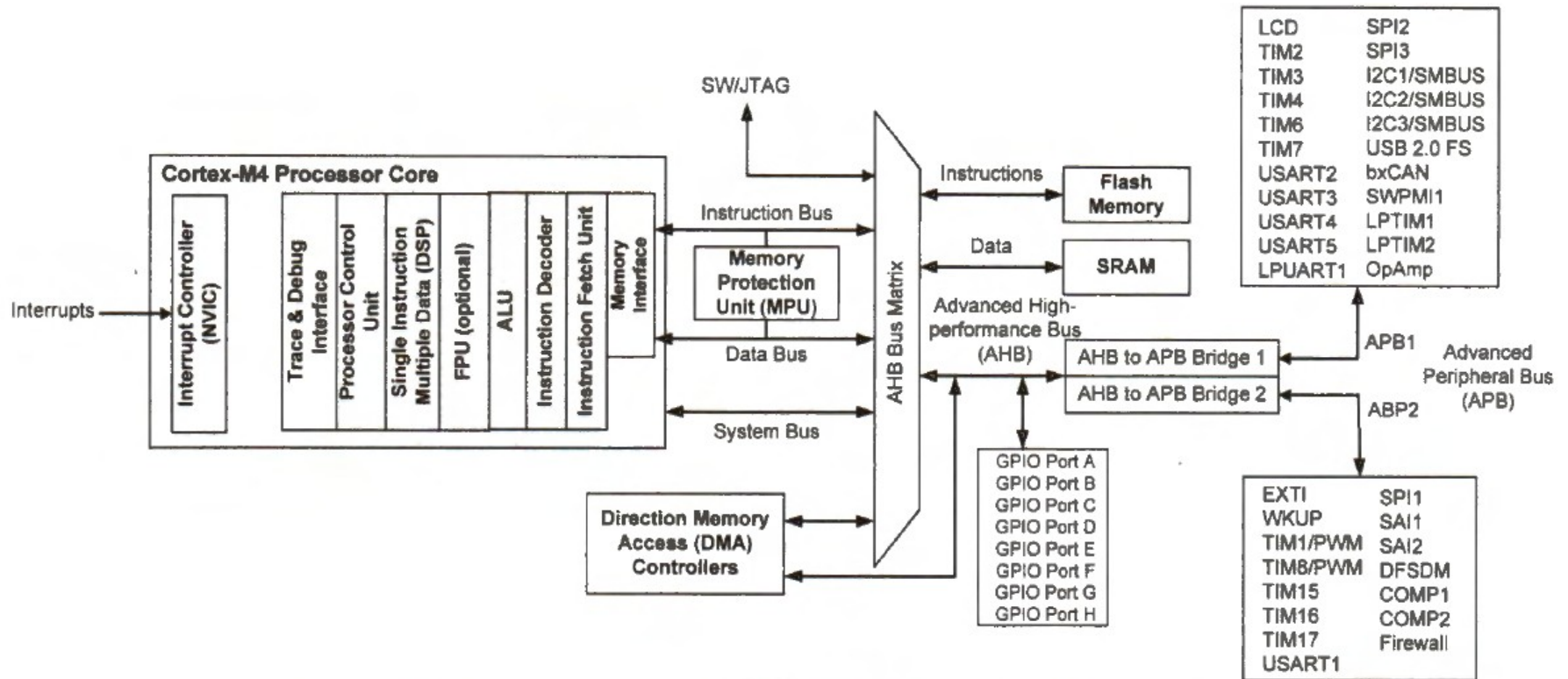
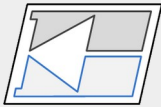


Figure 3-3. Organization of STM32L4 ARM Cortex-M4 processor

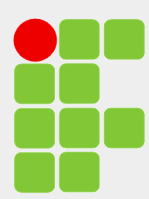
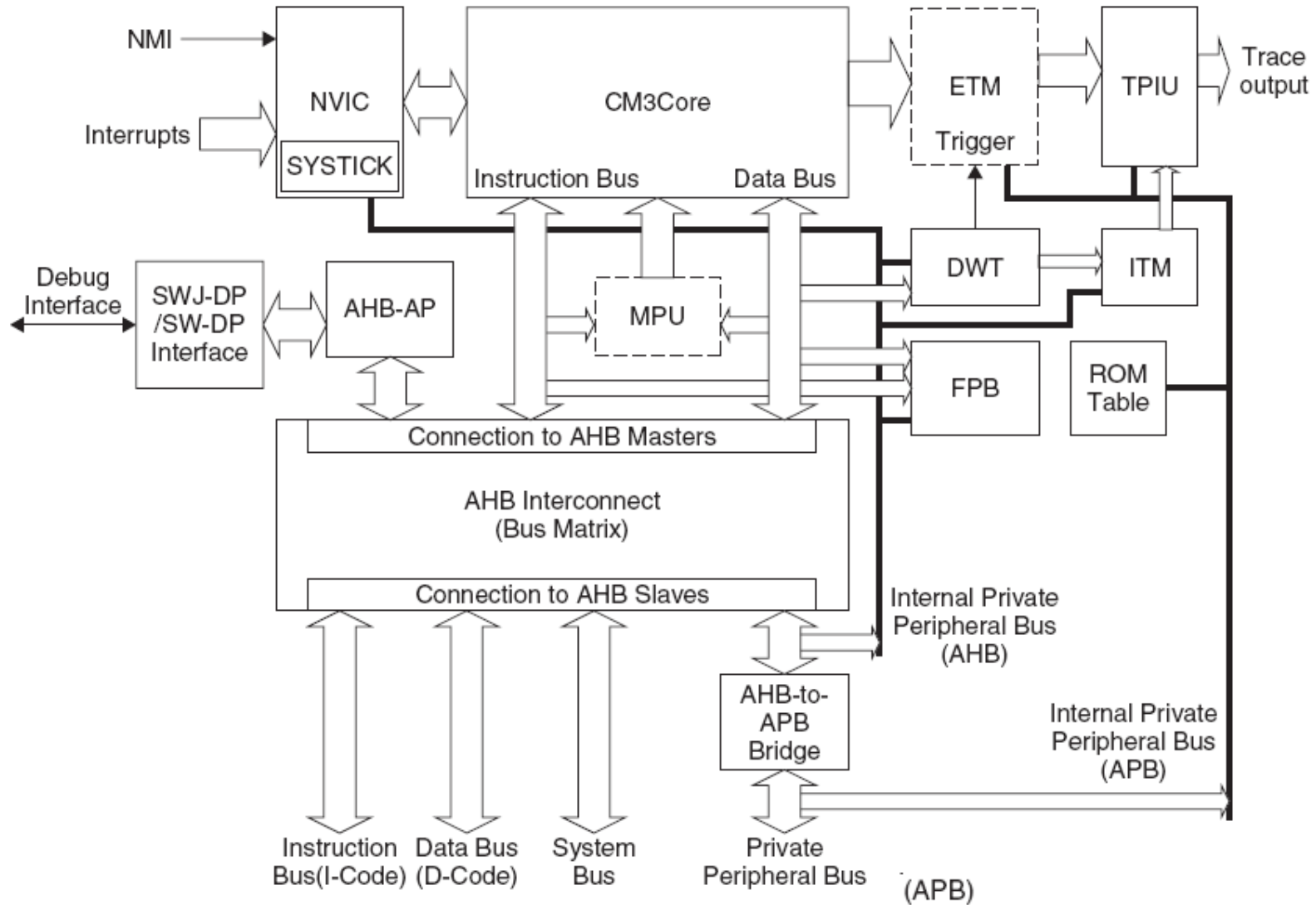
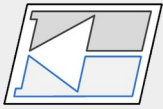
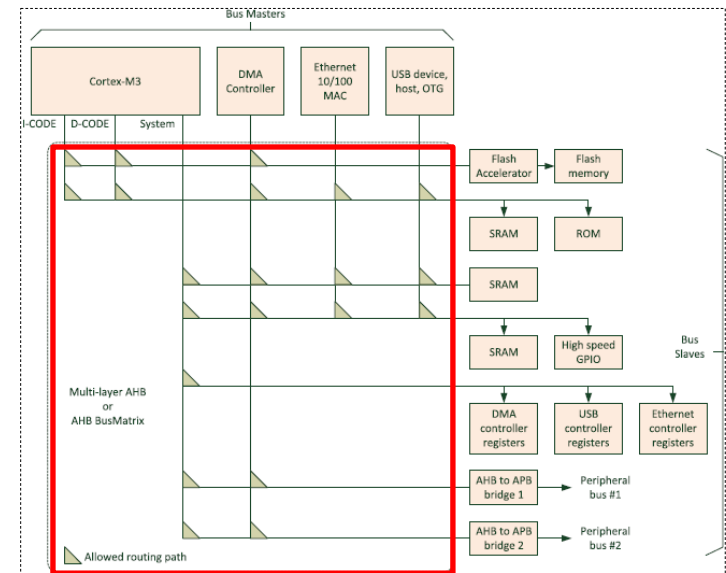
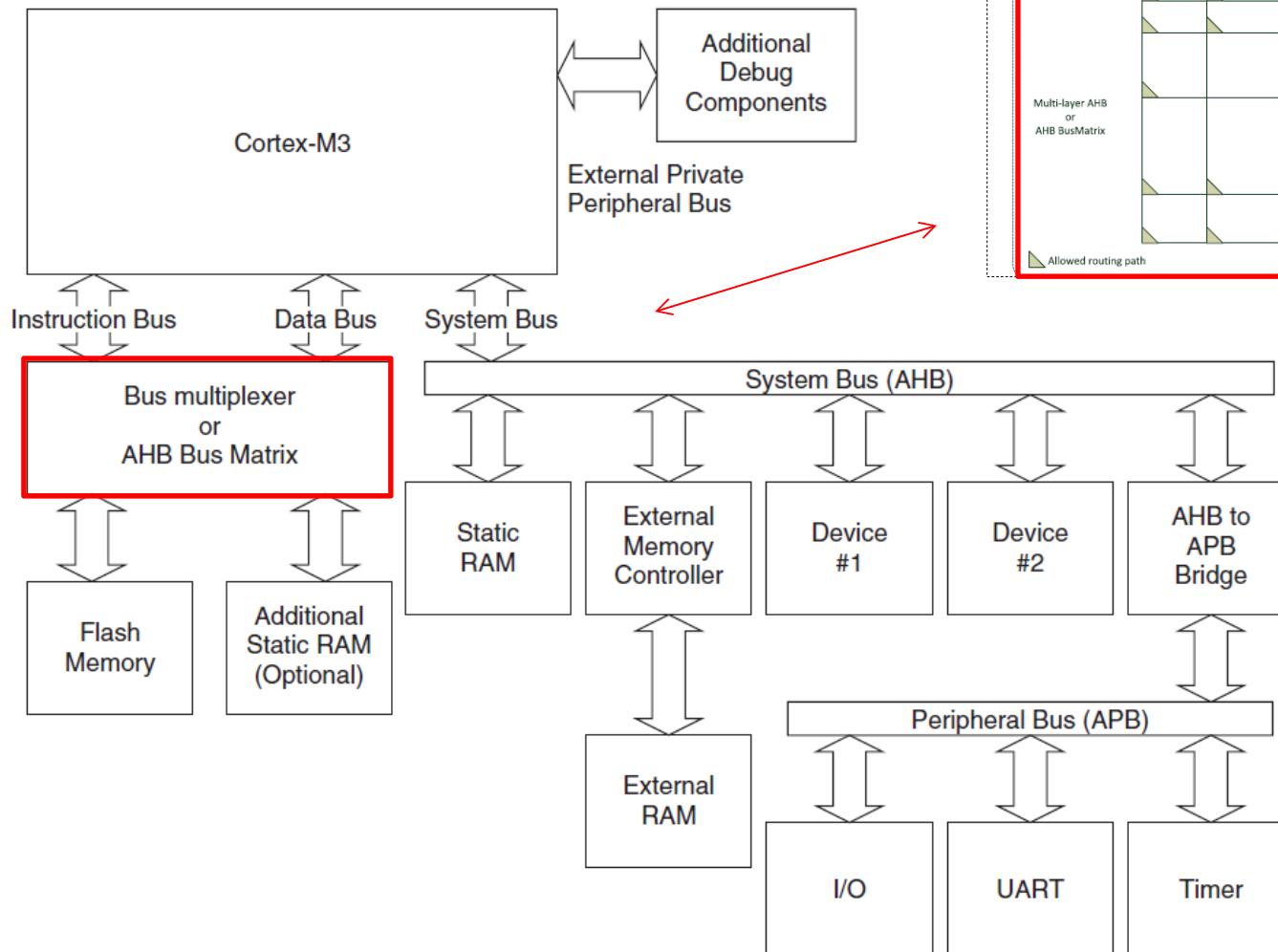
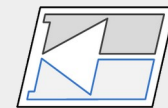
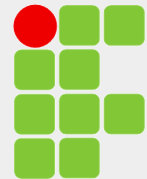


Diagrama de Blocos + detalhado

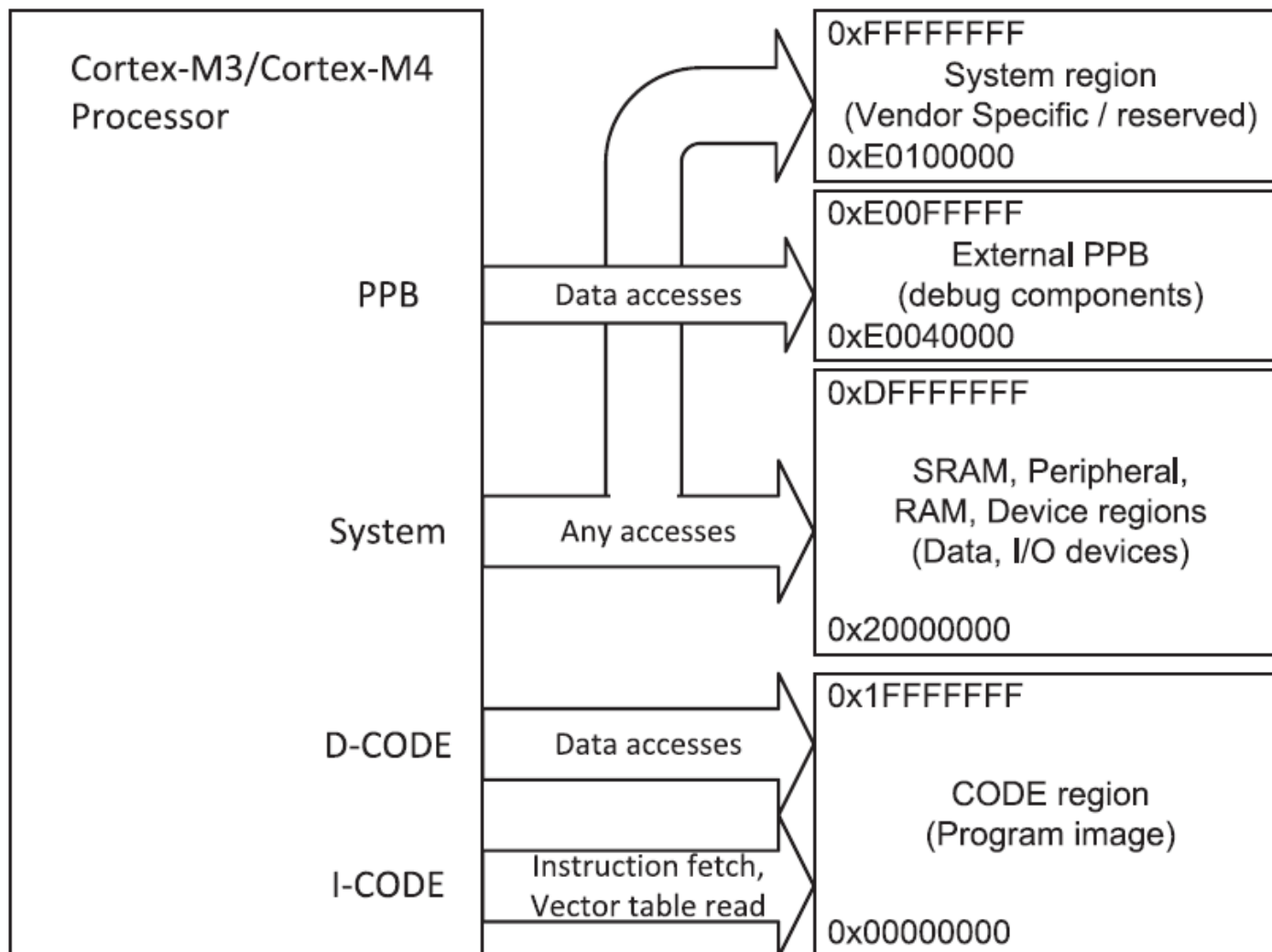


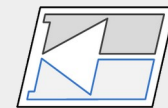
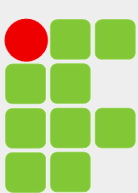
O sistema AMBA e a matriz de interconexão



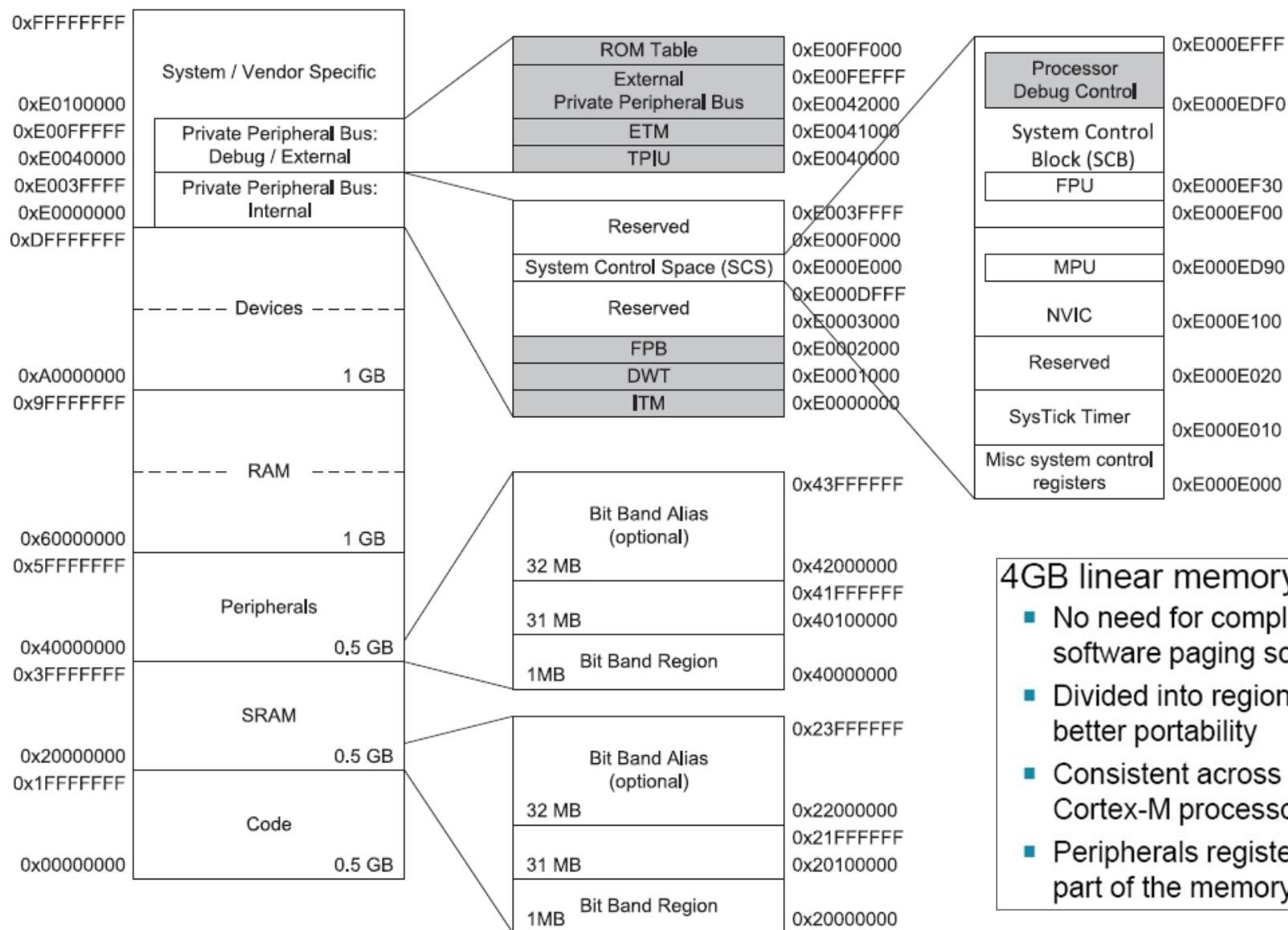


Mapa de Memória





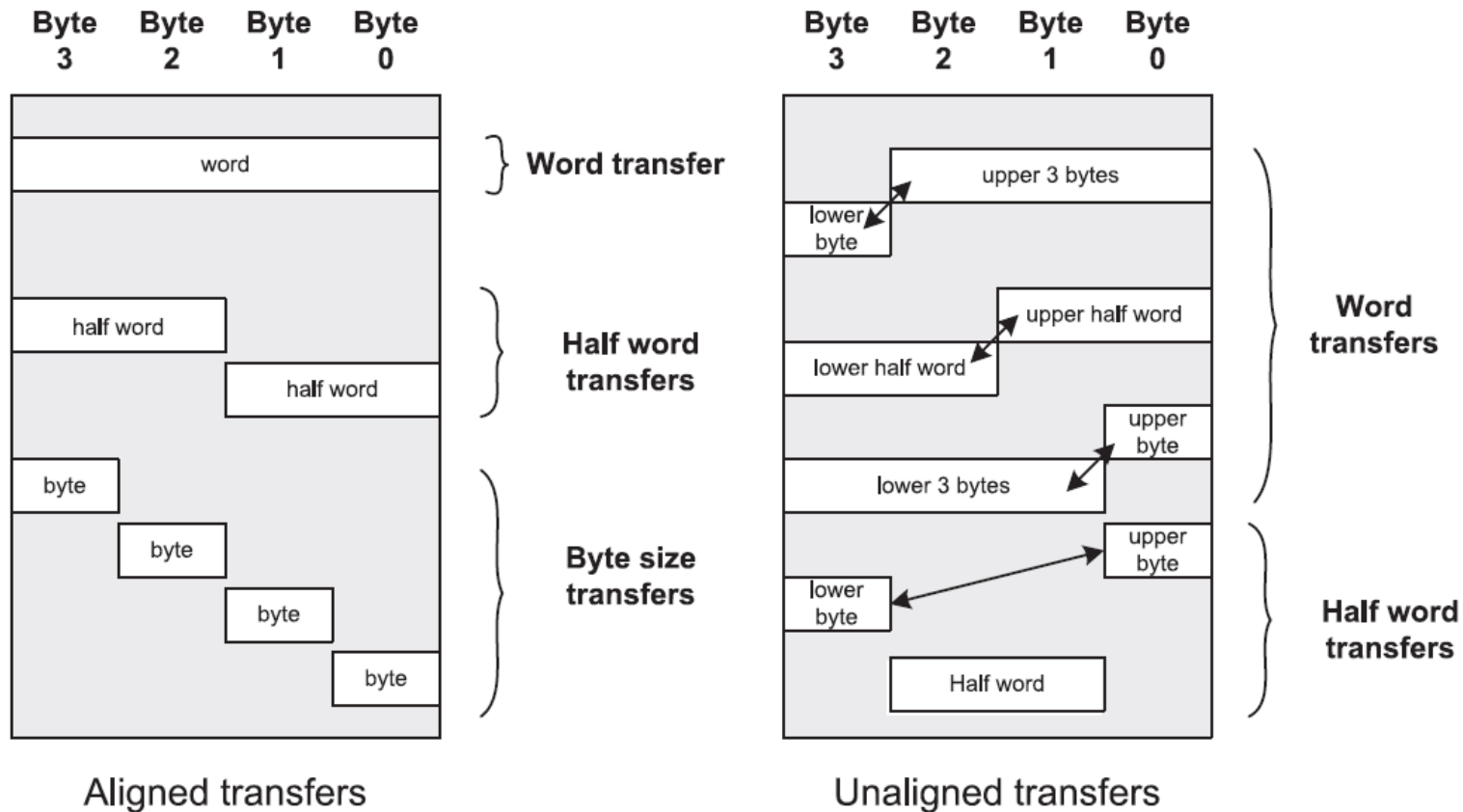
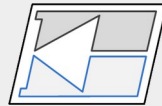
Mapa de Memória



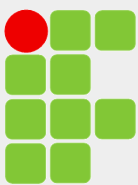
4GB linear memory space

- No need for complex software paging schemes
- Divided into regions for better portability
- Consistent across all Cortex-M processors
- Peripherals registers are part of the memory map

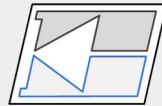
Alinhamento/Desalinhamento de dados



O Cortex-M3 pode trabalhar com dados desalinhados. Todavia, existe uma degradação de desempenho, e em alguns casos o desalinhamento não é suportado.



Bit Band



A operação com Bit Band permite que uma única operações de carga e armazenamento possa acessar um único bit da memória. No Cortex-M3 isso é possível em duas região pré-definida da memória. Uma delas fica localizada no primeiro 1MB da SRAM e a outra no primeiro 1MB destinada aos periféricos.

Essas regiões de memória podem ser acessadas normalmente ou através de uma região separada de memória chamada *bit band alias*. Quando um endereço do *bit band alias* é empregado, cada bit individual pode ser acessado no bit menos significativo (LSB) de cada palavra (cada bit tem um endereço próprio).

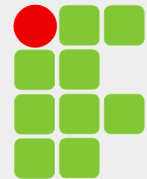
Em resumo, escreve-se no endereço do bit para ativa-lo ou limpá-lo. Uma leitura do endereço retorna o estado do bit.

Remapping of Bit-Band Addresses in the SRAM Region

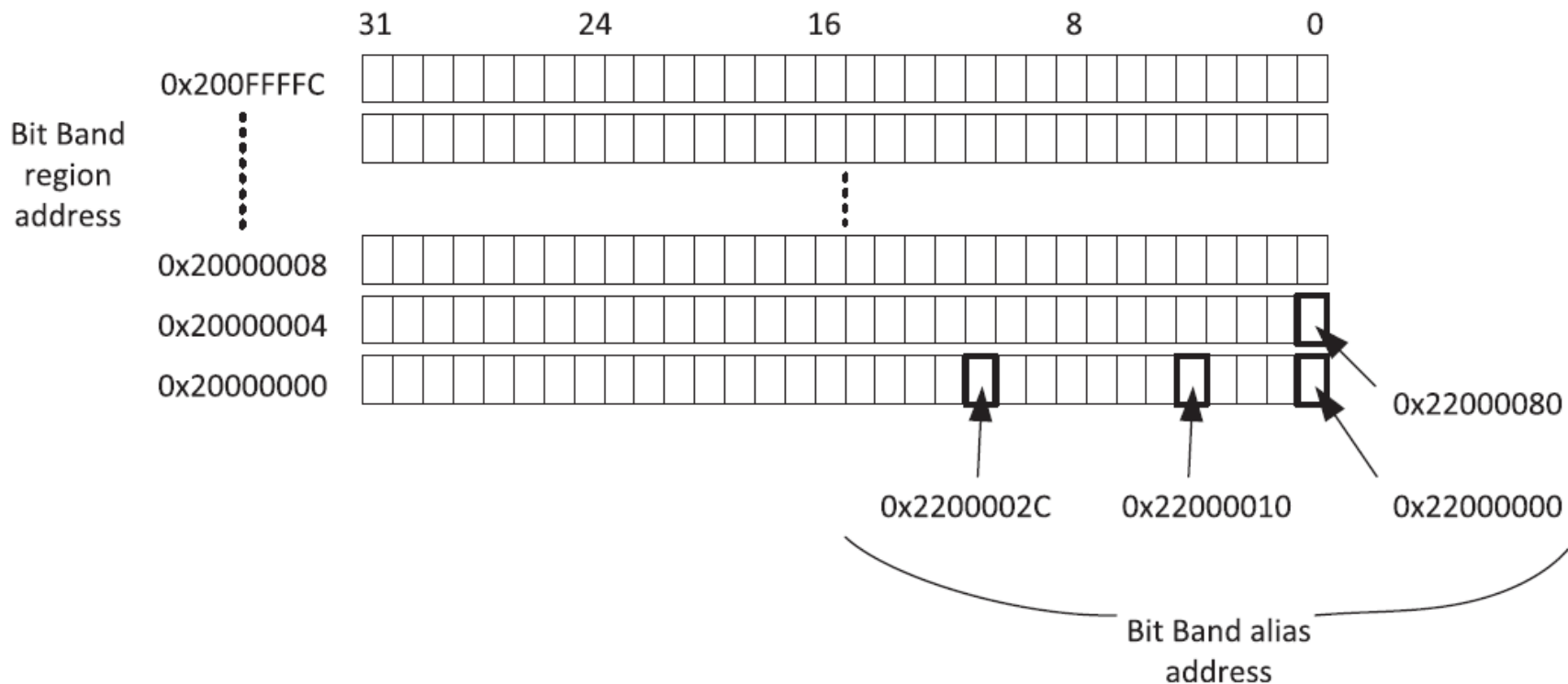
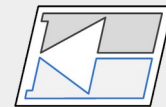
Bit-Band Region	Aliased Equivalent
0x20000000 bit [0]	0x22000000 bit [0]
0x20000000 bit [1]	0x22000004 bit [0]
0x20000000 bit [2]	0x22000008 bit [0]
...	...
0x20000000 bit [31]	0x2200007C bit [0]
0x20000004 bit [0]	0x22000080 bit [0]
...	...
0x20000004 bit [31]	0x220000FC bit [0]
...	...
0x200FFFFC bit [31]	0x23FFFFFC bit [0]

There are two regions of memory for bit-band operations:

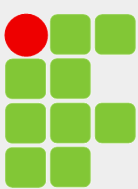
- 0x20000000—0x200FFFFF (SRAM, 1MB)
- 0x40000000—0x400FFFFF (Peripherals, 1MB)



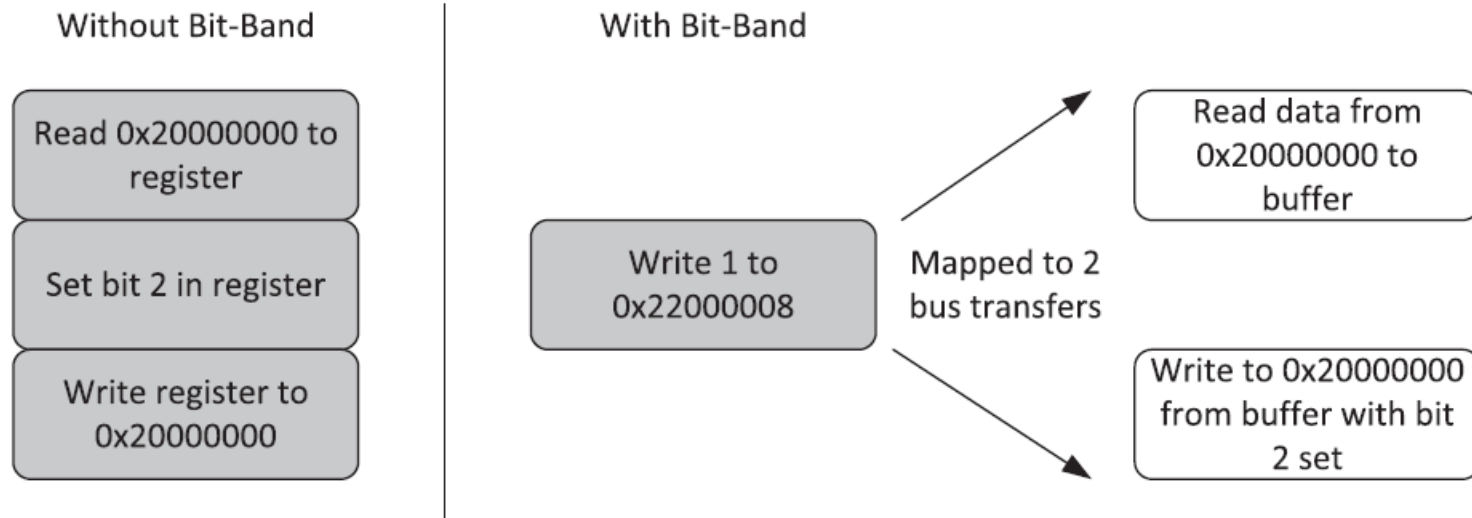
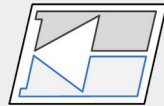
Bit Band



Bit accesses to bit-band region via the bit-band alias (SRAM region)



Escrevendo no Bit Band Alias



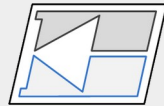
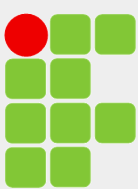
Write to bit-band alias

Without Bit-Band

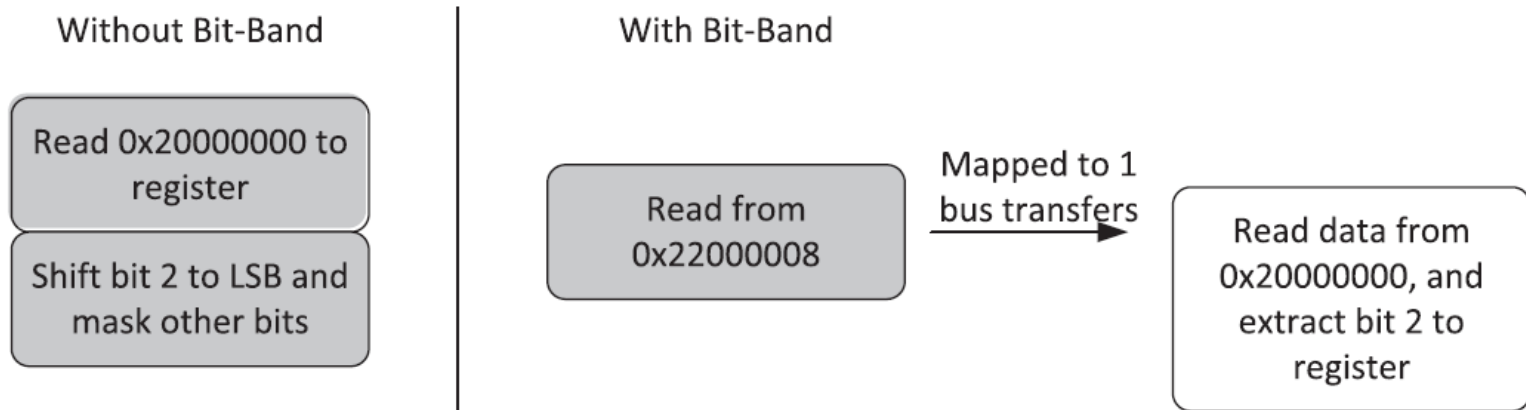
```
LDR    R0,=0x20000000 ; Setup address
LDR    R1, [R0]        ; Read
ORR.W  R1, #0x4         ; Modify bit
STR    R1, [R0] ; Write back result
```

With Bit-Band

```
LDR    R0,=0x22000008 ; Setup address
MOV     R1, #1         ; Setup data
STR     R1, [R0]       ; Write
```



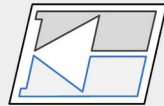
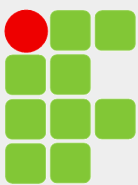
Lendo do Bit Band Alias



Read from the bit-band alias

Without Bit-Band
LDR R0,=0x20000000 ; Setup address
LDR R1, [R0] ; Read
UBFX.W R1, R1, #2, #1 ; Extract bit[2]

With Bit-Band
LDR R0,=0x22000008 ; Setup address
LDR R1, [R0] ; Read



Vantagens do Bit Band

Simplifica o acesso aos registradores de propósito específico, principalmente aos pinos de I/O facilitando a transferência de dados seriais. Resulta em um maior desempenho no trabalho com bits.

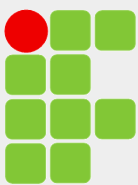
Simplifica as decisões de desvio se esse deve ser feito baseado em um bit de algum registrador. A operação resulta em:

- Ler o bit através do bit band alias.
- Comparar e desviar.

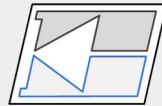
Ao invés de:

- Ler o registrador inteiro.
- Mascaram os bits indesejados.
- Comparar e desviar.

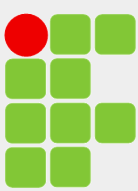
O bit band é essencial em situações de compartilhamento de recursos por processos. É uma operação atômica, a sequência READ-MODIFY-WRITE não pode ser interrompida.



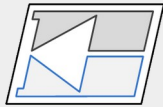
Exemplo em C



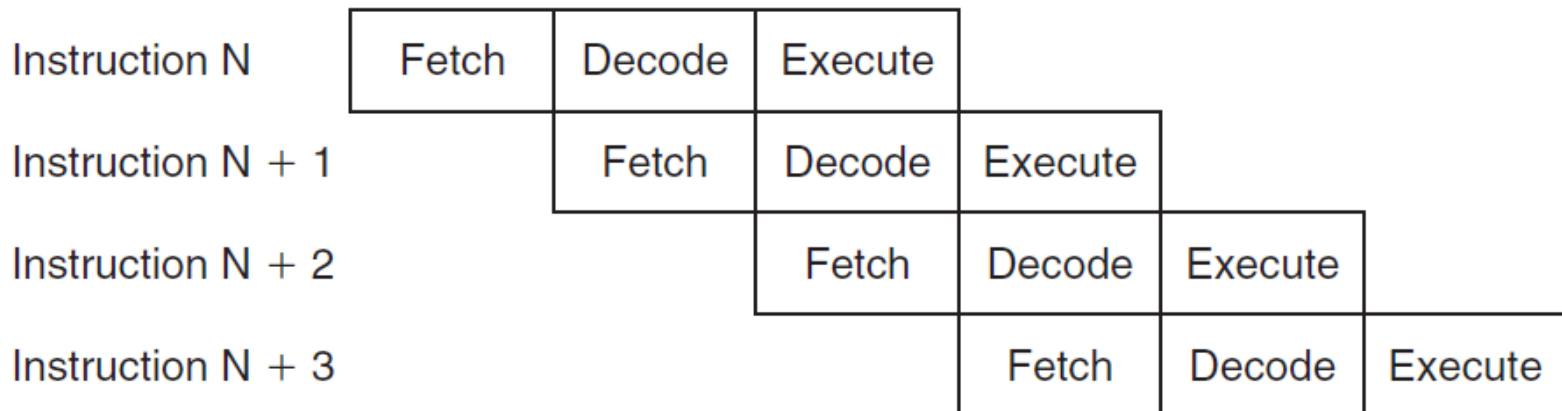
```
#define DEVICE_REG0      *((volatile unsigned long *) (0x40000000))
#define DEVICE_REG0_BIT0 *((volatile unsigned long *) (0x42000000))
#define DEVICE_REG0_BIT1 *((volatile unsigned long *) (0x42000004))
. . .
DEVICE_REG0 = 0xAB; // Accessing the hardware register by normal address
. . .
DEVICE_REG0 = DEVICE_REG0 | 0x2; // Setting bit 1 without using bit-band
. . .
DEVICE_REG0_BIT1 = 0x1; // Setting bit 1 via the bit-band alias address
```

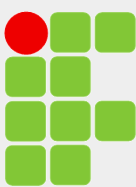


Pipeline



- ❑ Cada instrução é processada em um número de passos discretos.
- ❑ Múltiplas instruções podem ser processadas simultaneamente pelo início do processamento da próxima instrução antes da última ter terminado.
- ❑ Os processadores ARM Cortex-M3 empregam um pipeline simples de 3 estágios: Busca → Decodificação → Execução.
- ❑ O Hardware de cada estágio é independente de forma que 3 instruções podem ser processadas independentemente.
- ❑ Instruções de desvio limpam e recompõem o pipeline.





Pipeline

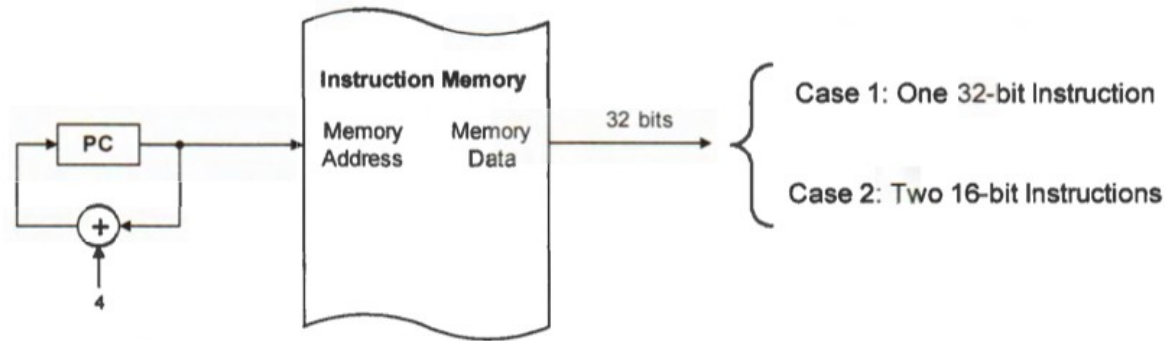
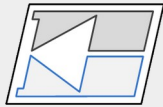


Figure 1-12. The program counter (PC) is incremented by 4. PC holds the memory address of the next instruction(s) to be fetched from the instruction memory. When the program reads PC, the returned value is the address of the current instruction plus 4.

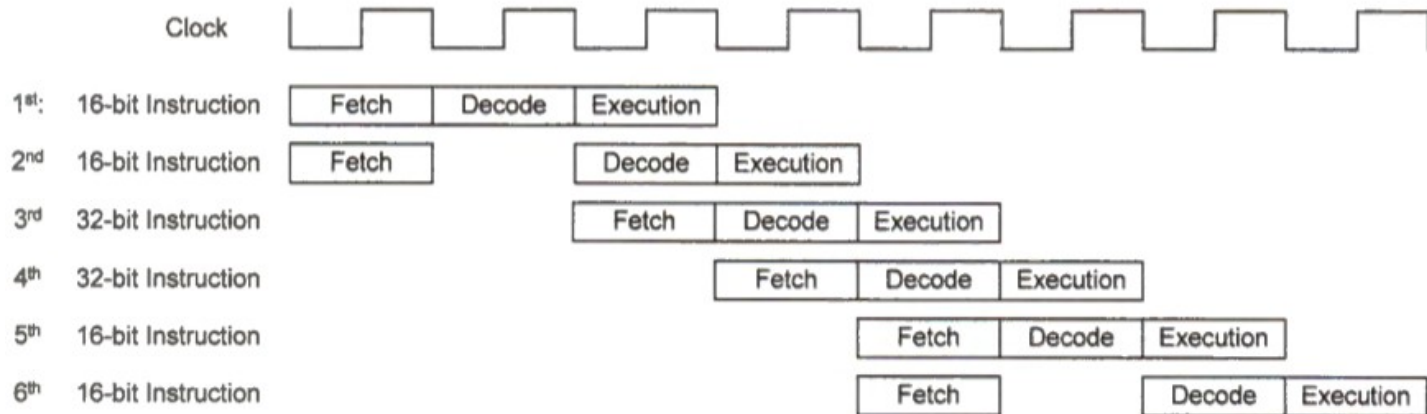
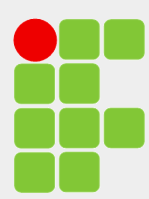
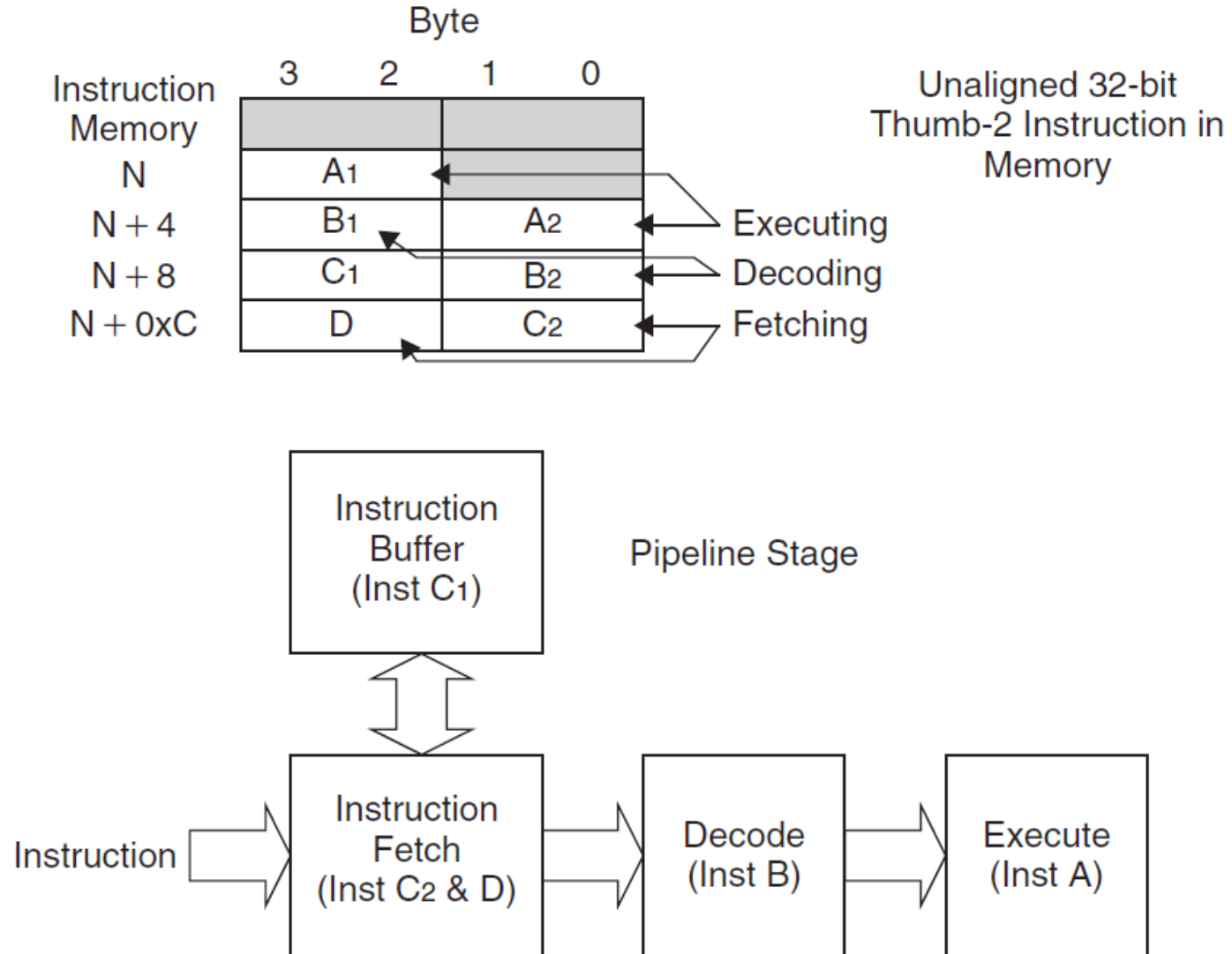
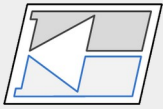
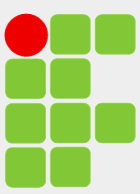


Figure 1-14. Three-stage *fetch-decode-execution* pipeline in Cortex-M4. Pipelining makes hardware resources fully utilized. Each time the processor fetches 4 bytes from the instruction memory, which includes two 16-bit instructions or one 32-bit instruction (See Chapter 13.2 for details). Therefore, after each fetch, PC is incremented by 4.

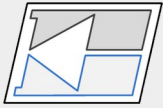


Pipeline

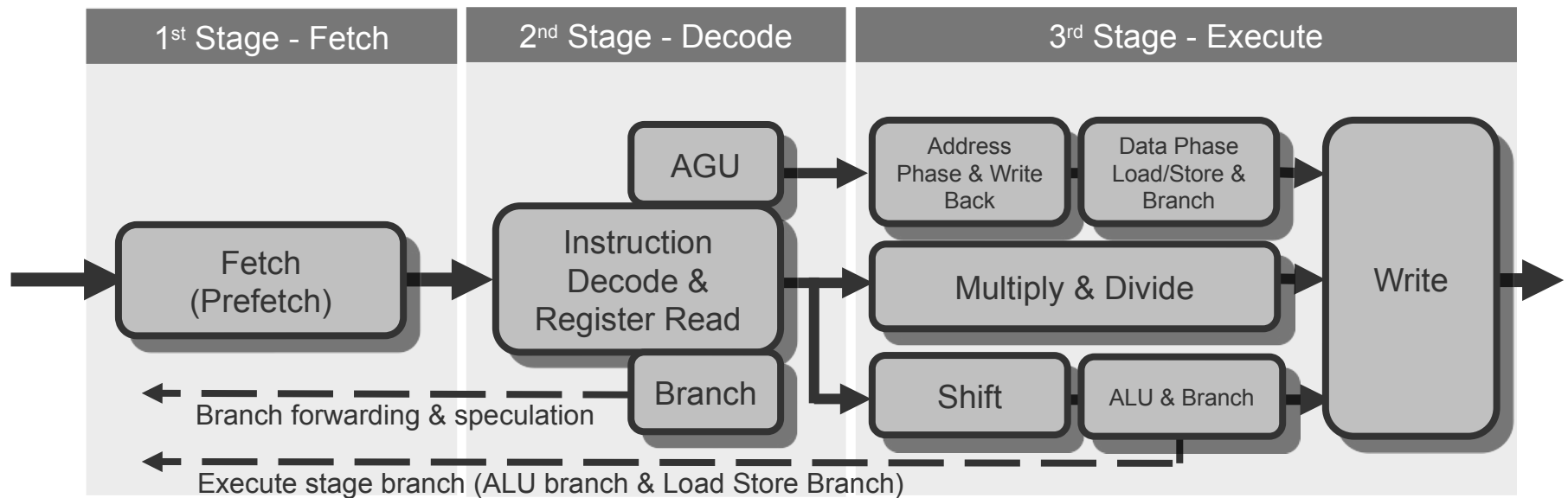




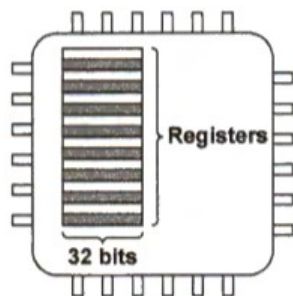
Pipeline



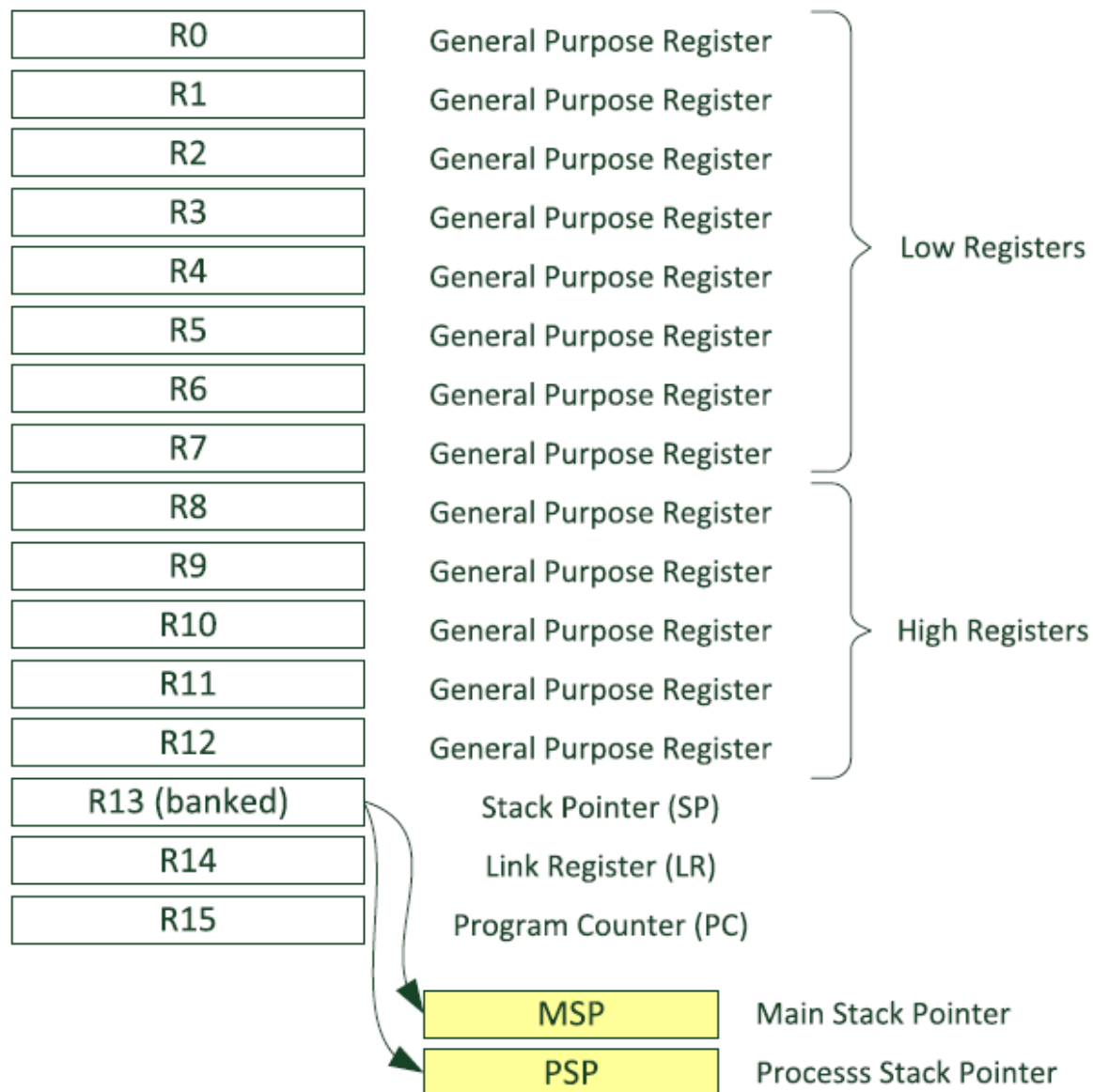
Cortex-M3 does more in each stage to increase overall performance.



Registradores

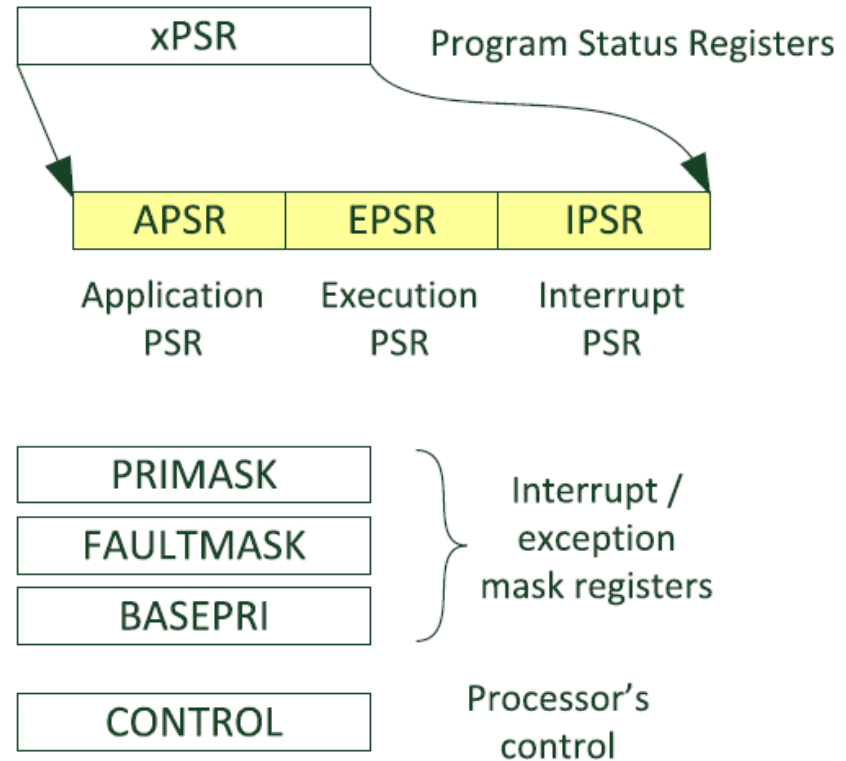


Algumas instruções
Thumb só podem acessar
R0-R7

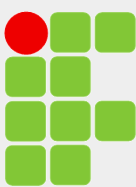


Registadores Especiais

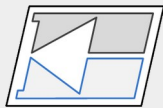
Não são usados no processamento normal.



Register	Function
xPSR	Provide ALU flags (zero flag, carry flag), execution status, and current executing interrupt number
PRIMASK	Disable all interrupts except the nonmaskable interrupt (NMI) and HardFault
FAULTMASK	Disable all interrupts except the NMI
BASEPRI	Disable all interrupts of specific priority level or lower priority level
CONTROL	Define privileged status and stack pointer selection



Registradores de Status



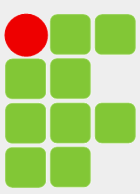
	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
APSR	N	Z	C	V	Q				GE*							
IPSR												Exception Number				
EPSR						ICI/IT	T				ICI/IT					

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
xPSR	N	Z	C	V	Q	ICI/IT	T		GE*	ICI/IT		Exception Number				

*GE is available in ARMv7E-M processors such as the Cortex-M4. It is not available in the Cortex-M3 processor.

- One Status Register consisting of
 - APSR - Application Program Status Register – ALU flags
 - IPSR - Interrupt Program Status Register – Interrupt/Exception No.
 - EPSR - Execution Program Status Register
 - IT field – If/Then block information
 - ICI field – Interruptible-Continuable Instruction information
- xPSR
 - Composite of the 3 PSRs
 - Stored on the stack on exception entry

Bit	Description
N	Negative flag
Z	Zero flag
C	Carry (or NOT borrow) flag
V	Overflow flag
Q	Sticky saturation flag (not available in ARMv6-M)
GE[3:0]	Greater-Than or Equal flags for each byte lane (ARMv7E-M only; not available in ARMv6-M or Cortex®-M3).
ICI/IT	Interrupt-Continuable Instruction (ICI) bits, IF-THEN instruction status bit for conditional execution (not available in ARMv6-M).
T	Thumb state, always 1; trying to clear this bit will cause a fault exception.
Exception Number	Indicates which exception the processor is handling.



Stack Pointer

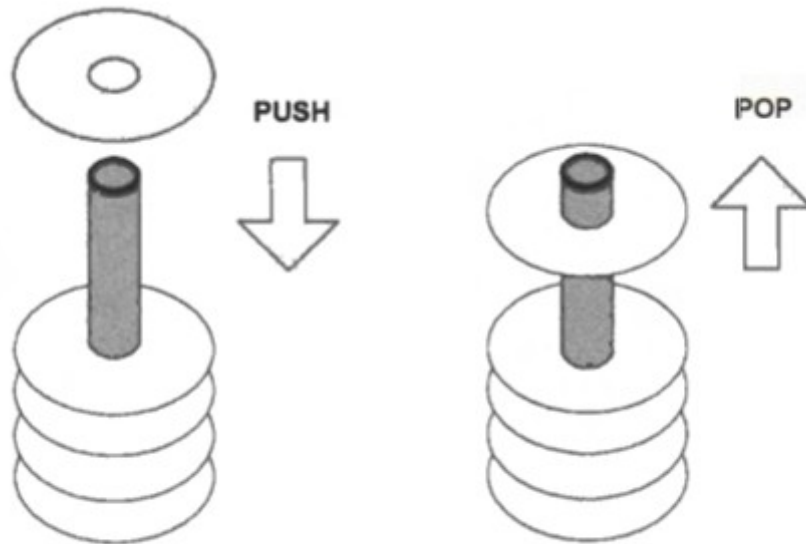
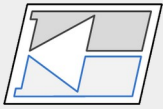
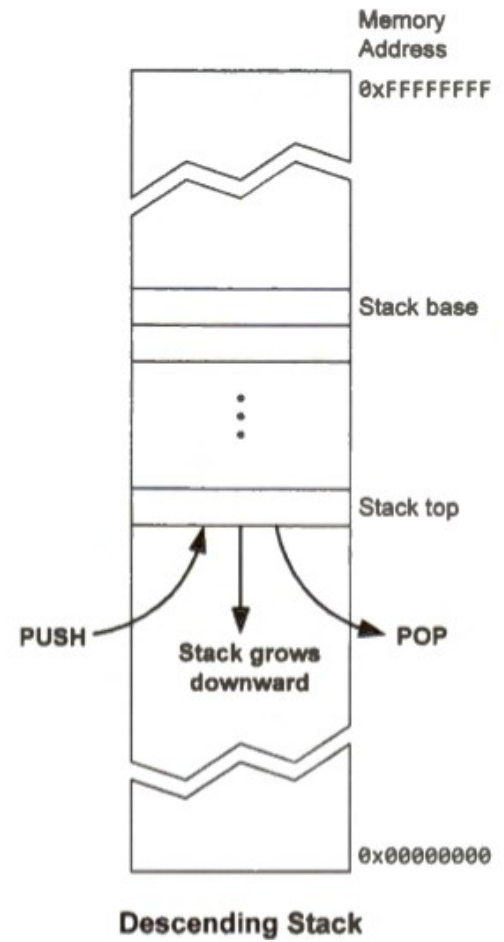
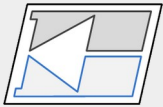
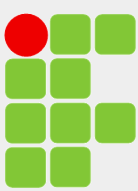


Figure 8-2. A stack is last-in-first-out (LIFO).

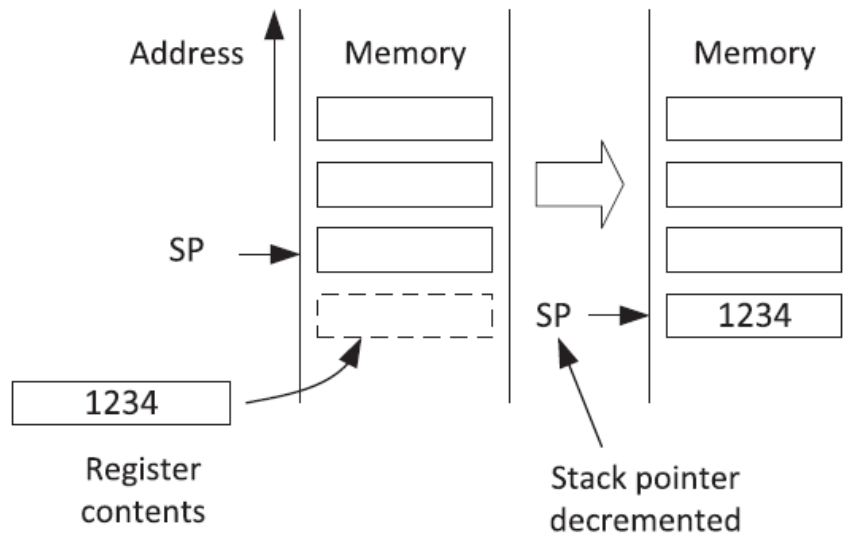




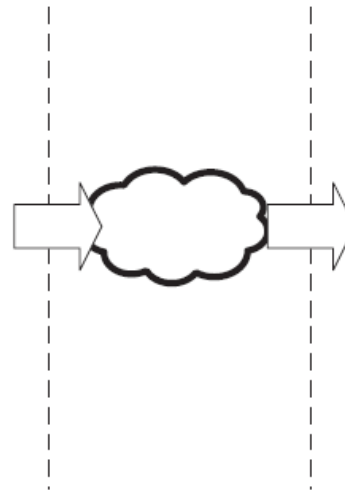
Stack Pointer

PUSH operation

Stack PUSH operation to back up register contents

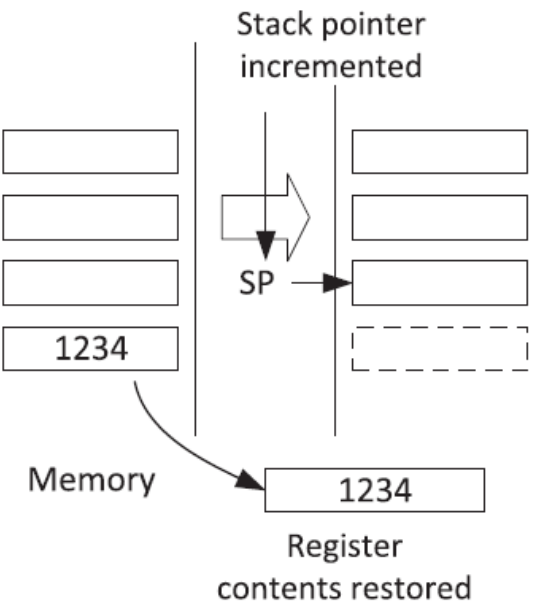


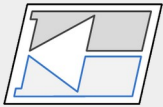
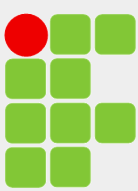
Data Processing
(Original register contents destroyed)



POP operation

Stack POP operation to restore register contents





Modos de operação

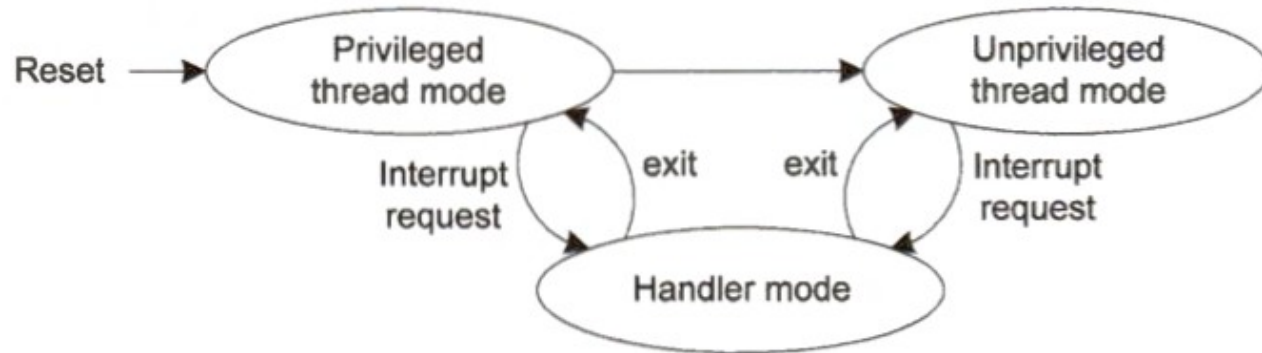
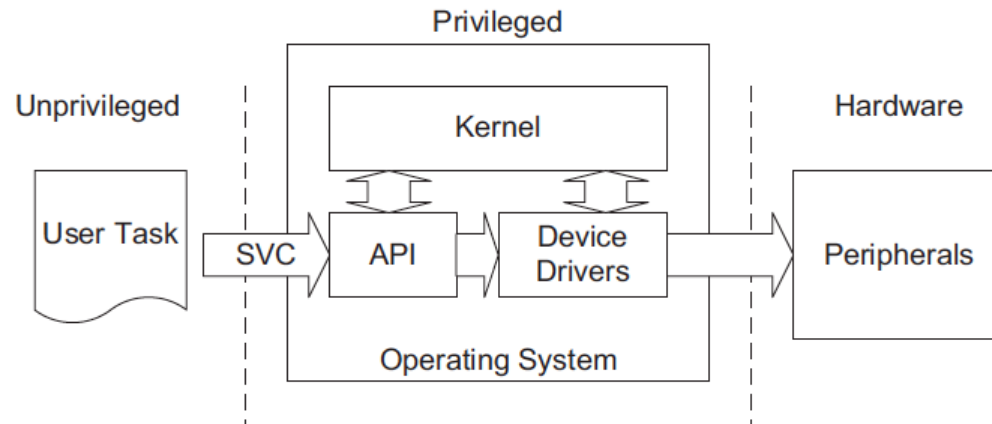
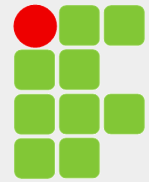


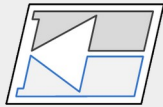
Figure 23-1. Thread mode and handler mode

Exemplo:

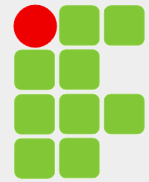




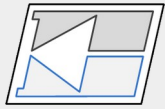
Suporte a Sistemas Operacionais



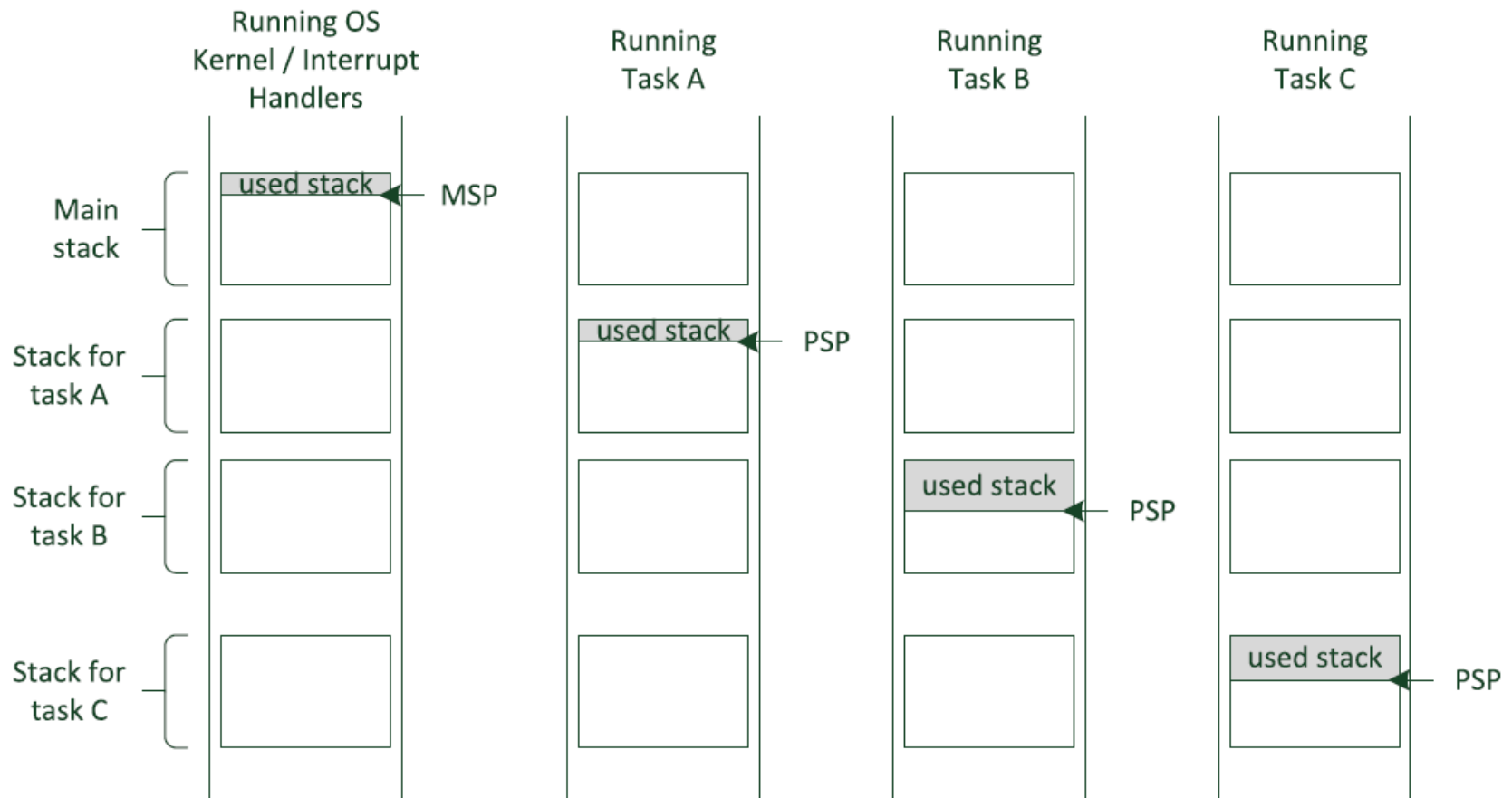
- Dois *Stack Pointers* disponíveis – o registrador MSP é usado pelo núcleo do SO e pelos tratadores de interrupção. O registrador PSP é usado pelas tarefas (um SP para cada tarefa).
- Temporizador dedicado (SysTick) – temporizador simples para uso do SO.
- Exceções SVC e PendSV - importantes para a troca de contexto.
- Execução em modo não privilegiado – modo de segurança restringindo o direito de acesso às tarefas. Os modos privilegiado e não privilegiado podem ser usados em conjunto com a Unidade de Proteção de Memória (MPU) para melhorar a robustez em sistemas embarcados.
- Acesso exclusivo – instruções *Load* e *Store* dedicadas, úteis em semáforos e exclusão mútua (MUTEX) pelo SO.
- Baixa latência nas interrupções – diminuindo o *overhead* na troca de contexto.



Suporte a Sistemas Operacionais



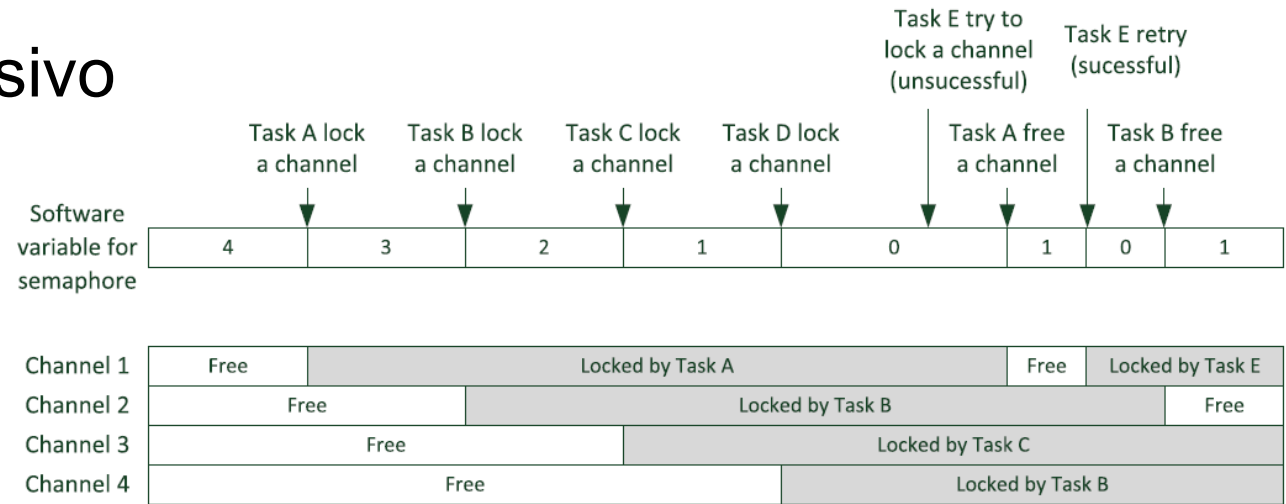
Um SP dedicado para cada tarefa no SO. Na troca de contexto o SO atualiza o PSP.



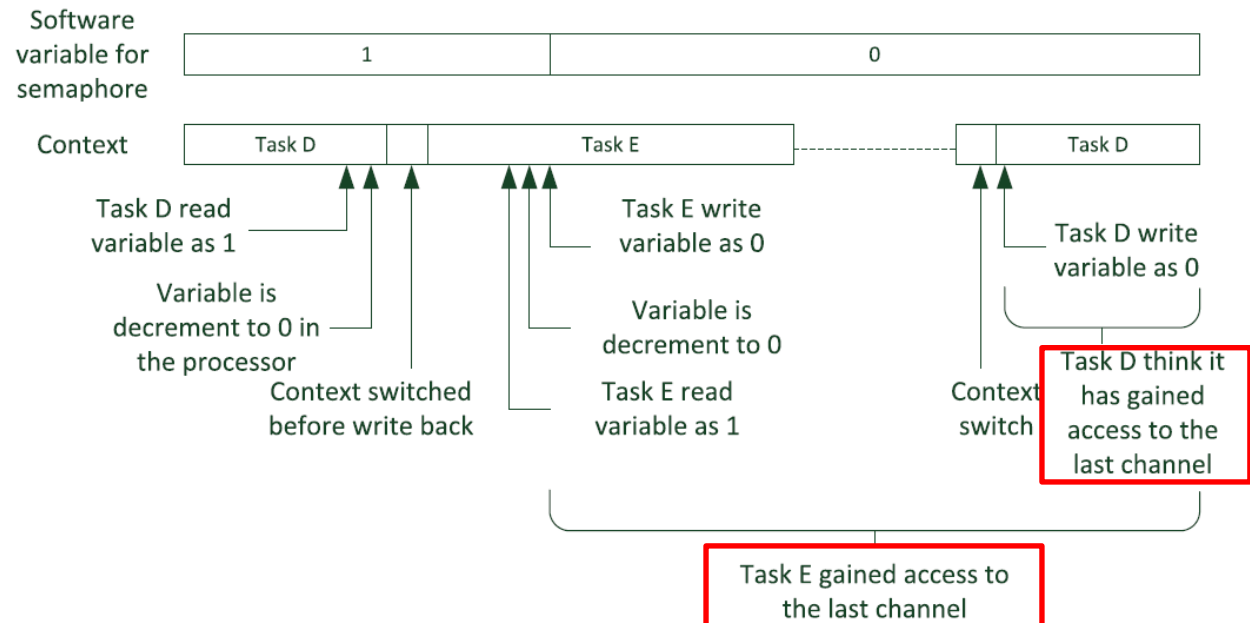
Acesso Exclusivo

O ARM possui um mecanismo para impedir que uma troca de contexto entre uma escrita e leitura leve a uma erro de interpretação.

Instruções:
STREX
LDREX

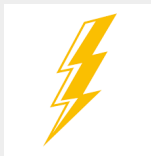
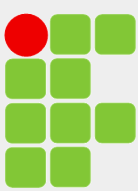


Semaphore operation example

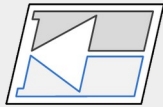


Instrução atômica?
Bit band?

Context switching at the middle of the read-modify-write sequence



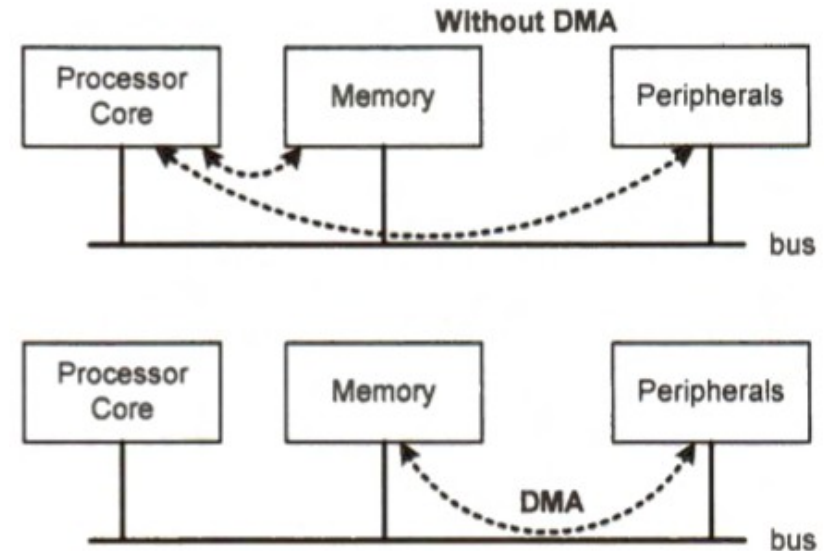
DMA

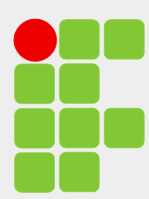


Direct Memory Access é uma técnica para a transferência de dados entre memória e periféricos ou entre memória sem usar vários ciclos do processador.

- * Libera o processador de esperar o dado de periféricos lentos.
- * Para periféricos rápidos como ADC e DAC, melhora o desempenho pois não exige a interferência do processador.
- * Reduz significativamente a taxa de interrupções geradas por periféricos muito rápidos.

With DMA support, a peripheral does not need to have local memory to store data. Instead, data can be efficiently stored in data memory via DMA. This design not only reduces the cost but also improves the energy efficiency of embedded systems.





DMA

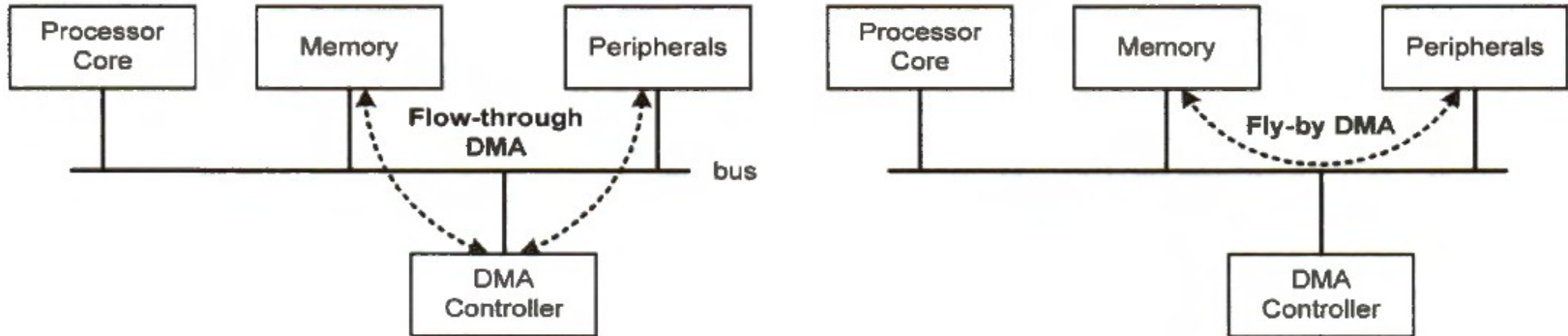
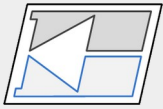
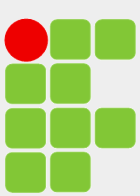


Figure 19-3. Flow-through *vs* fly-by DMA

DMA transfers can be either *flow-through* or *fly-by*, as shown in Figure 19-3.

- A *flow-through* (or fetch-and-deposit) DMA transfer involves two steps: (1) data are read one at a time from the source into a register in the DMA controller, and (2) data are written to the destination.
- A *fly-by* DMA transfer only requires one bus transfer. Data are directly transferred from the source to the destination without being read into the DMA controller.



DMA exemplo

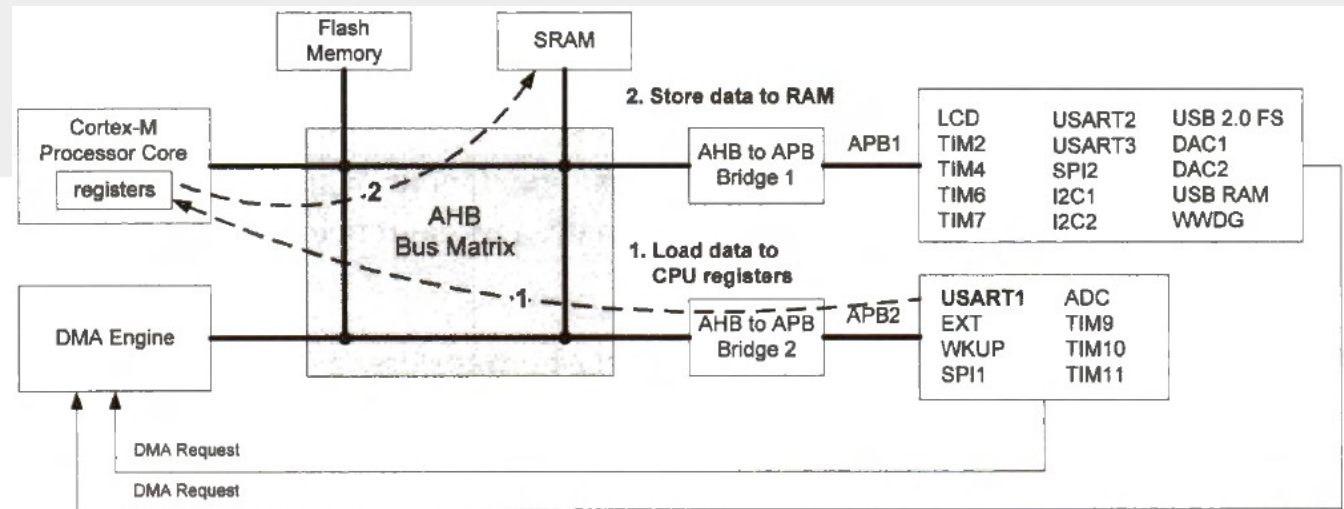


Figure 19-4. Receiving data from a USART serial port without DMA. The processor must execute load and store instructions to copy data from peripherals to the data memory.

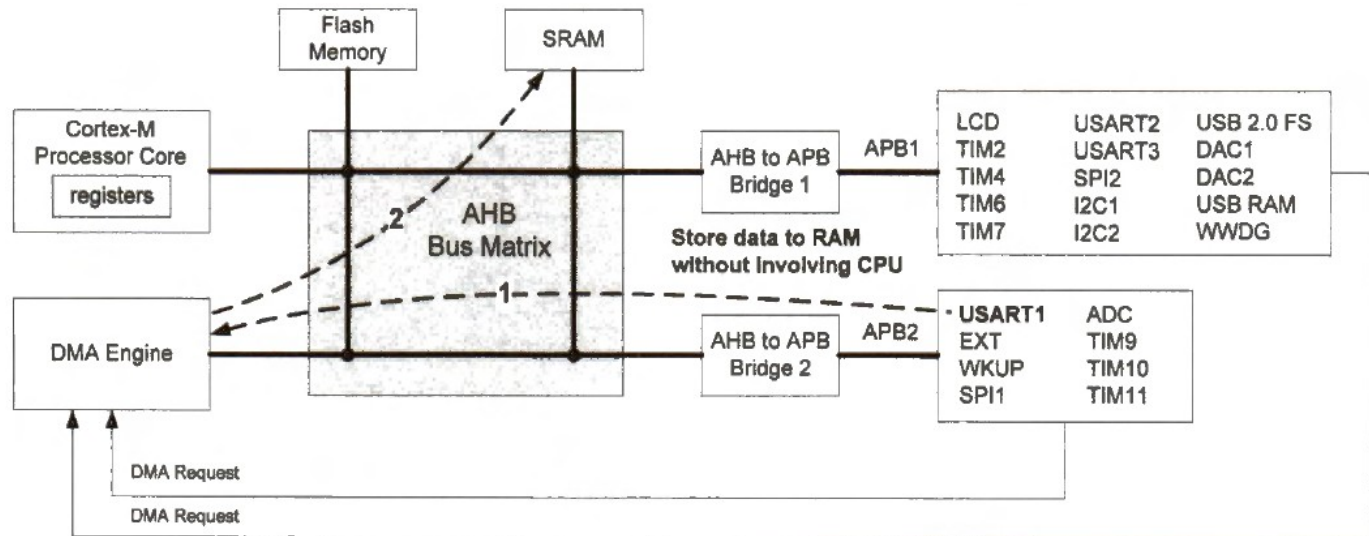
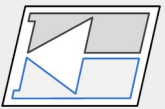
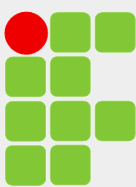


Figure 19-5. Receiving data from USART 1 by using DMA. The DMA engine reads data from the peripheral and then writes it to memory without the intervention of the processor. The DMA engine can inform the processor via interrupt upon completion.



DMA – modo circular

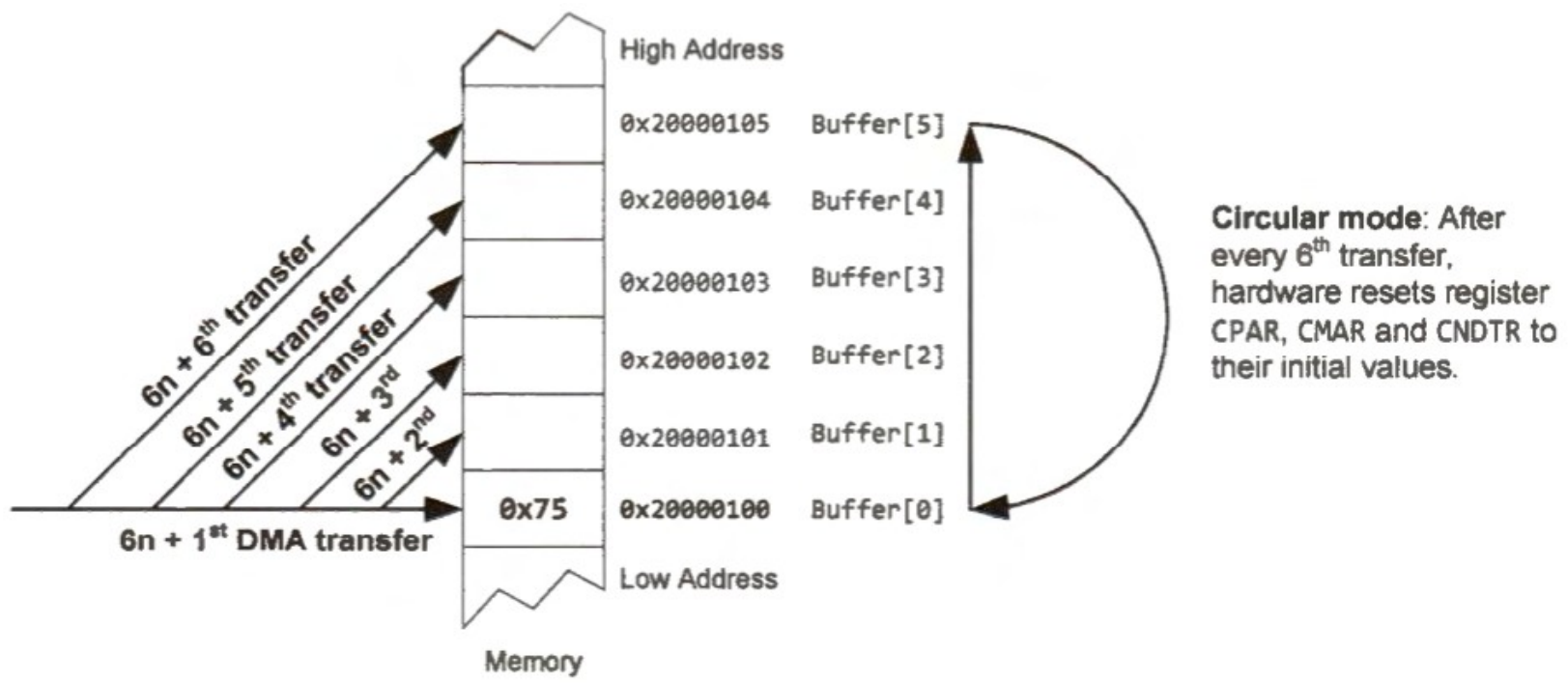
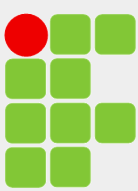


Figure 19-10. DMA in circular mode



DMA – modo circular – ex. ADC

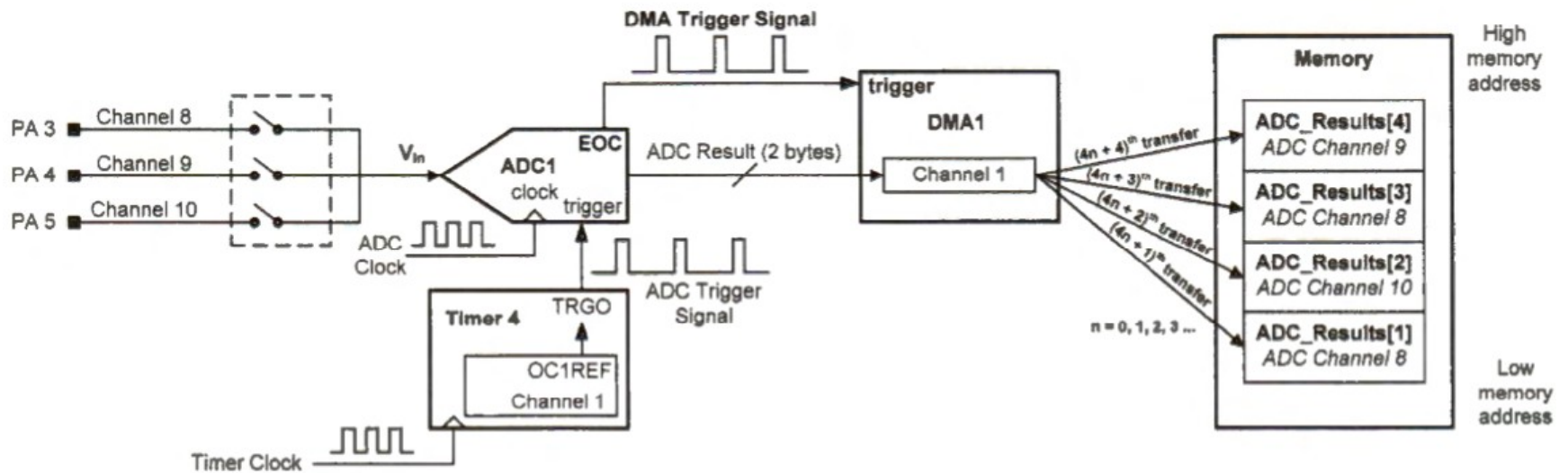
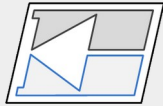
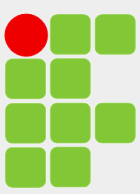
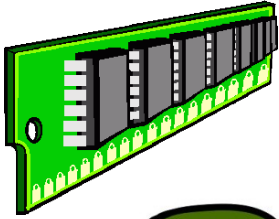
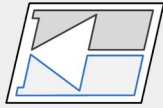


Figure 20-14. Configure DMA1 with circular mode



Memory Protection Unit - MPU

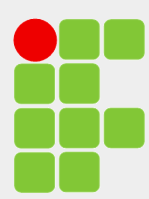


A Unidade de Proteção de Memória – MPU é um dispositivo programável que pode ser usado para definir permissões de acesso e atributos para diferentes regiões da memória.

No Cortex-M3 a MPU permite programar 8 regiões distintas da memória.

A MPU torna os sistemas mais robustos e seguros:

- Previne que tarefas corrompam o SP ou o uso de dados de outras tarefas ou do núcleo do SO.
- Previne que tarefas sem privilégio acessem certos periféricos que podem ser críticos para a confiabilidade e segurança do sistema.
- Permite definir espaços da RAM como não executáveis para prevenir a execução de códigos indesejáveis (*code injection attacks*).



Memory Protection Unit - MPU

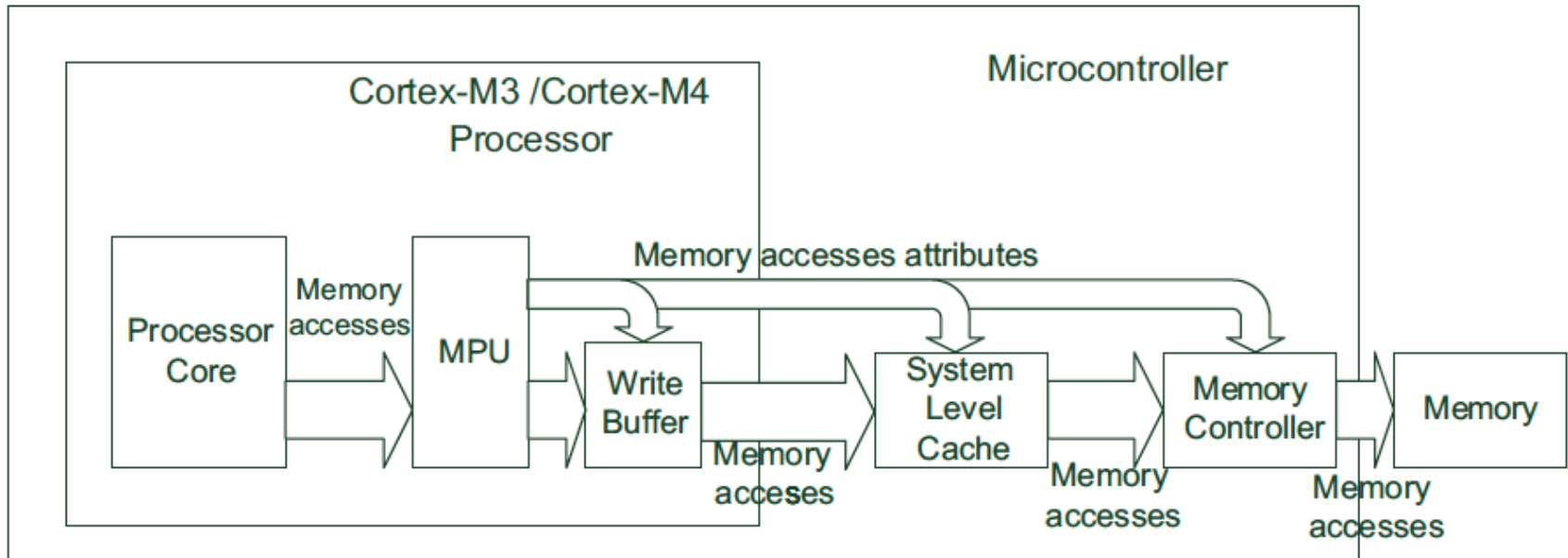
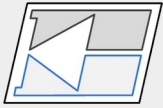
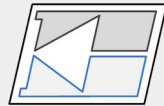
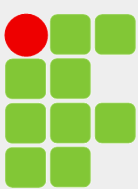


FIGURE 11.2

Memory attributes can be used inside the processor or by components on the bus system



Interrupções e Exceções

Exceções são todos os eventos que interrompem o fluxo normal do programa. Sendo que interrupção é considerada um evento normalmente gerado pelo hardware (periféricos ou entradas externas).

Quando um periférico ou hardware necessita de algum serviço do processador, tipicamente os seguintes passos ocorrem:

1. O periférico faz um pedido de interrupção ao processador.
2. O processador suspende a tarefa em execução.
3. O processador executa a rotina do serviço de interrupção (ISR) para tratar a interrupção (pode ser suspendida por software se necessário).
4. O processador volta a executar a tarefa suspensa previamente.

O Cortex-M possui o NVIC (*Nested Vectored Interrupt Controller*) para tratar as interrupções. Outras exceções que necessitam de tratamento incluem faltas e outras exceções para suporte de um SO. As partes do programa que tratam essas exceções são chamados tratadores de exceção (*exception handlers*).

*An interrupt is simply a
hardware-invoked
function call.*

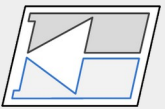
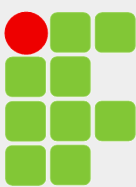
Tabela de Endereços das Exceções

O endereço de desvio para o tratamento de exceções-interrupções (ISRs) é armazenado em uma tabela. Cada posição possui o endereço específico onde consta o código a ser executado.

O endereço de início por definição é 0, mas pode ser alocado para qualquer outra posição da memória. Geralmente, é definido no código de *startup* fornecido pelo fabricante do chip.

Memory Address		Exception Number
0x0000004C	Interrupt#3 vector	19
0x00000048	Interrupt#2 vector	18
0x00000044	Interrupt#1 vector	17
0x00000040	Interrupt#0 vector	16
0x0000003C	SysTick vector	15
0x00000038	PendSV vector	14
0x00000034	Not used	13
0x00000030	Debug Monitor vector	12
0x0000002C	SVC vector	11
0x00000028	Not used	10
0x00000024	Not used	9
0x00000020	Not used	8
0x0000001C	Not used	7
0x00000018	Usage Fault vector	6
0x00000014	Bus Fault vector	5
0x00000010	MemManage vector	4
0x0000000C	HardFault vector	3
0x00000008	NMI vector	2
0x00000004	Reset vector	1
0x00000000	MSP initial value	0

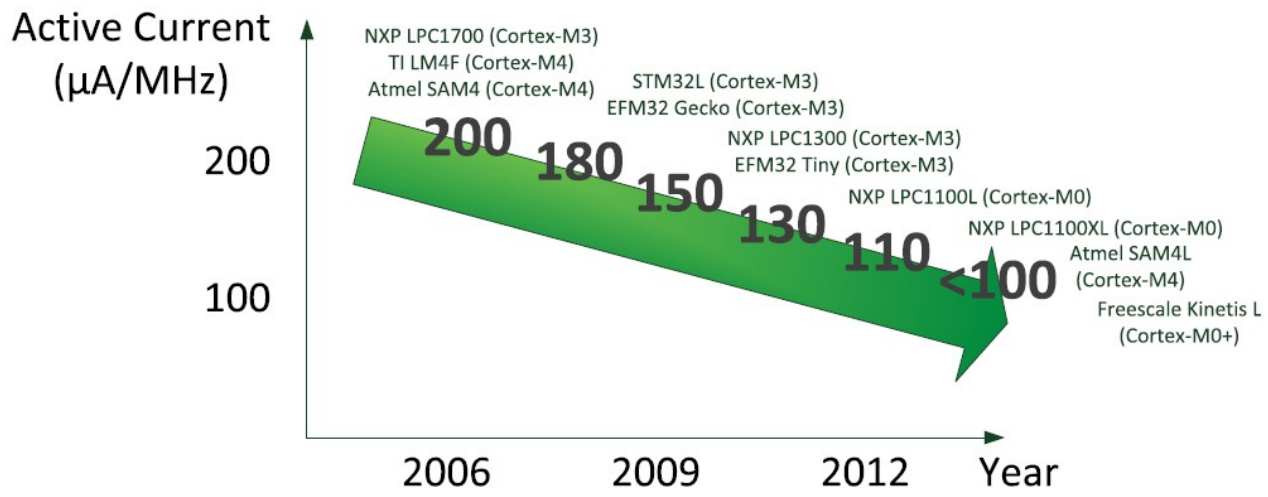
Note : LSB of each vector must be set to 1 to indicate Thumb state



Economia energia

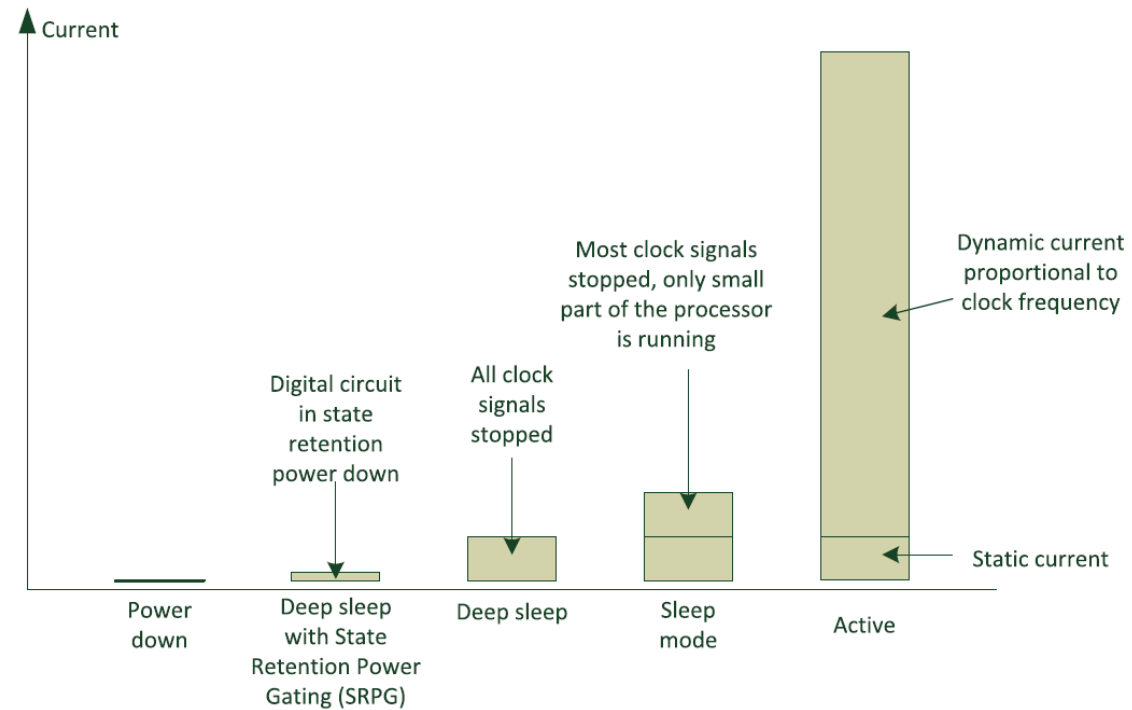
O Cortex-M possui várias características para baixo consumo energético:

- Vários modos de trabalho e *sleep* disponíveis.
- Ultra *low-power Real Time Clock* (RTC), watchdog e Brow-Out-Detector (BOD).
- Periféricos inteligentes que podem operar enquanto o processador permanece no modo *sleep*.
- Flexível controle do sistema de *clock*, permitindo desativar o sinal de *clock* em partes do hardware e ajustar a frequência de trabalho.



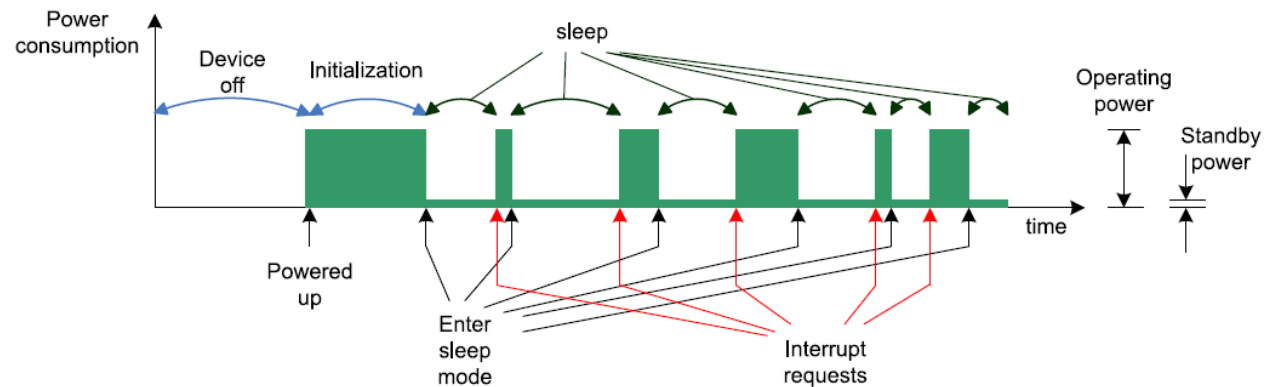
Active current	µA/MHz
Sleep mode current	µA
Energy efficiency	DMIPS/µW CoreMark/µW
Wake-up latency	clock cycles µsec

Modos de consumo

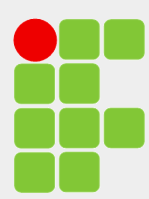


Various power modes including sleep modes

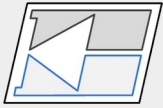
Usando interrupção



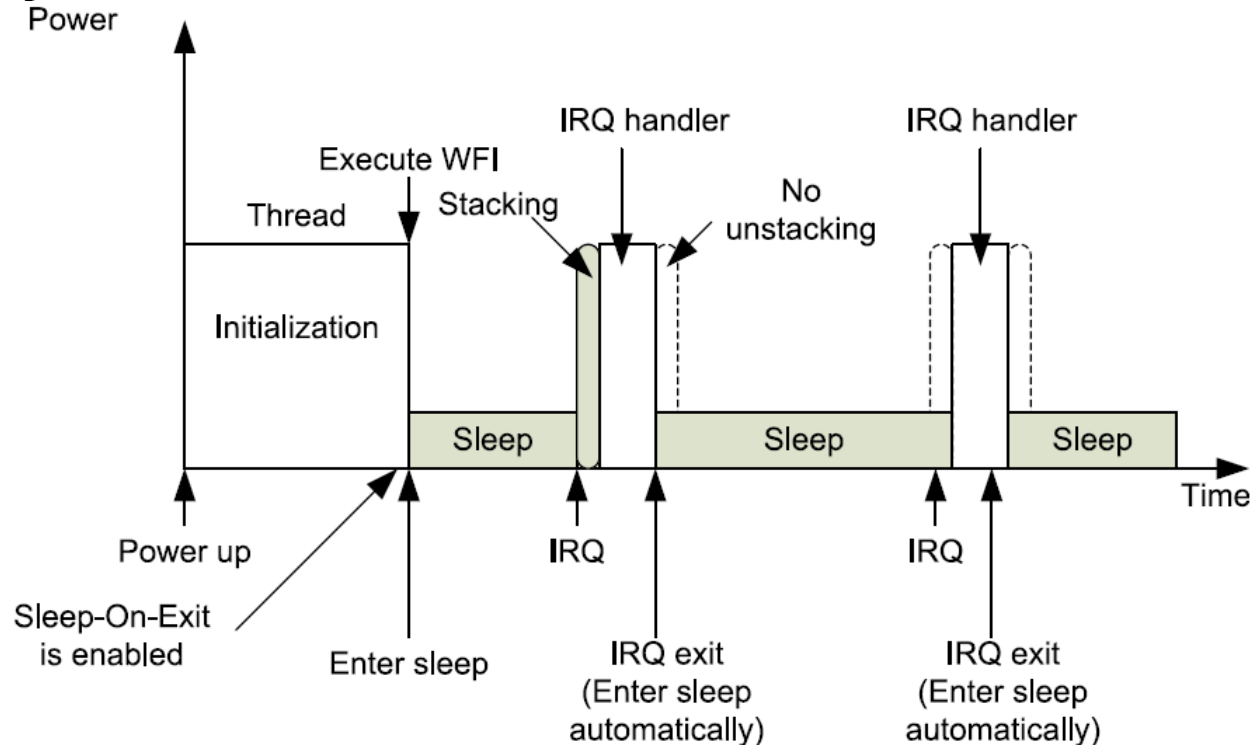
Activities in an interrupt-driven application



Sleep on Exit

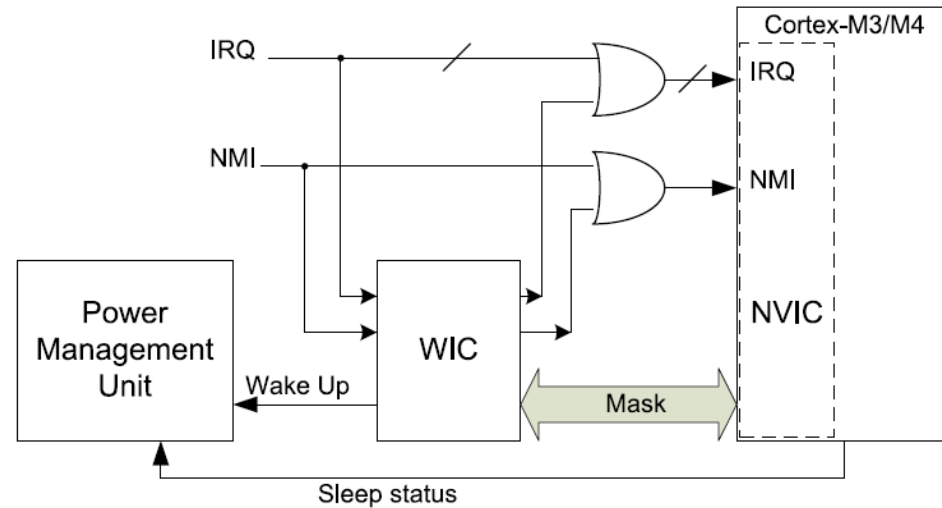


No Cortex-M é possível habilitar a opção de entrar em modo *sleep* automaticamente ao final de uma interrupção.



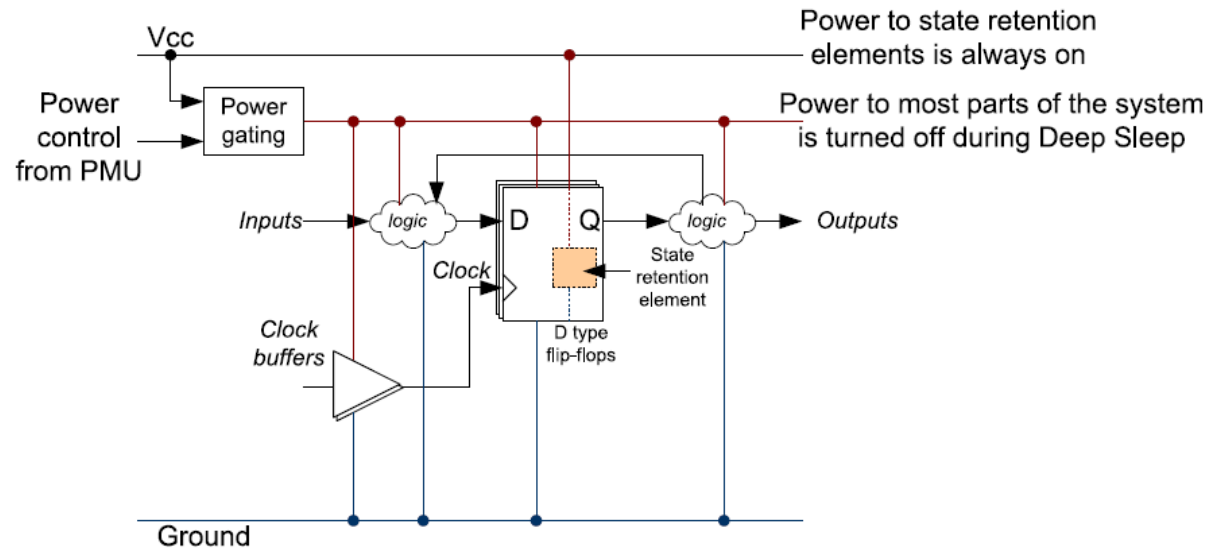
Sleep-on-Exit operations

Wake-up Interrupt Controller

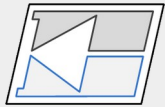
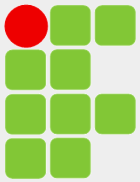


WIC mirrors the interrupt detection function when the clock signals to the processor stop

State Retention Power Gating



SRPG technology allows most parts of a digital system to be powered down without state loss



Reduzindo a potência ativa:

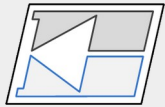
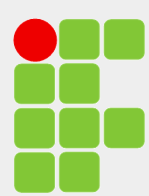
- Escolha o μ controlador correto.
- Trabalhe com a frequência suficiente.
- Selecione a fonte certa de *clock*.
- Desligue sinais de *clock* não empregados.
- Utilize as opções do sistema de *clock*.
- Fonte de alimentação projetada para uma menor perda.
- Rodar o programa da RAM (desligar a memória Flash).
- Usar uma configuração adequada dos pinos de I/O.

Reduzindo o número de ciclos ativos:

- Utilize modos de *sleep*.
- Reduza o tempo de processamento (otimizando o código).

Redução da corrente no modo *sleep*:

- Use o modo adequado de *sleep*.
- Utilize as opções de controle de potência.
- Desligue a memória Flash durante o modo *sleep*.

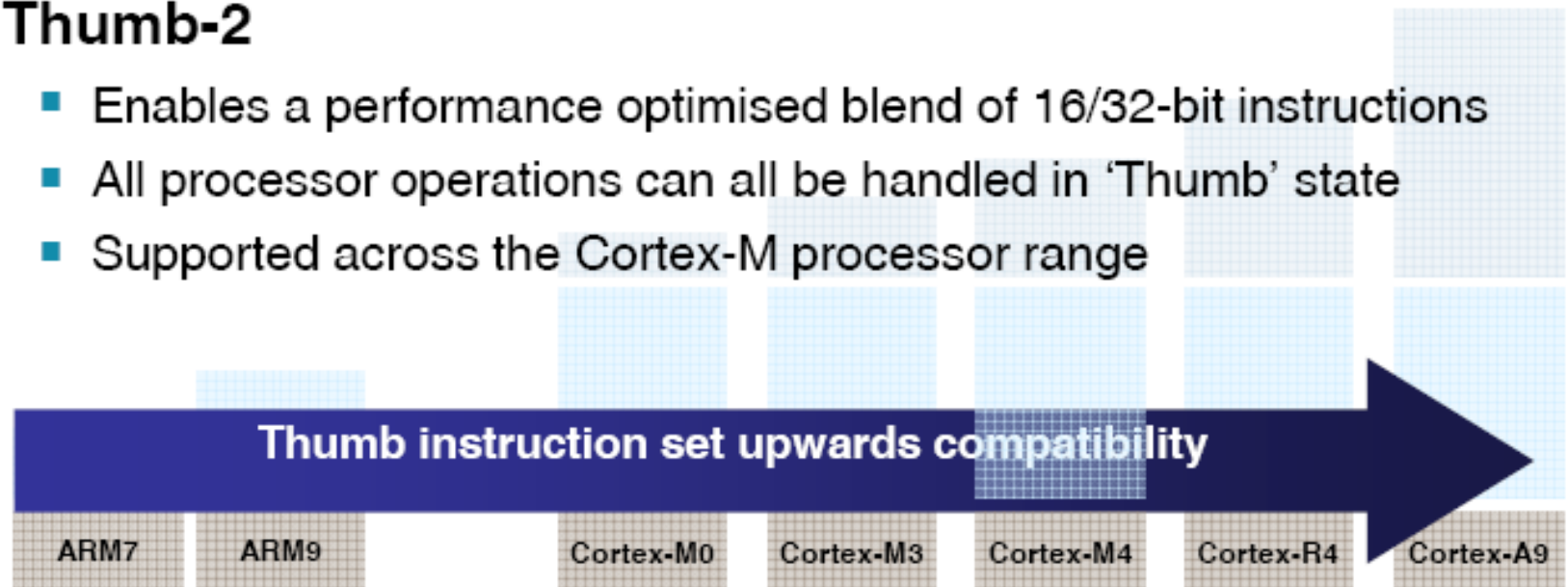


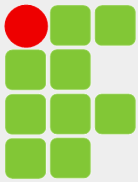
■ Thumb

- 32-bit operations in 16-bit instructions
- Introduced in ARM7TDMI[®] processor ('T' stands for Thumb)
- Subsequently supported in every ARM processor developed since

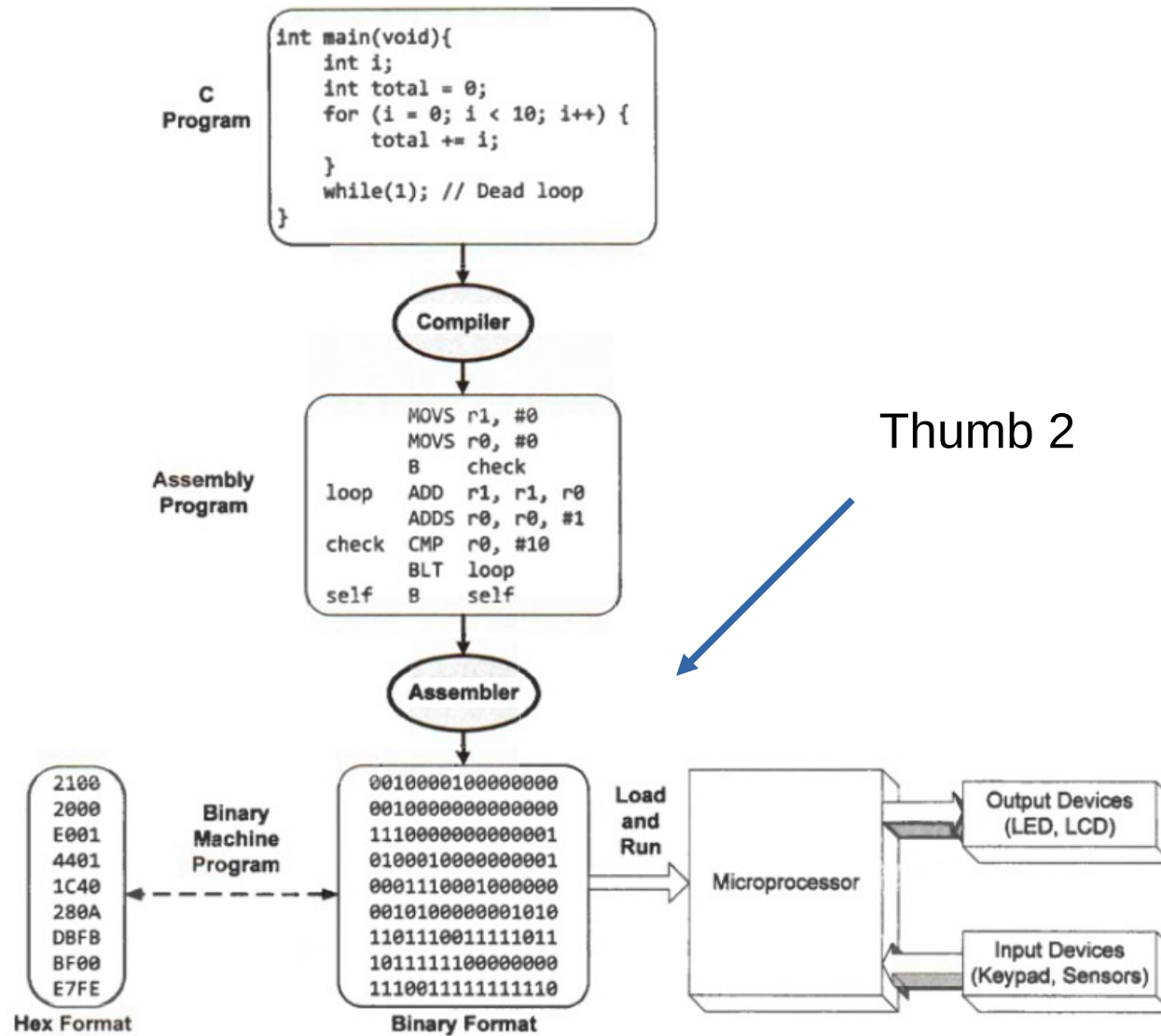
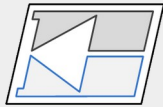
■ Thumb-2

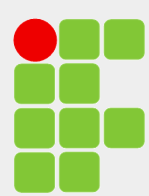
- Enables a performance optimised blend of 16/32-bit instructions
- All processor operations can all be handled in 'Thumb' state
- Supported across the Cortex-M processor range



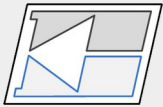


ISA



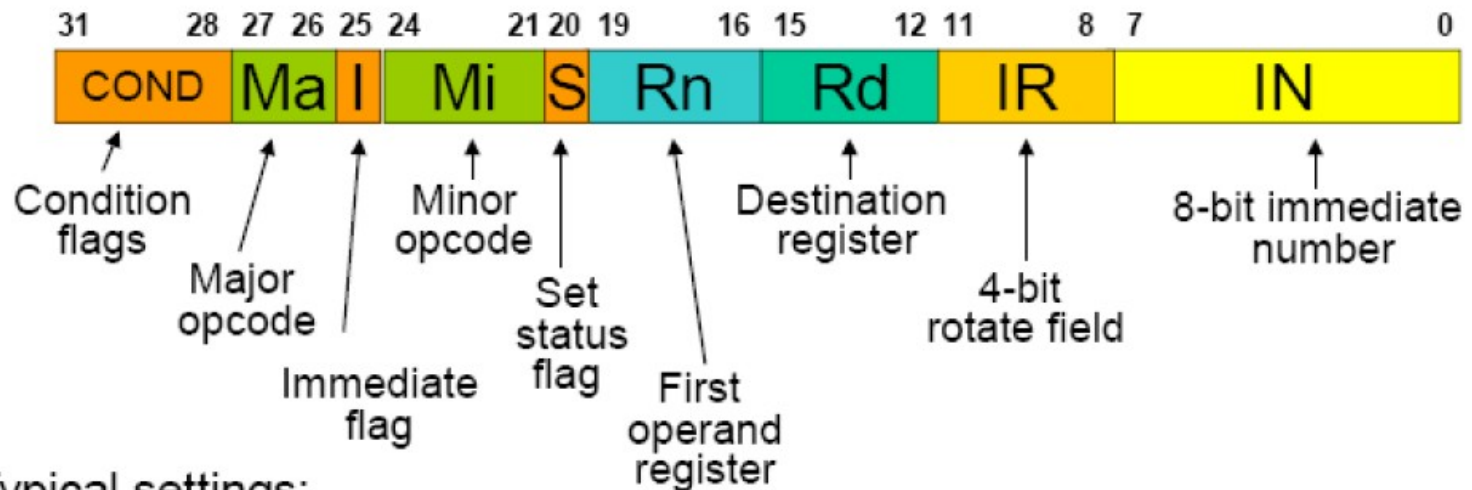


Codificação em 32 bits



Example: ADD instruction format

- ARM 32-bit encoding for ADD with immediate field



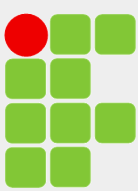
Typical settings:

Major opcode = 00 (this indicates data operation instructions)

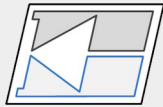
Minor opcode = 0100 (specifically, 100 $\Rightarrow$ ADD instruction)

Immediate flag = 1 (immediate field in operand 2)

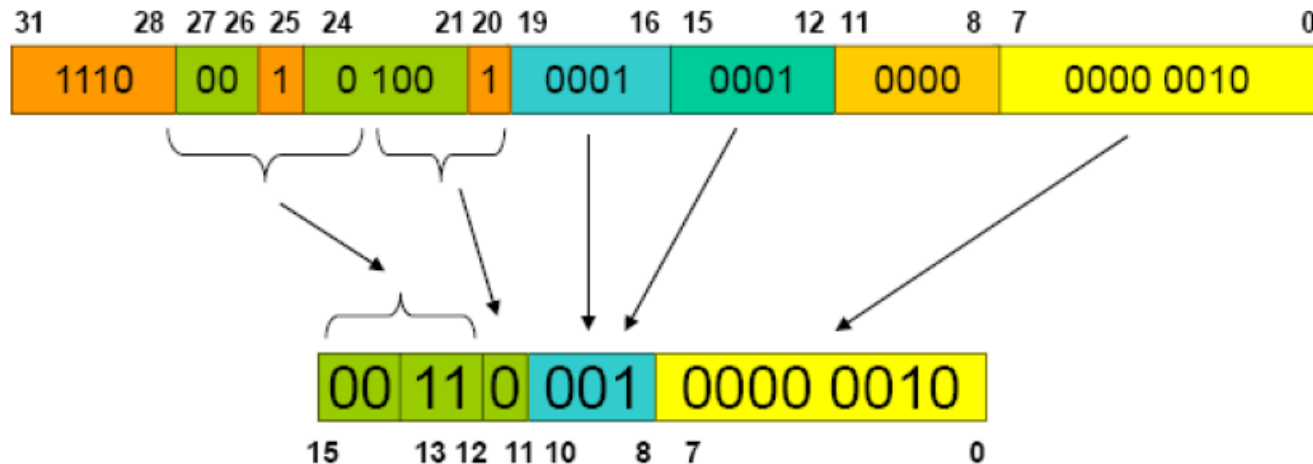
Set status flag = 1 (set carry flag after operation)



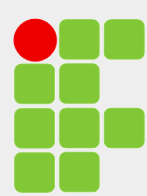
Codificação em 16 bits



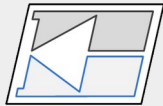
ARM 32-bit encoding: ADDS r1, r1, #2



- Equivalent 16-bit Thumb instruction: ADD r1, #2
 - No condition flag
 - No rotate field for the immediate number
 - Use 3-bit encoding for the register
 - Shorter opcode with implicit flag settings (e.g. the set status flag is always set)



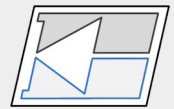
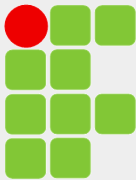
Densidade de código



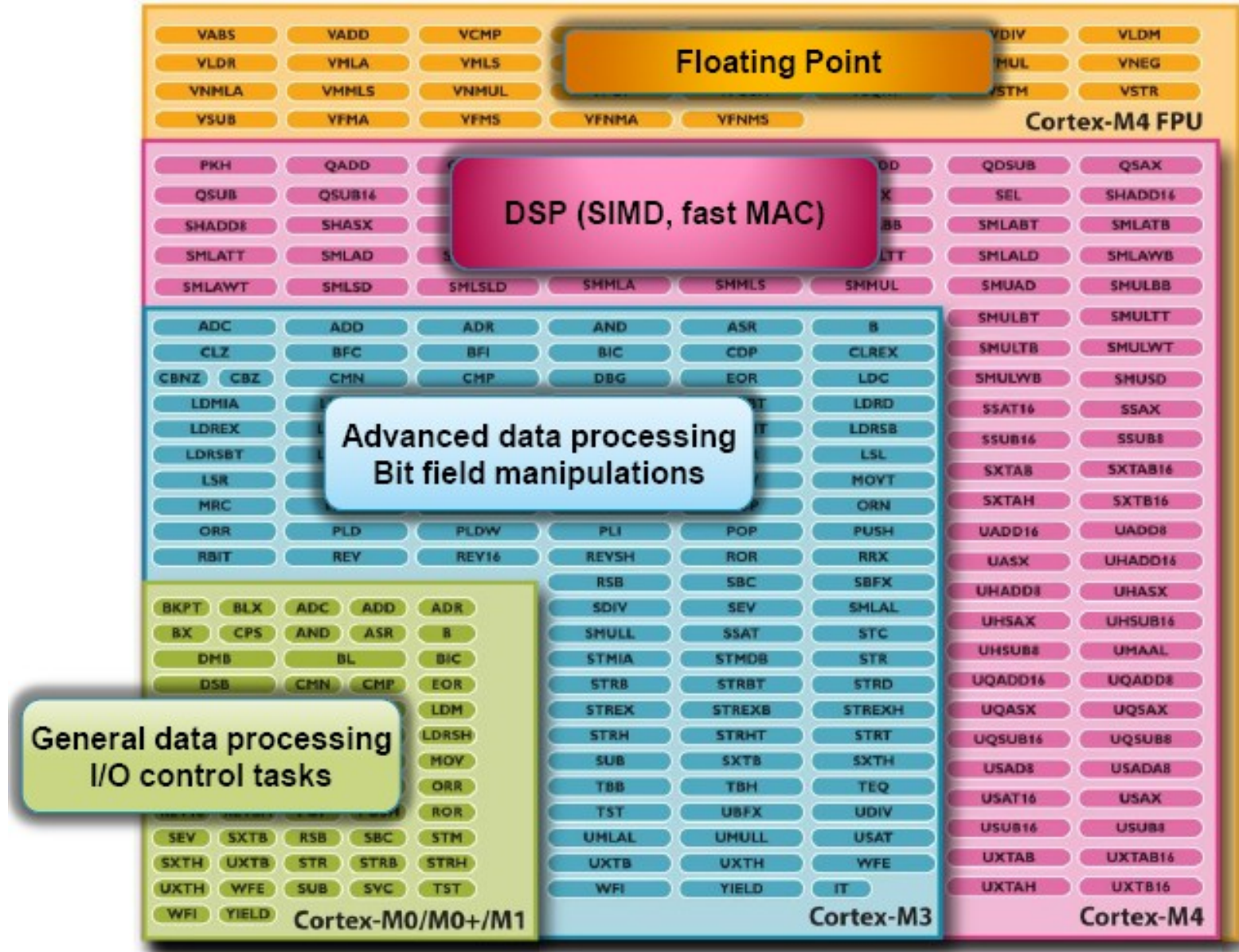
- Cortex-M shows smaller code size than 8/16-bit devices
- Consider a 16-bit multiply operation
 - Required for 10-bit ADC data filtering, encryption algorithms, audio

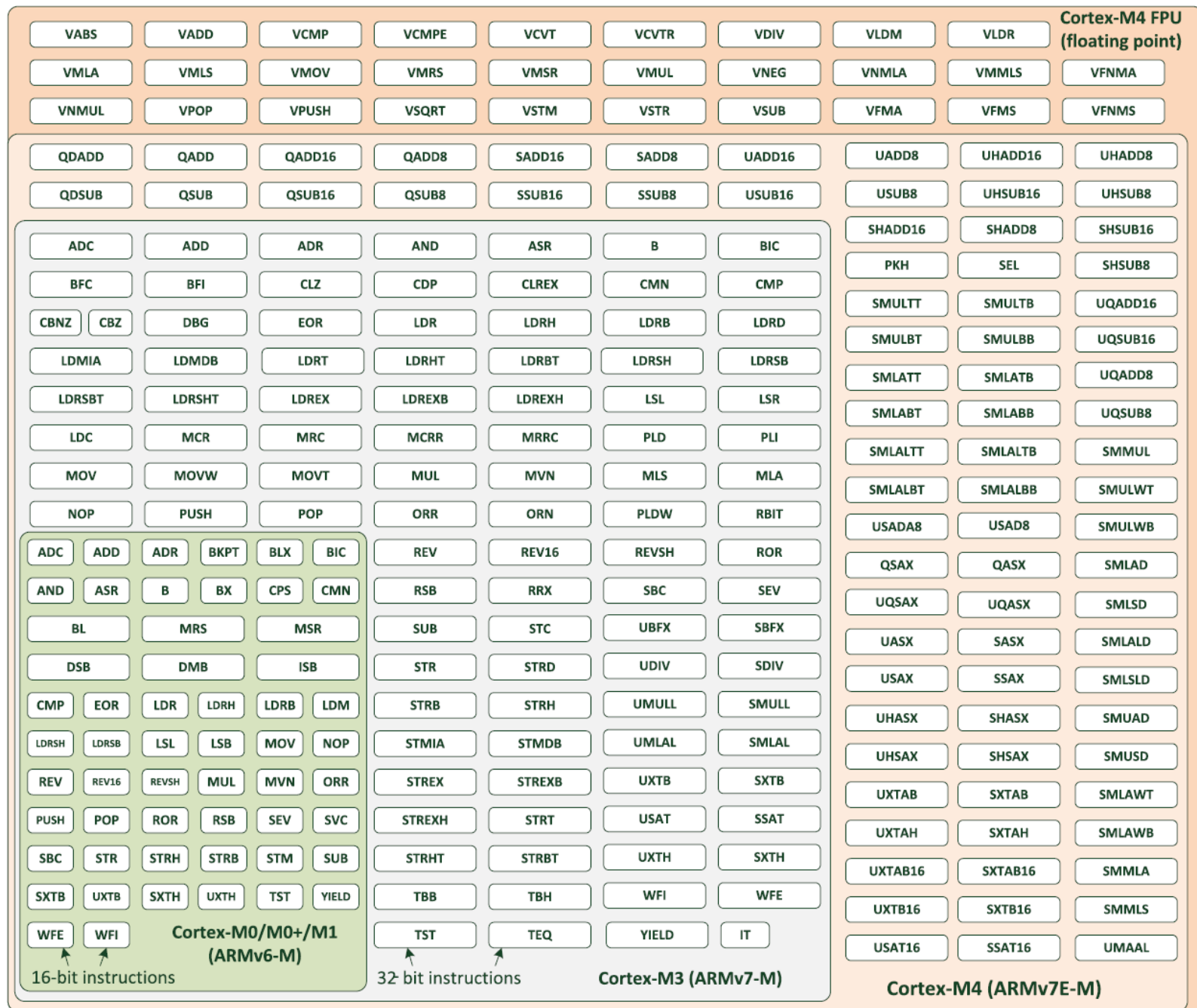
8-bit example (8051)		16-bit example	ARM Cortex-M
<pre>MOV A, XL ; 2 bytes MOV B, YL ; 3 bytes MUL AB; 1 byte MOV R0, A; 1 byte MOV R1, B; 3 bytes MOV A, XL ; 2 bytes MOV B, YH ; 3 bytes MUL AB; 1 byte ADD A, R1; 1 byte MOV R1, A; 1 byte MOV A, B ; 2 bytes ADDC A, #0 ; 2 bytes MOV R2, A; 1 byte MOV A, XH ; 2 bytes MOV B, YL ; 3 bytes</pre>	<pre>MUL AB; 1 byte ADD A, R1; 1 byte MOV R1, A; 1 byte MOV A, B ; 2 bytes ADDC A, R2 ; 1 bytes MOV R2, A; 1 byte MOV A, XH ; 2 bytes MOV B, YH ; 3 bytes MUL AB; 1 byte ADD A, R2; 1 byte MOV R2, A; 1 byte MOV A, B ; 2 bytes ADDC A, #0 ; 2 bytes MOV R3, A; 1 byte</pre>	<pre>MOV R1, &MulOp1 MOV R2, &MulOp2 MOV SumLo, R3 MOV SumHi, R4 (Memory mapped multiply unit)</pre>	<pre>MULS r0, r1, r0</pre>
Time: 48 clock cycles* Code size: 48 bytes		Time: 8 clock cycles Code size: 8 bytes	Time: 1 clock cycle Code size: 2 bytes

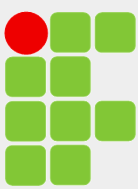
* 8051 needs at least one cycle per instruction byte fetch as they only have an 8-bit interface



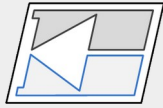
Instruções







Sufixo nas Instruções



Cada instrução de processamento de dados pode ter um código para execução condicional.

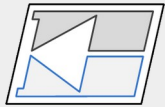
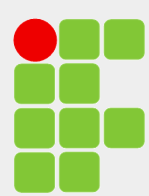
Resulta em um melhor fluxo de instruções através do pipeline.

A execução condicional aumenta a densidade do código e o

Normal	Conditional
CMP r3,#0	CMP r3,#0
BEQ skip	ADDNE r0,r1,r2
ADD r0,r1,r2	
skip	

Suffixes for Cortex-M Assembly Language

Suffixes	Descriptions
S	Update APSR (Application Program Status Register, such as Carry, Overflow, Zero and Negative flags); for example: ADD <u>S</u> R0, R1 ; this ADD operation will update APSR
EQ, NE, CS, CC, MI, PL, VS, VC, HI, LS, GE, LT, GT, LE	Conditional execution. EQ = Equal, NE = Not Equal, LT = Less Than, GT = Greater Than, etc. On the Cortex-M processors these conditions can be applied to conditional branches; for example: BEQ label ; Branch to label if previous operation result in ; equal status or conditionally executed instructions (IF-THEN instruction); for example: ADDEQ R0, R1, R2 ; Carry out the add operation if the previous ; operation results in equal status
.N, .W	Specify the use of 16-bit (narrow) instruction or 32-bit (wide) instruction.
.32, .F32	Specify the operation is for 32-bit single-precision data. In most toolchains, the .32 suffix is optional.
.64, F64	Specify the operation is for 64-bit double-precision data. In most toolchains, the .64 suffix is optional.



Sufixos nas Instruções

```
CMP R0, R1    ; Compare R0 and R1
ITTEE GT     ; If R0 > R1 Then (first 2 statements execute
               ; if true,
               ; other 2 statements execute if false)
MOVGT R2, R0 ;      R2 = R0
MOVGT R3, R1 ;      R3 = R1
MOVLE R2, R0 ; Else R2 = R1
MOVLE R3, R1 ;      R3 = R0
```

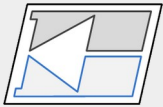
CMP	r0, r1	←	r0 - r1, compare r0 with r1 and set flags
ADD <u>GT</u>	r2, r2, #1	←	if > r2=r2+1 flags remain unchanged
ADD <u>LE</u>	r3, r3, #1	←	if <= r3=r3+1 flags remain unchanged

loop

ADD	r2, r2, r3	←	r2=r2+r3
SUB <u>S</u>	r1, r1, #0x01	←	decrement r1 and set flags
B <u>NE</u>	loop	←	if Z flag clear then branch

```
ADDS    R0, #1 ; Use 16-bit Thumb instruction by default
           ; for smaller size
ADDS.N R0, #1 ; Use 16-bit Thumb instruction (N=Narrow)
ADDS.W R0, #1 ; Use 32-bit Thumb-2 instruction (W=wide)
```

Comparação Assembly condicional

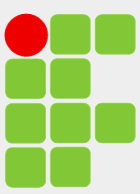


“Normal” Assembler

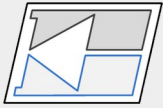
```
gcd    cmp r0, r1      ;reached the end?
        beq stop
        blt less       ;if r0 > r1
        sub r0, r0, r1  ;subtract r1 from r0
        bal gcd
less   sub r1, r1, r0   ;subtract r0 from r1
        bal gcd
stop
```

ARM Conditional Assembler

```
gcd    cmp    r0, r1      ;if r0 > r1
        subgt r0, r0, r1  ;subtract r1 from r0
        sublt r1, r1, r0  ;else subtract r0 from r1
        bne   gcd         ;reached the end?
```

Exemplo



C source code

```
if (r0 == 0)
{
    r1 = r1 + 1;
}
else
{
    r2 = r2 + 1;
}
```

ARM instructions

unconditional

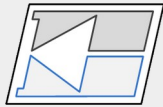
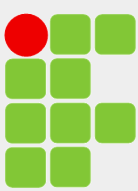
```
CMP r0, #0
BNE else
ADD r1, r1, #1
B end
else
    ADD r2, r2, #1
end
...
```

- 5 instructions
- 5 words
- 5 or 6 cycles

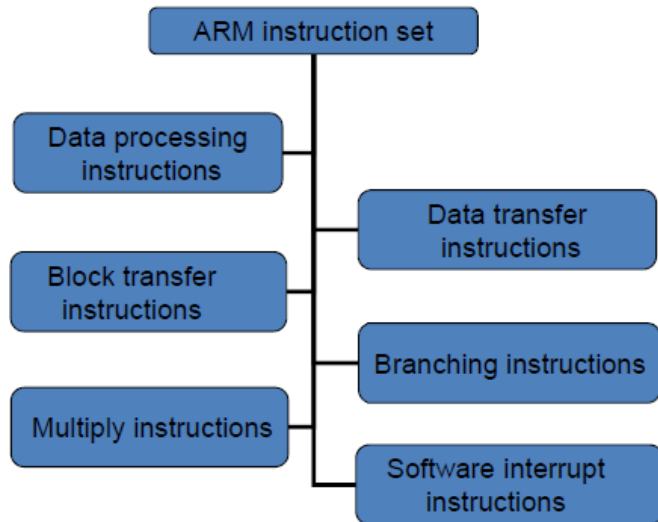
conditional

```
CMP r0, #0
ADDEQ r1, r1, #1
ADDNE r2, r2, #1
...
```

- 3 instructions
- 3 words
- 3 cycles

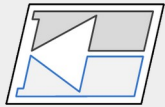
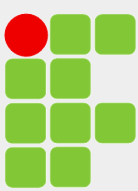


Instruções M3



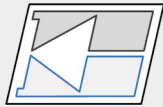
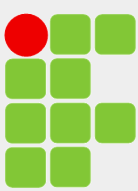
- angle brackets, $\langle \rangle$, enclose alternative forms of the operand
- braces, $\{ \}$, enclose optional operands
- the Operands column is not exhaustive
- Op2 is a flexible second operand that can be either a register or a constant
- most instructions can use an optional condition code suffix.

Mnemonic	Operands	Brief description
ADC, ADCS	{Rd,} Rn, Op2	Add with Carry
ADD, ADDS	{Rd,} Rn, Op2	Add
ADD, ADDW	{Rd,} Rn, #imm12	Add
ADR	Rd, label	Load PC-relative Address
AND, ANDS	{Rd,} Rn, Op2	Logical AND
ASR, ASRS	Rd, Rm, <Rs n>	Arithmetic Shift Right
B	label	Branch
BFC	Rd, #lsb, #width	Bit Field Clear
BFI	Rd, Rn, #lsb, #width	Bit Field Insert
BIC, BICS	{Rd,} Rn, Op2	Bit Clear
BKPT	#imm	Breakpoint



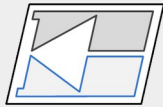
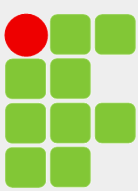
Instruções

BL	label	Branch with Link
BLX	Rm	Branch indirect with Link
BX	Rm	Branch indirect
CBNZ	Rn, label	Compare and Branch if Non Zero
CBZ	Rn, label	Compare and Branch if Zero
CLREX	-	Clear Exclusive
CLZ	Rd, Rm	Count Leading Zeros
CMN	Rn, Op2	Compare Negative
CMP	Rn, Op2	Compare
CPSID	i	Change Processor State, Disable Interrupts
CPSIE	i	Change Processor State, Enable Interrupts
DMB	-	Data Memory Barrier
DSB	-	Data Synchronization Barrier
EOR, EORS	{Rd,} Rn, Op2	Exclusive OR
ISB	-	Instruction Synchronization Barrier
IT	-	If-Then condition block



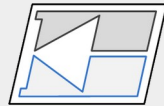
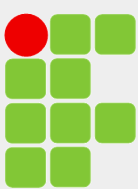
Instruções

LDM	Rn{!}, reglist	Load Multiple registers, increment after
LDMDB, LDMEA	Rn{!}, reglist	Load Multiple registers, decrement before
LDMFD, LDMIA	Rn{!}, reglist	Load Multiple registers, increment after
LDR	Rt, [Rn, #offset]	Load Register with word
LDRB, LDRBT	Rt, [Rn, #offset]	Load Register with byte
LDRD	Rt, Rt2, [Rn, #offset]	Load Register with two bytes
LDREX	Rt, [Rn, #offset]	Load Register Exclusive
LDREXB	Rt, [Rn]	Load Register Exclusive with Byte
LDREXH	Rt, [Rn]	Load Register Exclusive with Halfword
LDRH, LDRHT	Rt, [Rn, #offset]	Load Register with Halfword
LDRSB, LDRSBT	Rt, [Rn, #offset]	Load Register with Signed Byte
LDRSH, LDRSHT	Rt, [Rn, #offset]	Load Register with Signed Halfword
LDRT	Rt, [Rn, #offset]	Load Register with word
LSL, LSLS	Rd, Rm, <Rs #n>	Logical Shift Left
LSR, LSRS	Rd, Rm, <Rs #n>	Logical Shift Right
MLA	Rd, Rn, Rm, Ra	Multiply with Accumulate, 32-bit result



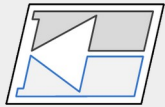
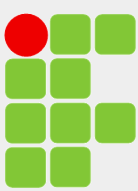
Instruções

MLS	Rd, Rn, Rm, Ra	Multiply and Subtract, 32-bit result
MOV, MOVS	Rd, Op2	Move
MOVT	Rd, #imm16	Move Top
MOVW, MOV	Rd, #imm16	Move 16-bit constant
MRS	Rd, spec_reg	Move from Special Register to general register
MSR	spec_reg, Rm	Move from general register to Special Register
MUL, MULS	{Rd,} Rn, Rm	Multiply, 32-bit result
MVN, MVNS	Rd, Op2	Move NOT
NOP	-	No Operation
ORN, ORNS	{Rd,} Rn, Op2	Logical OR NOT
ORR, ORRS	{Rd,} Rn, Op2	Logical OR
POP	reglist	Pop registers from stack
PUSH	reglist	Push registers onto stack
RBIT	Rd, Rn	Reverse Bits
REV	Rd, Rn	Reverse byte order in a word
REV16	Rd, Rn	Reverse byte order in each halfword



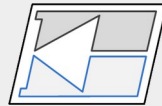
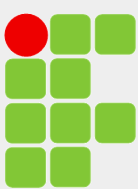
Instruções

REVSH	Rd, Rn	Reverse byte order in bottom halfword and sign extend
ROR, RORS	Rd, Rm, <Rs #n>	Rotate Right
RRX, RRXS	Rd, Rm	Rotate Right with Extend
RSB, RSBS	{Rd,} Rn, Op2	Reverse Subtract
SBC, SBCS	{Rd,} Rn, Op2	Subtract with Carry
SBFX	Rd, Rn, #lsb, #width	Signed Bit Field Extract
SDIV	{Rd,} Rn, Rm	Signed Divide
SEV	-	Send Event
SMLAL	RdLo, RdHi, Rn, Rm	Signed Multiply with Accumulate (32 x 32 + 64), 64-bit result
SMULL	RdLo, RdHi, Rn, Rm	Signed Multiply (32 x 32), 64-bit result
SSAT	Rd, #n, Rm {,shift #s}	Signed Saturate
STM	Rn{!}, reglist	Store Multiple registers, increment after
STMDB, STMEA	Rn{!}, reglist	Store Multiple registers, decrement before
STMFD, STMIA	Rn{!}, reglist	Store Multiple registers, increment after
STR	Rt, [Rn, #offset]	Store Register word



Instruções

STRB, STRBT	Rt, [Rn, #offset]	Store Register byte
STRD	Rt, Rt2, [Rn, #offset]	Store Register two words
STREX	Rd, Rt, [Rn, #offset]	Store Register Exclusive
STREXB	Rd, Rt, [Rn]	Store Register Exclusive Byte
STREXH	Rd, Rt, [Rn]	Store Register Exclusive Halfword
STRH, STRHT	Rt, [Rn, #offset]	Store Register Halfword
STRT	Rt, [Rn, #offset]	Store Register word
SUB, SUBS	{Rd,} Rn, Op2	Subtract
SUB, SUBW	{Rd,} Rn, #imm12	Subtract
SVC	#imm	Supervisor Call
SXTB	{Rd,} Rm {,ROR #n}	Sign extend a byte
SXTH	{Rd,} Rm {,ROR #n}	Sign extend a halfword
TBB	[Rn, Rm]	Table Branch Byte
TBH	[Rn, Rm, LSL #1]	Table Branch Halfword
TEQ	Rn, Op2	Test Equivalence
TST	Rn, Op2	Test



Instruções

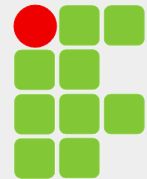
UBFX	Rd, Rn, #lsb, #width	Unsigned Bit Field Extract
UDIV	{Rd,} Rn, Rm	Unsigned Divide
UMLAL	RdLo, RdHi, Rn, Rm	Unsigned Multiply with Accumulate (32 x 32 + 64), 64-bit result
UMULL	RdLo, RdHi, Rn, Rm	Unsigned Multiply (32 x 32), 64-bit result
USAT	Rd, #n, Rm {,shift #s}	Unsigned Saturate
UXTB	{Rd,} Rm {,ROR #n}	Zero extend a Byte
UXTH	{Rd,} Rm {,ROR #n}	Zero extend a Halfword
WFE	-	Wait For Event
WFI	-	Wait For Interrupt

Exemplos:

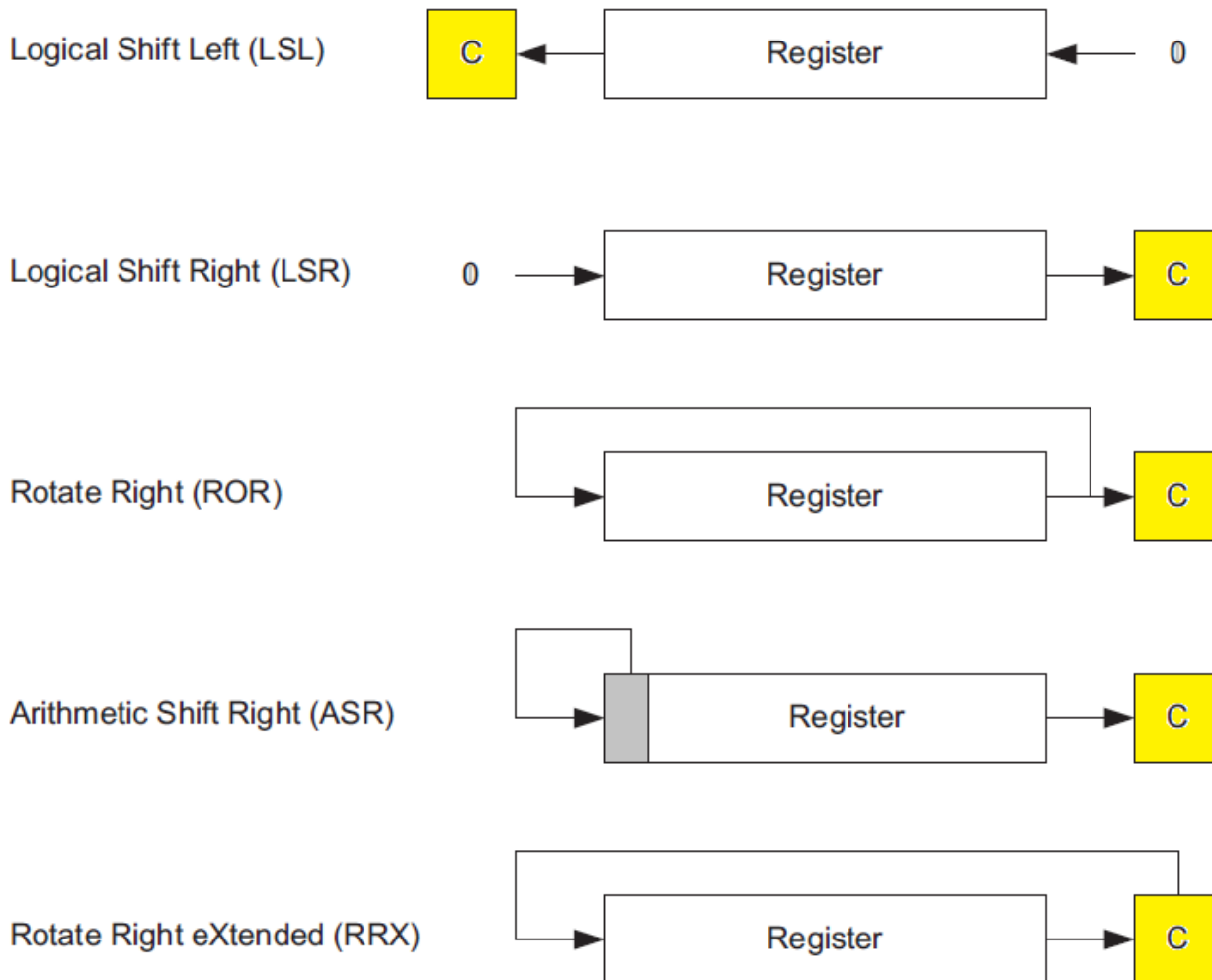
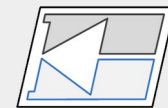
```
PUSH {R0, R4-R7, R9} ; PUSH R0, R4, R5, R6, R7, R9 into stack
POP  {R2, R3}        ; POP R2 and R3 from stack

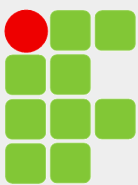
MOVW R0, #0x789A ; Set R0 to 0x0000789A
MOVT R0, #0x3456 ; Set upper 16-bit of R0 to 0x3456,
                  ; now R0 = 0x3456789A

LDR R3, [R0, R2, LSL #2] ; Read memory[R0+(R2 << 2)] into R3
```

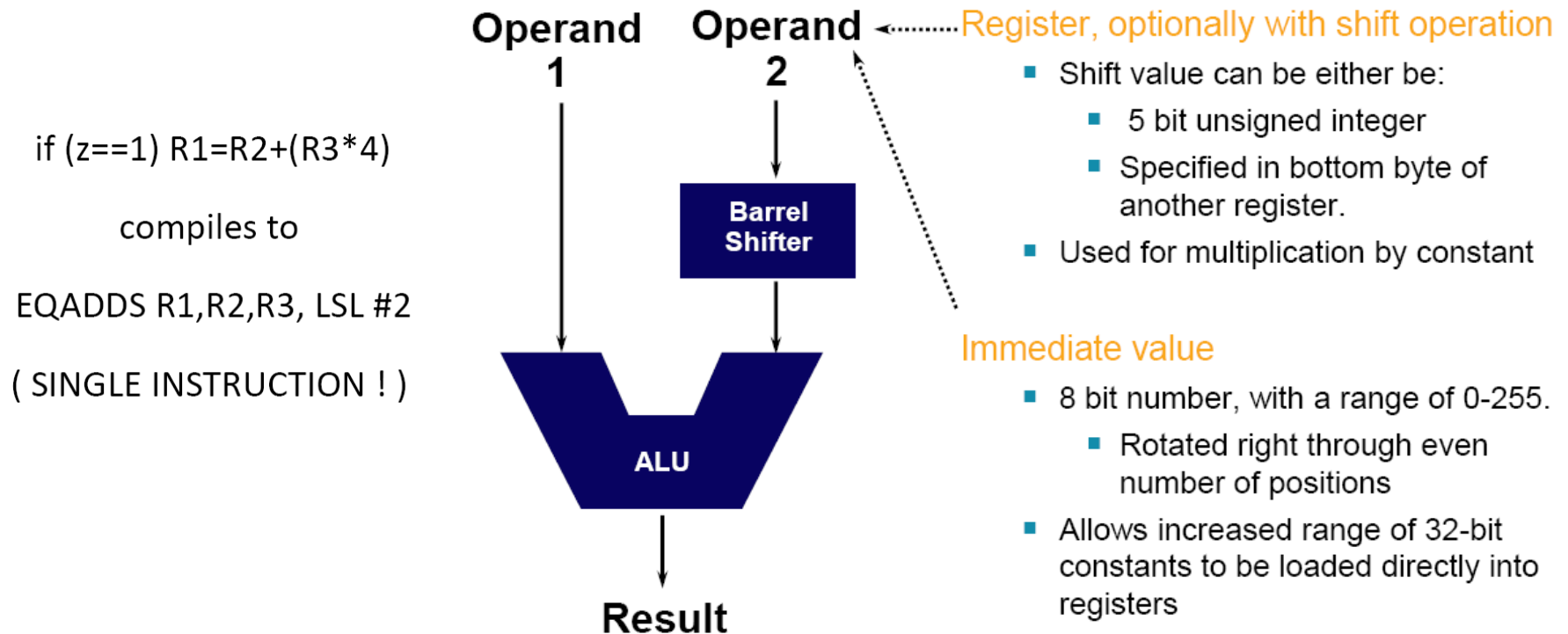
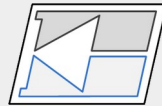


Instruções de Deslocamento e Rotação

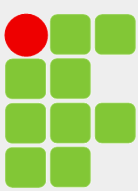




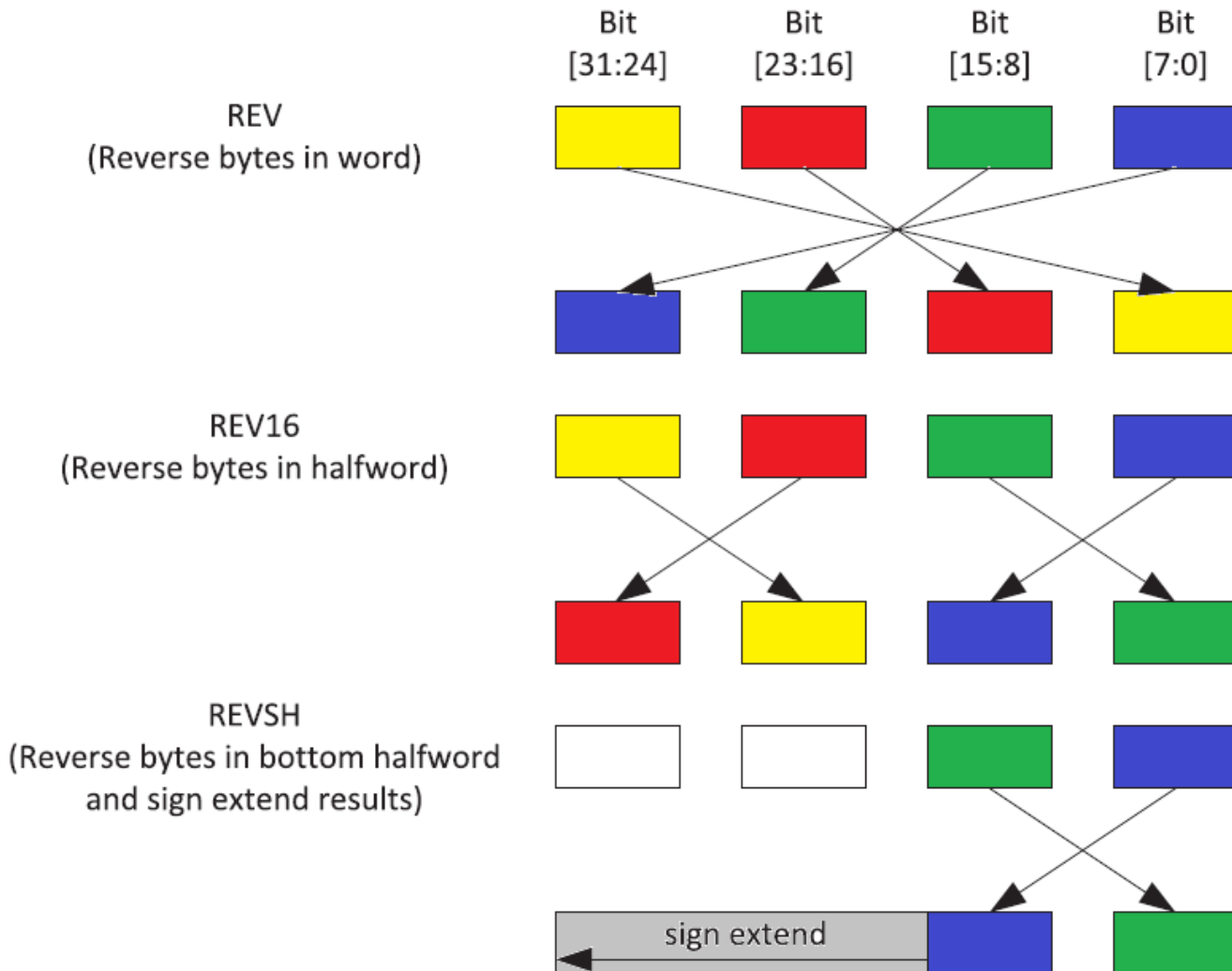
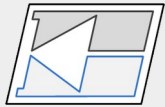
Barrel Shifter

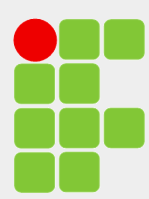


Eficiente codificação de programas de alto nível:
códigos condicionais + instruções de processamento de dados + *barrel shifter*.

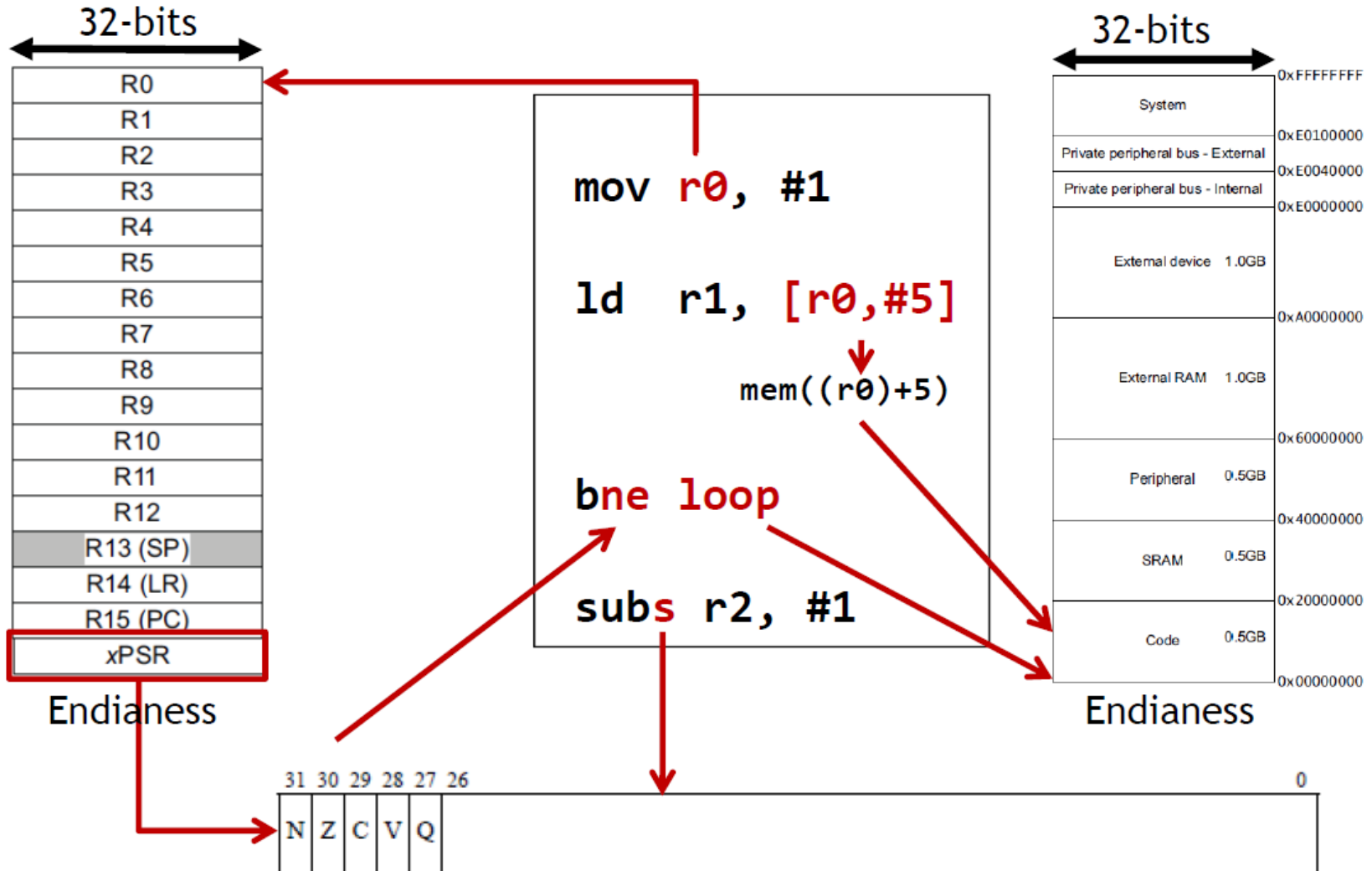
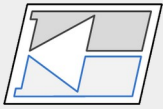


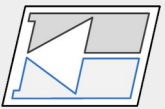
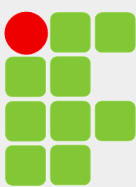
Operações de Reversão





Exemplos de Código



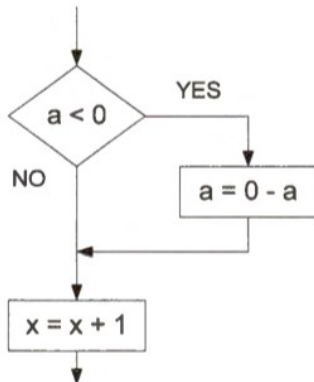


Exemplos de Código

C Program	Assembly Program
<pre>uint32_t x, y, z; x = 0x00000001; y = 0xFFFFFFFF; if (x > y) z = 1; else z = 0;</pre>	<pre>MOV r5, #0x00000001 ; r5 = x MOV r6, #0xFFFFFFFF ; r6 = y CMP r5, r6 BLS else ; branch if ≤ then MOV r7, #1 ; z = 1 B endif ; skip next instruction else MOV r7, #0 ; z = 0 endif</pre>

Example 6-1. Implementation of *if*-statement that compares two unsigned integers

```
C Program  
if (a < 0) {  
    a = 0 - a;  
}  
x = x + 1;
```

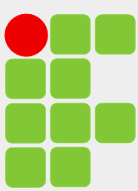


```
; r1 = a, r2 = x  
CMP r1, #0           ; Compare a with 0  
BGE endif            ; Go to endif if a ≥ 0  
then RSB r1, r1, #0   ; a = - a  
endif ADD r2, r2, #1  ; x = x + 1
```

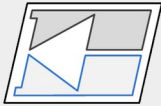
Example 6-3. An *if-then* statement can be implemented by using a conditional branch.

```
; r1 = a, r2 = x  
CMP r1, #0           ; Compare a with 0  
RSBLT r1, r1, #0     ; a = 0 - a if a < 0  
ADD r2, r2, #1       ; x = x + 1
```

Example 6-4. An *if-then* statement can be carried out by using conditional execution.



Exemplos de Código

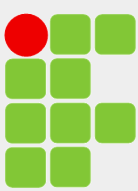


C Program	Assembly Program
<pre>// x is a signed integer if(x <= 20 x >= 25){ a = 1; }</pre>	<pre>; r0 = x, r1 = a CMP r0, #20 ; compare x and 20 BLE then ; go to then if x <= 20 CMP r0, #25 ; compare x and 25 BLT endif ; go to endif if x < 25 then MOV r1, #1 ; a = 1 endif</pre>

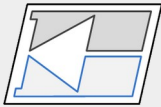
Example 6-5. A generic approach to implementing *if-then* with a compound logical OR

C Program	Assembly Program
<pre>// x is a signed integer if(x <= 20 x >= 25){ a = 1; }</pre>	<pre>; r0 = x, r1 = a CMP r0, #20 ; compare x and 20 MOVLE r1, #1 ; a = 1 if x <= 20 CMP r0, #25 ; compare x and 25 MOVGE r1, #1 ; a = 1 if x >= 25</pre>

Example 6-6. Using Conditional execution to implement a compound logical OR.

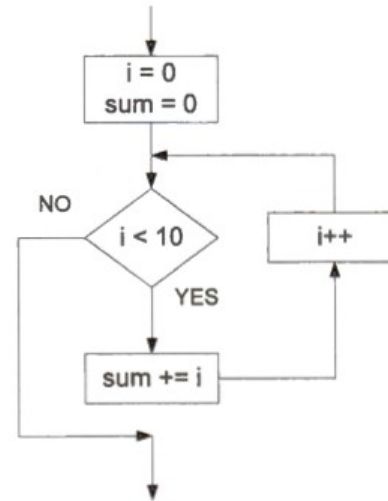


Exemplos de Código



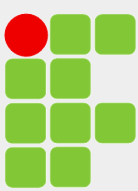
C Program

```
int i;  
int sum = 0;  
  
for(i = 0; i < 10; i++){  
    sum += i;  
}
```

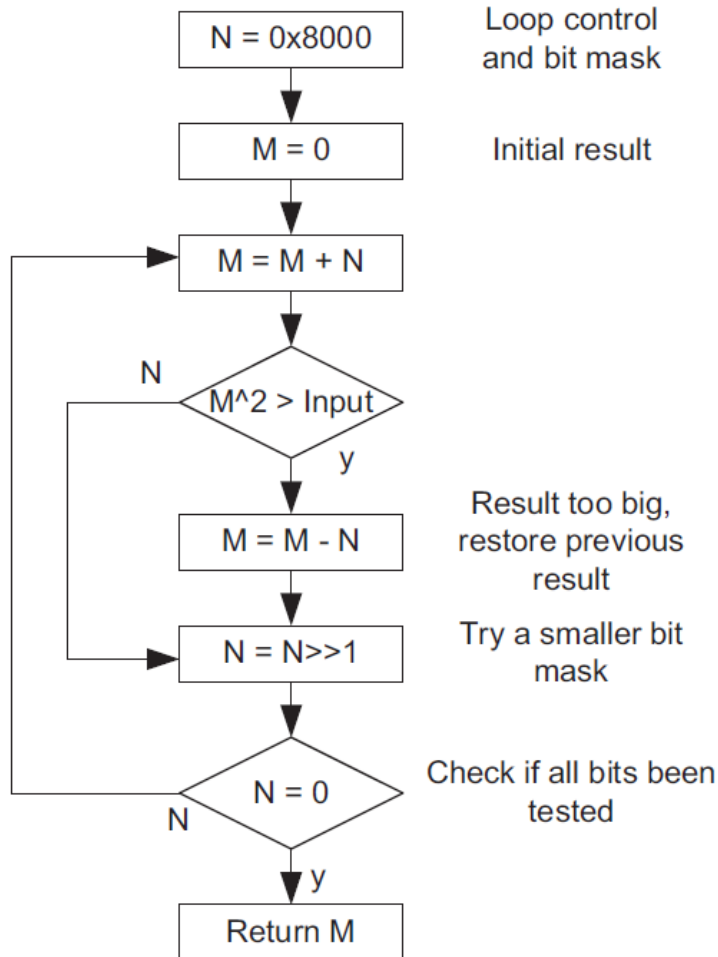
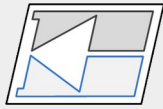


Assembly Program 1	Assembly Program 2	Assembly Program 3
<pre>MOV r0, #0 ; i MOV r1, #0 ; sum B check loop ADD r1, r1, r0 ADD r0, r0, #1 check CMP r0, #10 BLT loop endloop</pre>	<pre>MOV r0, #0 ; i MOV r1, #0 ; sum loop CMP r0, #10 BGE endloop ADD r1, r1, r0 ADD r0, r0, #1 B loop endloop</pre>	<pre>MOV r0, #0 ; i MOV r1, #0 ; sum loop CMP r0, #10 ADDLT r1, r1, r0 ADDLT r0, r0, #1 BLT loop endloop</pre>

Example 6-11. Assembly implementation of *for* loop based on conditional branch and conditional execution



Exemplos de Código



simple_sqrt FUNCTION

; Input : R0

; Output : R0 (square root result)

MOVW R1, #0x8000 ; R1 = 0x00008000

MOVS R2, #0 ; Initialize result

simple_sqrt_loop

ADDS R2, R2, R1 ; $M = (M + N)$

MULS R3, R2, R2 ; $R3 = M^2$

CMP R3, R0 ; If $M^2 > \text{Input}$

IT HI ; Greater Than

SUBHI R2, R2, R1 ; $M = (M - N)$

LSRS R1, R1, #1 ; $N = N \gg 1$

BNE simple_sqrt_loop

MOV R0, R2 ; Copy to R0 and return

BX LR ; Return

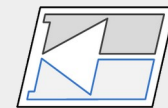
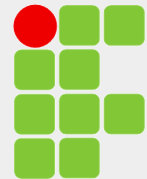
ENDFUNC

```
; Hello World program
```

```
PRESERVE8 ; Indicate the code here preserve
           ; 8 byte stack alignment
THUMB     ; Indicate THUMB code is used
AREA |.text|, CODE, READONLY ; Start of CODE area
EXPORT __main
ENTRY
__main FUNCTION
    MOVS R0, #'\\n'
    BL Putc
MainLoop
    LDR R0,=HELLO_TXT ; Get address of the string
    BL Puts           ; Display string
    BL Delay          ; Delay
    B MainLoop
ENDFUNC
;-----
Puts FUNCTION
    ; Subroutine to send string to display
    ; Input R0 = starting address of string.
    ; The string should be null terminated
    PUSH {R4, LR} ; Save registers
    MOVs R4, R0
    ; Copy address to R1, because R0 will
    ; be used
PutsLoop ; as input for Putc
    LDRB R0,[R4],#1
    ; Read one character and increment address (post index)
    CBZ R0, PutsLoopExit ; if character is null, goto end
    BL Putc ; Output character to UART
    B PutsLoop ; Next character
PutsLoopExit
    POP {R4, PC} ; Return
ENDFUNC
;-----
; Send a character via ITM
ITM_BASE EQU 0xE0000000
ITM_PORT0 EQU (ITM_BASE+0x000)
```

```
ITM_TER EQU (ITM_BASE+0xE00)
ITM_TCR EQU (ITM_BASE+0xE80)

Putc FUNCTION
    ; Function to display one character
    ; Input R0 - character to be displayed
    LDR R1,=ITM_TCR ; 0xE0000E80
    LDR R2,[R1]
    MOVs R3, #1 ; Check ITMENA bit. If 0, exit
    TST R2, R3
    BEQ PutcExit
    LDR R1,=ITM_TER ; 0xE0000E00
    LDR R2,[R1] ; Check Port 0 is enabled. If 0, exit
    TST R2, R3
    BEQ PutcExit
    LDR R1,=ITM_PORT0 ; 0xE0000000
PutcWait
    LDR R2,[R1] ; Read status
    CMP R2, #0
    BEQ PutcWait
    STRB R0,[R1] ; Write a character
PutcExit
    BX LR
ENDFUNC
;-----
Delay FUNCTION
    LDR R0, =0x01000000
DelayLoop
    SUBS R0, R0, #1
    BNE DelayLoop
    BX LR
ENDFUNC
;-----
; Text to be displayed
HELLO_TXT
    DCB "Hello world\\n", 0 ; Null terminated string
    ALIGN 4
END ; End of file
```



Cortex M4 - Float Point

In C programming, you can define data as floating point values. For example, a value can be declared as single precision:

```
float pi = 3.141592F;
```

or double precision:

```
double pi = 3.1415926535897932384626433832795;
```

Floating point data allows the processor to handle a much wider data range (compared to integers or fixed point data) as well as very small values.

M4

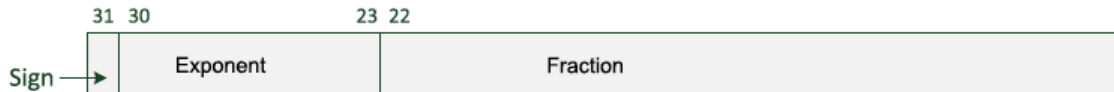


FIGURE 13.1

Single-precision data format

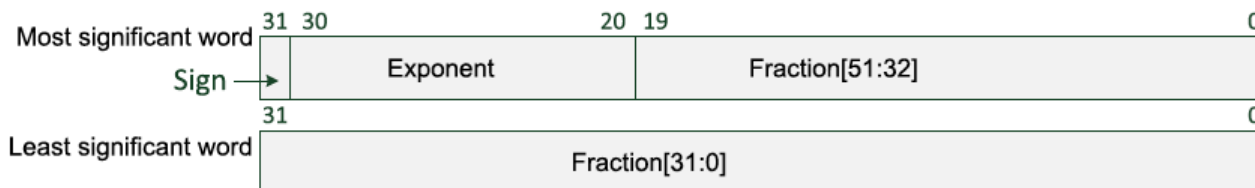
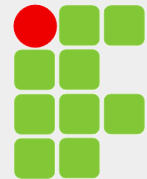
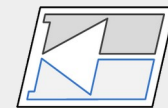


FIGURE 13.8

Double-precision data format

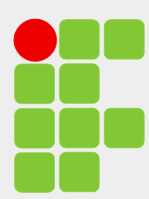


Cortex M4 - Float Point

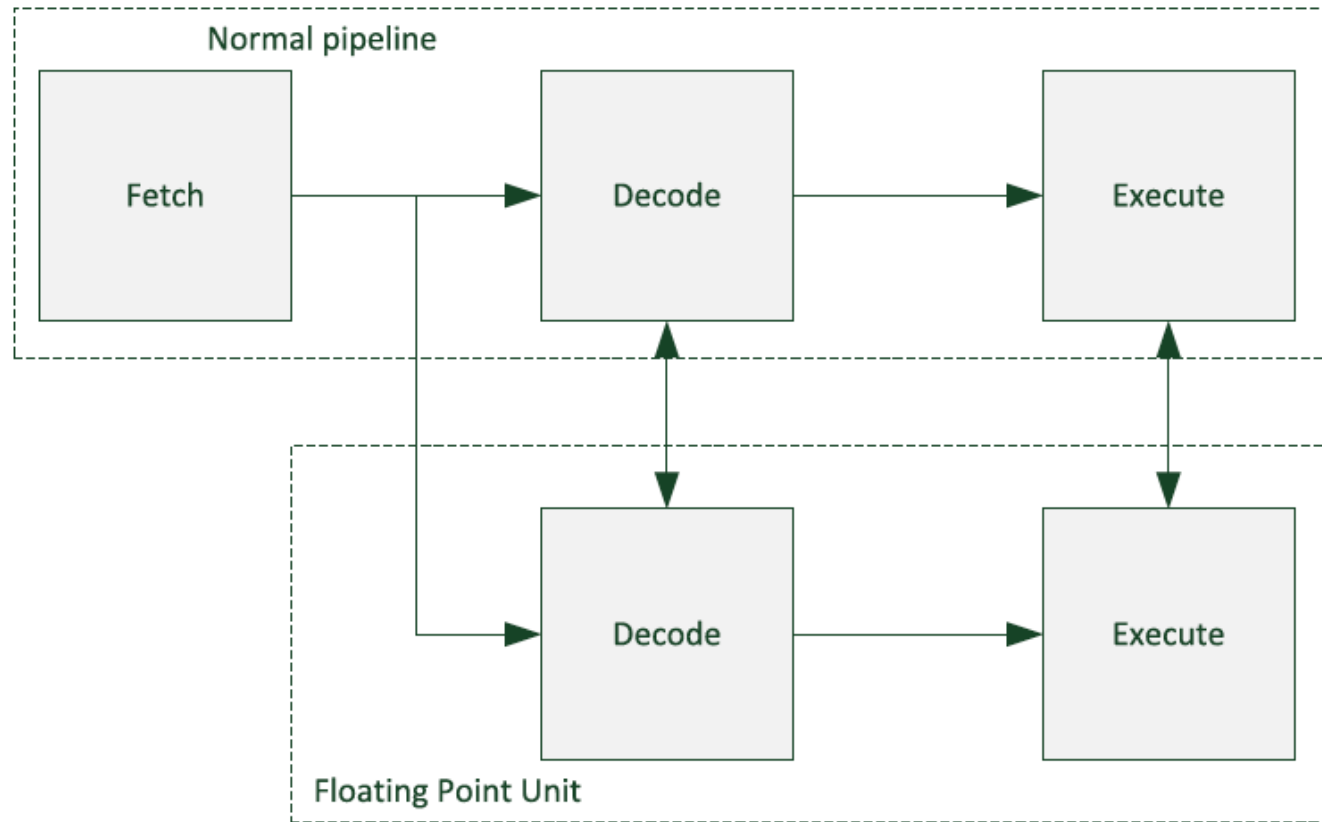
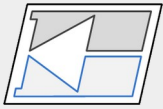


The floating point design supports:

- A floating point register bank that contains thirty-two 32-bit registers, which can be used as 32 registers, or be used in pairs as 16 double-word registers.
- Single-precision floating point calculations
- Conversion instructions for:
 - “integer $\leftrightarrow$ single-precision floating point”
 - “fixed point $\leftrightarrow$ single-precision floating point”
 - “half precision $\leftrightarrow$ single-precision floating point”
- Data transfers of single-precision and double-word data between floating point register bank and memory
- Data transfer of single-precision between floating point register bank and integer register bank



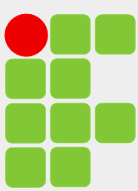
Cortex M4 - Float Point



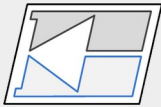
Cortex-M4

FIGURE 13.11

Co-processor pipeline concept



Cortex M4 - Float Point



Possui 32
registradores
organizados em 16
registradores de 64
bits.

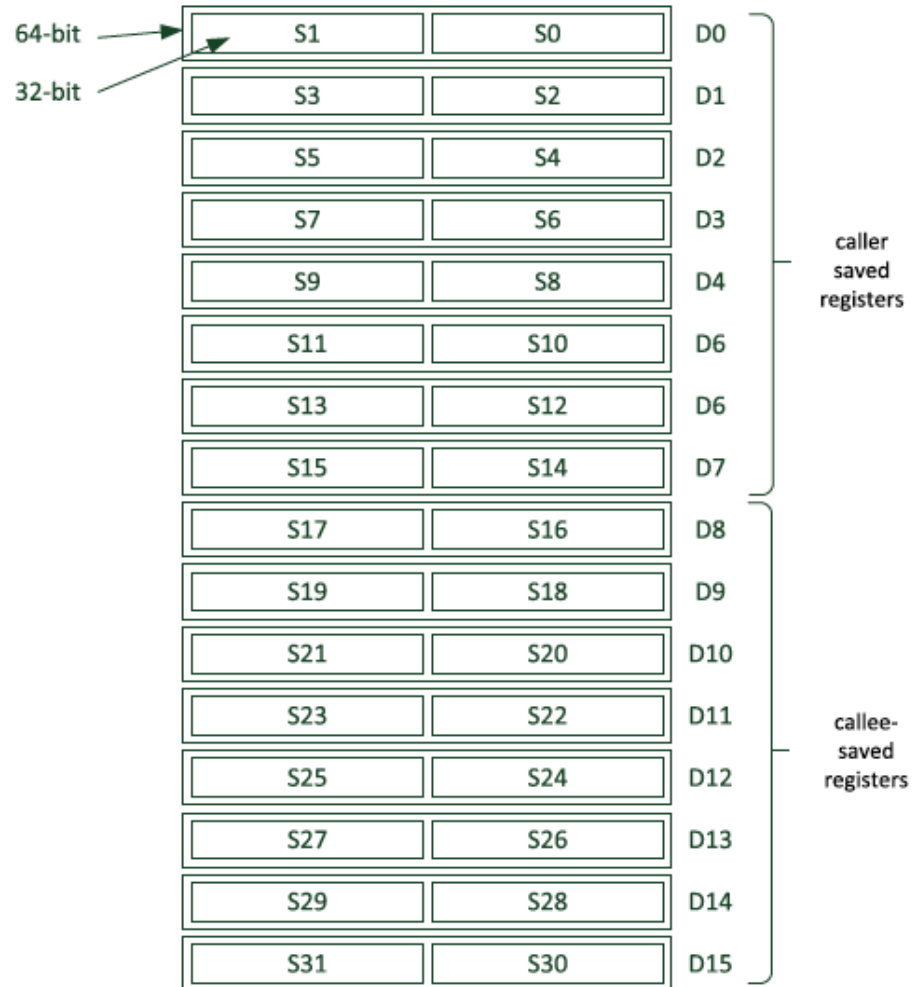
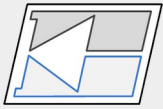
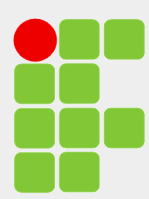


FIGURE 13.13

Floating point register bank



Cortex M4 - DSP

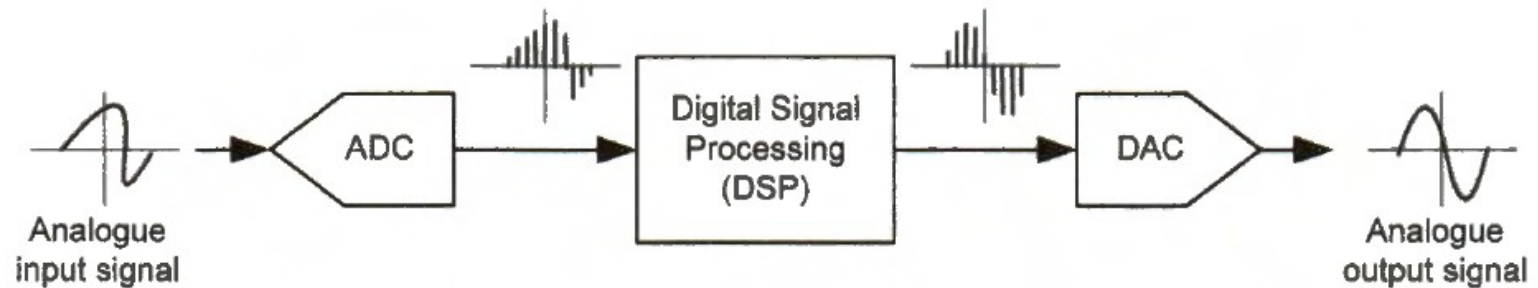
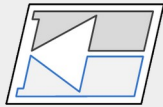
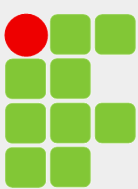


Figure 24-1. Common diagram of a DSP system

The recent boom in connected devices creates a need for products with both microcontroller and DSP features. This is evident in devices that handle multimedia content. They require both peripheral connectivity and some amount of DSP processing capability. Traditionally, these devices had two separate processors – a microcontroller and a DSP. However, with DSP extensions, many multimedia devices can be built around a single Cortex[®]-M4 processor. Therefore, the Cortex-M4 processor with DSP extensions solves the limitations of both traditional DSPs and microcontrollers. This yields lower power consumption, ease of integration, and most importantly, lower overall system cost.



24.2 Fixed-point Data Types in DSP

As discussed in Chapter 12, signed fixed-point numbers are denoted as Q_n , where n is the number of fractional bits. A Q_n number is a signed $(n + 1)$ -bit two's complement integer, with the fixed-point value represented as that integer divided by 2^n .

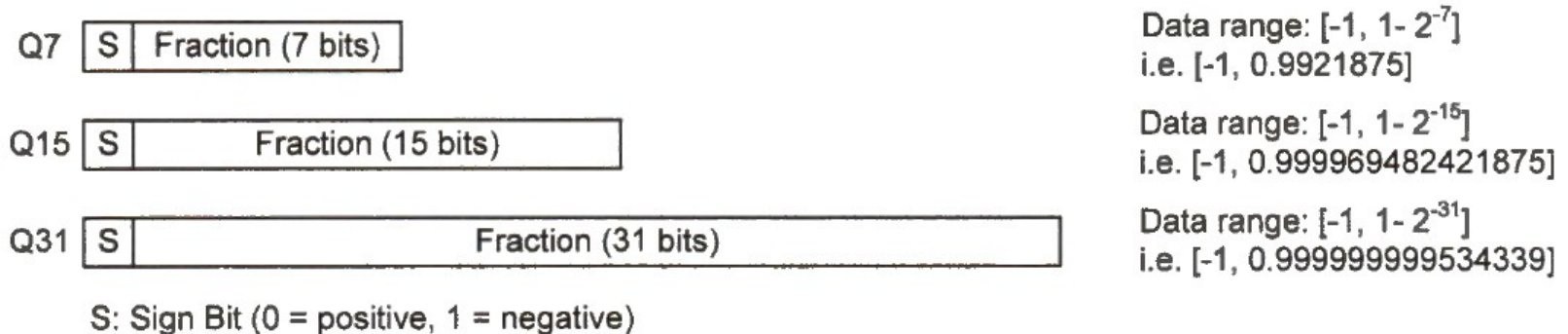
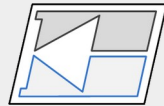
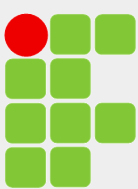


Figure 24-2. Signed fixed-point format Q7, Q15, and Q31



Cortex M4 - DSP

To use the Cortex-M4 DSP instructions, start by including the main CMSIS file `core_cm4.h`. This defines a variety of integer, floating-point, and fractional data types as shown in the table below:

Signed Integers

```
int8_t   8-bit  
int16_t  16-bit  
int32_t  32-bit  
int64_t  64-bit
```

Unsigned Integers

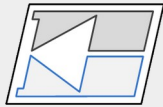
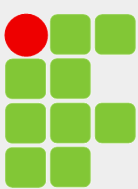
```
uint8_t  8-bit  
uint16_t 16-bit  
uint32_t 32-bit  
uint64_t 64-bit
```

Floating-Point

```
float32_t single precision 32-bit  
float64_t double precision 64-bit
```

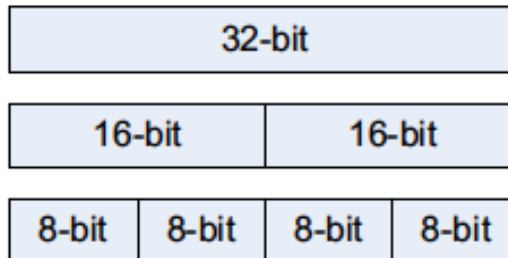
Fractional

```
q7_t     8-bit  
q15_t    16-bit  
q31_t    32-bit  
q63_t    64-bit
```



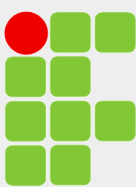
21.4.3 SIMD data

The Cortex[®]-M4 also provides SIMD instructions that operate on packed 8- or 16-bit integers. A 32-bit register can hold either 1×32 -bit value, 2×16 -bit values, or 4×8 -bit values as illustrated below:

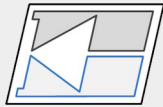


Instructions which operate on 8- or 16-bit data types are useful for processing data, such as video or audio, which do not require full 32-bit precision.

To use the SIMD instructions from C code, you load values into `int32_t` variables and then invoke the corresponding SIMD intrinsic instructions.



Cortex M4 - DSP



SADD8 Rd, Rn, Rm

$$C3 = A3 + B3$$

$$C2 = A2 + B2$$

$$C1 = A1 + B1$$

$$C0 = A0 + B0$$

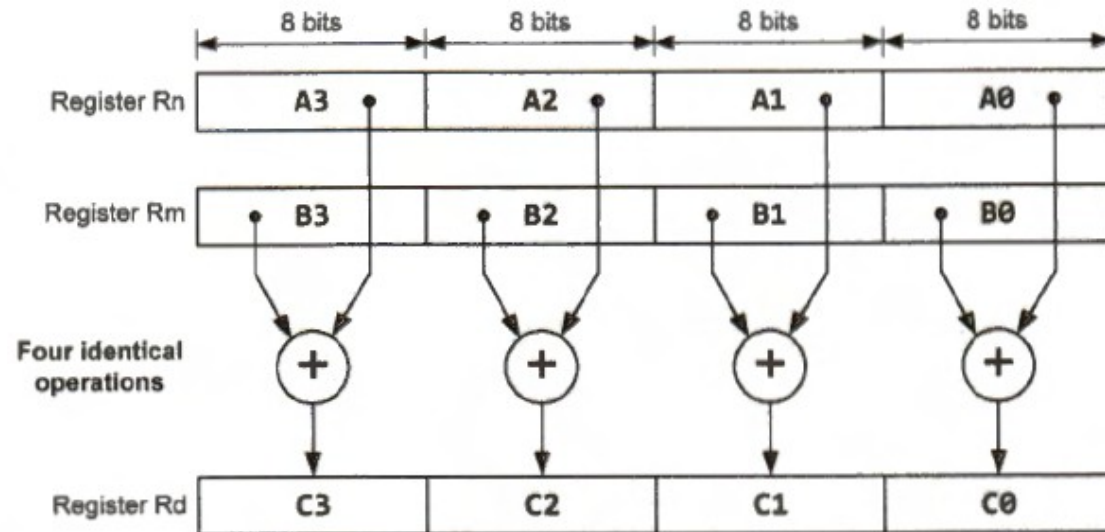
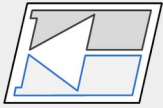
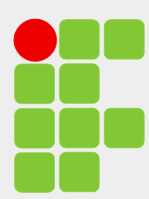


Figure 24-8. Example SIMD instruction (SADD8) that adds four pairs of 8-bit signed integers in parallel. The results are packed into one destination register (Rd).



Saturação

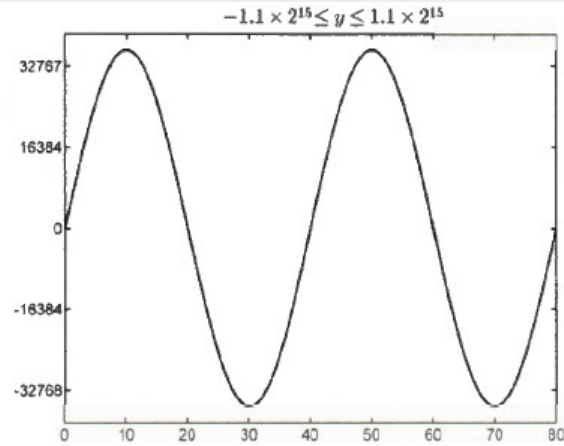


Figure 24-4. Equivalent integer values of a signal. Overflow and underflow occur if represented in Q15

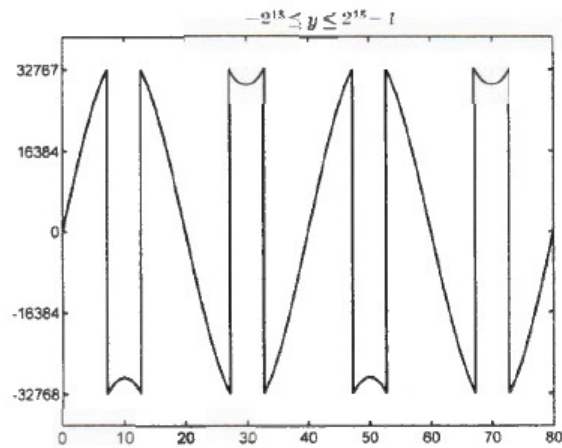


Figure 24-5. Wrapped-around output if there is no saturation

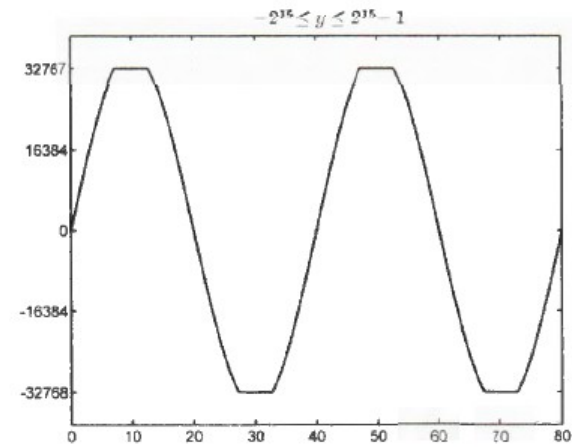
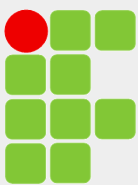
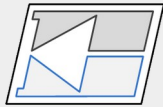


Figure 24-6. Saturated output



CONCLUSÃO



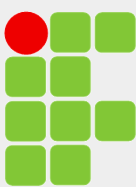
A tecnologia ARM é bem sucedida nas mais diferentes áreas tecnológicas devido as suas excelentes características. A arquitetura é projetada pensando no máximo desempenho, o que a faz dominante no segmento dos processadores digitais.

Os μ controladores Cortex-M3 possuem:

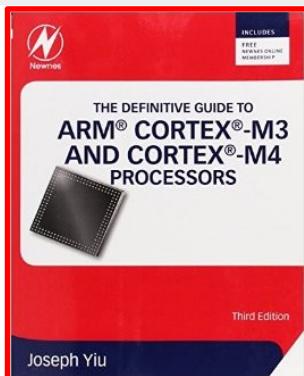
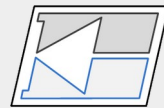
- ✓ Suporte a SOs (MPU, NVIC e 2 modos de operação).
- ✓ Multiplicação e divisão por hardware.
- ✓ Acesso individual a bits (Bit Band).
- ✓ Vários modos de economia de energia.
- ✓ Densidade de Código (Thumb2).
- ✓ Eficiência (instruções compostas) executadas em um ou poucos ciclos de máquina.
- ✓ Inúmeros fabricantes e ferramentas de desenvolvimento.

Os μ controladores Cortex-M4 possuem:

- ✓ Instruções para DSP e podem ter ponto flutuante.

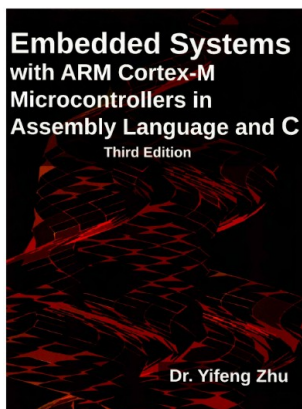


Bibliografia



Yiu, Joseph. ***The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors***. 3a ed. Elsevier, 2014.

Zhu, Yifeng, ***Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C***. 3ed. E-Man Press LLC, 2017.



SLOSS, Andrew N, Dominic Symes and Chris Wright. ***ARM System Developer's Guide: Designing and Optimizing System Software***. 1ed. Elsevier. 2004.

Cortex-M3 Devices Generic User Guide – ARM, 2010.

Cortex M3 Reference Manual – ARM, 2005.

PAJAK, Dominic. ***ARM Cortex-M Processors One architecture from 8-bit applications to 32-bit signal processing*** – ARM.

BUNGO, Joe. ***The ARM Architecture*** – ARM University Program.

www.arm.com

